SERVER USER MANUAL · 사용자 매뉴얼

NeoFlow

NeoFlow Server 사용자 매뉴얼

NeoFlow Studio 기반 ETL 실행 · 스케줄 · 모니터링 통합 플랫폼

항목	내용
문서명	NeoFlow Server 사용자 매뉴얼
문서 버전	v1.0.3
작성일	2026-10-06
작성자 / 제조사	주식회사 위블리스
대상 제품 버전	NeoFlow v1.0 (화면 표시 버전 1.0.0)

© 2026 주식회사 위블리스

개정 이력

버전	일자	작성자	변경 내용	승인
v1.0	2026-07-10	주식회사 위블리스	최초 작성	—
v1.0.1	2026-09-29	주식회사 위블리스	설치·삭제·업그레이드 장, 백업·복구, 보유 기간, 라이선스 관리 등 신설 및 전체 현행화	—
v1.0.2	2026-09-30	주식회사 위블리스	실시간 대기 목록의 우선순위 배지·순서, API 토큰 마지막 사용 기록 주기, DAG Run 진행률 기준·거부된 태스크 사유 표시, 새 DAG 비활성 저장 안내 반영, 화면 그림 갱신	—
v1.0.3	2026-10-06	주식회사 위블리스	CPU 사용률을 최근 15 초 평균으로 측정(대시보드 시스템 상태 카드·알림 임계값·경고 규칙), 성능 화면에 ADMIN 이 아닌 역할이 주소로 들어갈 때의 동작 정정	—

© 2026 주식회사 위블리스

목차

개정 이력 2
목차 3
1. 개요 5
1.1 핵심 가치 5
1.2 이 매뉴얼 보는 법 5
2. 핵심 개념 6
2.1 다섯 가지 핵심 개념 6
2.2 Job 으로 충분한가, DAG 가 필요한가 6
3. 사용자 역할과 권한 7
3.1 권한 매트릭스 (요약) 7
4. 시작하기 8
4.1 로그인 8
4.2 초기 비밀번호 변경 8
4.3 회원가입과 관리자 승인 9
4.4 화면 구성 둘러보기 10
5. 대시보드 11
6. 작업 (Job) 13
6.1 Job 목록 13
6.2 Job 업로드 16
6.3 Job 상세 19
6.4 Job 파라미터 23
7. 스케줄 26
8. DAG 워크플로우 28
8.1 DAG 목록 28
8.2 DAG 생성 29
8.3 DAG 상세와 실행 30
8.4 DAG Run 모니터링 33
9. 데이터 리니지 35
10. 운영 40
10.1 실시간 실행 40
10.2 실행 상세 41
10.3 장애 진단 43
유사 장애 (AI) 44
커스텀 룰 44
10.4 성능 (ADMIN) 45
10.5 SLA 위반 45
10.6 로그 46
10.7 로그인 기록 (ADMIN) 47
10.8 감사 로그 (ADMIN) 48
10.9 운영 리포트 (PDF 인쇄 · 주간 리포트) 50
11. 설정 관리 52
11.1 사용자 관리 (ADMIN) 52
11.2 알림 54
11.3 경고 규칙 56
11.4 설정 58

라이선스 61
공유 라이브러리 (NEO_LIBS) 62
11.5 공통 설정 (Global Properties) 63
11.6 웹훅 (ADMIN) 65
11.7 API 토큰 67
12. 설치·삭제·업그레이드 71
12.1 시스템 요구사항 71
12.2 설치 72
12.3 설치 확인과 초기 로그인 73
12.4 부팅 로그의 보안 경고 74
12.5 업그레이드 74
12.6 삭제 75
12.7 백업 및 복구 76
12.8 고가용성(HA) 운영 77
13. 부록 79
13.1 키보드 단축키 79
13.2 Cron 치트시트 79
13.3 HTTP 상태 코드 80
13.4 오류 메시지와 조치 80
13.5 용어집 81

1. 개요

NeoFlow 는 NeoFlow Studio 로 작성한 ETL Job 을 운영 환경에서 안정적으로 실행·스케줄·모니터링하기 위한 통합 플랫폼입니다. 매일 정해진 시간에 NeoFlow Studio Job 을 실행하고, 실패하면 알림을 받고, 여러 Job 을 묶어 워크플로우(DAG)로 만드는 일을 한 화면에서 처리합니다.

이 매뉴얼은 NeoFlow 의 화면 구성과 주요 기능을 실제 사용 흐름에 따라 서술형으로 안내합니다. 각 장에는 해당 화면의 실제 캡처를 함께 실어, 화면을 보며 따라갈 수 있도록 구성했습니다.

1.1 핵심 가치

NeoFlow 가 제공하는 가치는 다음 여섯 가지 영역으로 요약됩니다.

영역	제공 기능
실행 안정성	HA 에서도 예약분을 한 번만 실행하는 스케줄러, 멈춘 서버의 실행 정리·이어받기, 타임아웃 격리
가시성	실시간 로그 스트림, 실패 원인 자동 진단, SLA 위반 추적, 실행 이력 검색
자동화	DAG 워크플로우, 조건부 트리거, 경고 규칙, Webhook·이메일 알림
운영 효율	Bulk 작업, 변경 이력 비교(Diff), PDF 인쇄, 주간 리포트 메일
보안	가입 승인 게이트, RBAC 역할 기반 권한, 감사 로그, JWT + API Token, Brute-force 잠금, 계정 변경 시 즉시 로그아웃
확장성	HA Active-Active(Nginx LB), 실행 큐 기반 동시성 제어 (Job 은 서버의 OS 프로세스로 실행)

1.2 이 매뉴얼 보는 법

본문에서 메뉴 경로는 [메뉴] > [하위 메뉴] 형태로, 키 입력은 Ctrl + K 형태로 표기합니다. 중요한 내용은 아래 세 가지 안내 상자로 강조합니다.

> **참고** 참고 — 알아두면 도움이 되는 부가 정보입니다.

> **TIP** TIP — 더 효율적으로 사용하는 방법입니다.

> **주의** 주의 — 데이터 손실·오류를 유발할 수 있어 각별히 주의해야 하는 항목입니다.

2. 핵심 개념

NeoFlow 매뉴얼 전반에서 다음 다섯 가지 개념이 반복해서 등장합니다. 이 어휘를 먼저 익혀두면 이후 장을 훨씬 수월하게 읽을 수 있습니다.

2.1 다섯 가지 핵심 개념

Job 은 실행의 기본 단위로, NeoFlow Studio 에서 export 한 .zip 패키지 한 개가 곧 Job 한 개입니다. Schedule 은 그 Job 을 자동으로 실행시키는 cron 규칙이며, Job 하나에는 스케줄을 0 개 또는 1 개만 둘 수 있습니다. Execution 은 Job 또는 DAG Task 가 실제로 한 번 실행된 인스턴스로, 요청되면 REQUESTED(요청됨)로 기록된 뒤 실행 큐에 들어가 QUEUED(대기 중) → RUNNING(실행 중, 일시정지하면 PAUSED) → COMPLETED / FAILED / CANCELLED / TIMEOUT 상태를 거칩니다. 재시도 횟수가 지정된 Job 이 실패하면 남은 횟수만큼 RETRY(재시도 중) 상태로 다시 큐에 들어갑니다. Job 타임아웃을 넘겨 강제 종료된 실행은 FAILED 가 아니라 TIMEOUT 으로 기록됩니다. 상태별 화면 표기와 가능한 조작은 10.2 를 참조하세요.

DAG 는 여러 Job/Task 를 의존성 그래프로 묶은 워크플로우로, Job 하나로 끝나지 않고 여러 단계가 순서대로 또는 조건부로 실행되어야 할 때 사용합니다. 마지막으로 SLA 는 'Job 이 목표 시간 안에 끝나야 한다'는 약속으로, 실행을 요청할 때 그 Job 의 SLA 목표를 실행 기록에 함께 남겨 두고, 완료된 실행의 소요 시간이 그 목표를 넘으면 위반으로 판정합니다. 위반은 대시보드의 SLA 준수율·SLA 위반 패널, SLA 위반 화면(10.5), 주간 리포트에 집계됩니다.

개념	한 줄 설명
Job	실행 단위. NeoFlow Studio .zip 한 개 = Job 한 개 (ACTIVE / INACTIVE)
Schedule	Job 을 자동 실행시키는 cron 규칙 (화면 입력 5 필드, 저장값 6 필드)
Execution	Job/Task 의 1 회 실행 인스턴스. 트리거: MANUAL/SCHEDULED/API/WEBHOOK/DAG
DAG	여러 Job/Task 를 의존성으로 묶은 워크플로우
SLA	목표 소요시간 약속. 위반은 대시보드·SLA 위반 화면·주간 리포트에 집계

2.2 Job 으로 충분한가, DAG 가 필요한가

단일 NeoFlow Studio Job 을 정해진 시각에 실행하는 것으로 충분하다면 Job + Schedule 만으로 끝납니다. 반면 여러 단계가 순서대로 이어지거나 실패 시 후속 처리가 필요하다면 DAG 가 적합합니다.

시나리오	권장
단일 NeoFlow Studio Job 1 회 실행	Job + Schedule
Job 1 종료 후 Job 2 실행 (순차)	DAG (Job 노드 2 개)
Job 실패 시 알림/후속 작업 실행	DAG + 트리거 규칙 '하나 실패'(ONE_FAILED)
5 단계 ETL 파이프라인	DAG

3. 사용자 역할과 권한

NeoFlow 는 RBAC(Role-Based Access Control) 모델로 권한을 관리하며, 한 사용자에게는 하나의 역할이 부여됩니다. 일상 운영에서 쓰이는 핵심 역할은 세 가지이고, 권한은 누적됩니다. 즉 JOB_MANAGER 는 VIEWER 의 모든 권한을 포함하고, ADMIN 은 JOB_MANAGER 의 모든 권한을 포함합니다.

역할	일일 동선	비고
VIEWER	대시보드 → 알림 → 실행 이력 → 로그	조회·모니터링 전담 (실행 권한 없음)
JOB_MANAGER	Jobs → 업로드 → 파라미터 → 실행 → 스케줄 → DAG	VIEWER 권한 + Job·DAG 작성 및 실행
ADMIN	Users → Settings → Audit	JOB_MANAGER 권한 + 시스템 관리

사용자 관리 화면에서 부여하는 역할은 관리자 / 매니저 / 조회자 세 가지이며(사용자 API·CSV 의 저장 코드는 ADMIN / MANAGER / USER), 내부 권한 모델에서는 각각 ADMIN / JOB_MANAGER / VIEWER 로 매핑됩니다. 조회자는 읽기 전용 역할이므로, 회원가입으로 만들어진 계정은 승인을 받아도 조회만 가능합니다. Job 을 실행하거나 등록하려면 관리자가 역할을 매니저 이상으로 올려야 합니다.

3.1 권한 매트릭스 (요약)

주요 작업별로 역할이 무엇을 할 수 있는지 정리하면 다음과 같습니다. 사용자 관리·감사 로그·로그인 기록·웹훅처럼 ADMIN 전용인 메뉴는 권한이 없으면 사이드바에서 숨겨집니다. 그 밖의 화면에서도 실행·중지·업로드·삭제·편집처럼 권한이 없는 동작의 버튼은 보이지 않으며, 켜짐·꺼짐을 함께 보여 주는 스위치(DAG 스케줄 자동 실행, 경고 규칙 켜기/끄기)는 숨기지 않고 비활성으로 표시됩니다. 조회자가 주소로 편집 화면(Job 업로드, DAG 생성·편집)에 직접 들어가면 폼이 열리지 않고 바로 위 화면으로 돌아갑니다.

작업	조회자	매니저	관리자
Job · DAG · 실행 이력 · 로그 · 진단 조회, 목록 내보내기	≣	≣	≣
API 토큰 발급 · 폐기 (본인 토큰)	≣	≣	≣
활성 경고 확인(Ack)	≣	≣	≣
Job 실행, 실행 취소 · 일시정지 · 재개 · 재시도	×	≣	≣
Job 업로드 · 복제 · 삭제 · 패키지 다운로드, 스케줄 · 파라미터 편집	×	≣	≣
DAG 작성 · 수정 · 삭제 · 실행 · 취소	×	≣	≣
SLA 목표 · 카테고리 일괄 변경, 공통 설정 값 편집	×	≣	≣
Job 일괄 실행 · 삭제 · 상태 변경 · 취소	×	×	≣
리니지 자산 · 연결 편집, 가져오기 승인	×	×	≣
경고 규칙 작성 · 해결, Webhook	×	×	≣
사용자 관리 · 감사 로그 · 로그인 기록 · 성능 · Job 변경 이력, 시스템 설정	×	×	≣

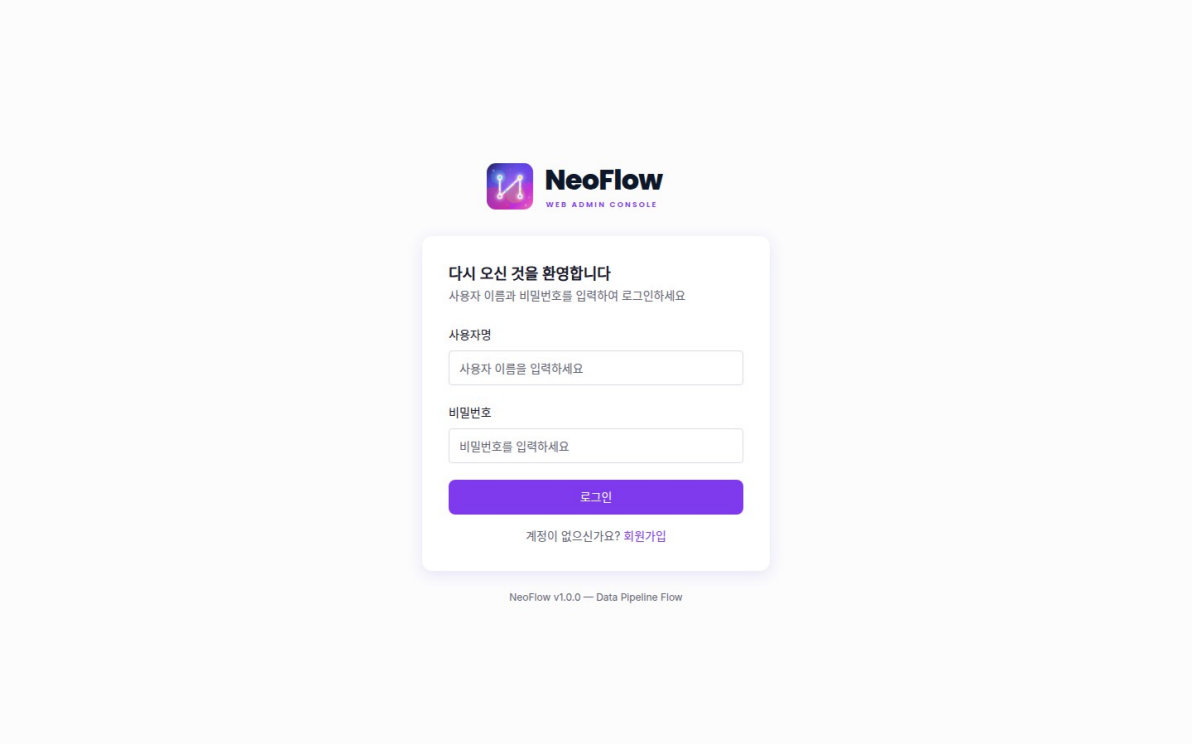
> **참고** 역할·상태 변경은 즉시 반영됩니다. 관리자가 역할을 바꾸거나 비활성·정지·삭제하면 그 사용자의 로그인이 곧바로 끝나고, 다시 로그인하면 새 권한이 적용됩니다. 비활성·정지된 사용자의 API 토큰도 인증되지 않습니다.

4. 시작하기

이 장에서는 NeoFlow 에 처음 접속해 로그인하고, 전체 화면이 어떻게 구성되어 있는지 살펴봅니다.

4.1 로그인

브라우저로 NeoFlow 에 접속하면 로그인 화면이 나타납니다. 사용자명(username)과 비밀번호를 넣고 로그인 버튼을 누르거나 Enter 를 치면 됩니다. 이메일 주소로는 로그인할 수 없습니다. 로그인에 성공하면 JWT access 토큰이 발급되어 자동으로 대시보드로 이동합니다. 로그인 유지 시간은 access 토큰의 수명(기본 24 시간, 관리자가 [설정 > 보안] 탭의 '세션 유지 시간'으로 조정)이며, 사용하지 않은 시간이 아니라 로그인 시각부터 셉니다. 자동 연장은 없으므로 시간이 지나면 다시 로그인 화면으로 돌아옵니다. 관리자가 계정의 역할·상태를 바꾸거나 비밀번호를 재설정해도 즉시 로그인 화면으로 이동합니다.



▲ 로그인 화면 — 사용자명/비밀번호 입력 후 JWT 발급

> **주의** 비밀번호를 연속으로 틀리면(기본 5 회) 계정이 일정 시간(기본 15 분) 잠기며, 잠긴 동안에는 비밀번호가 맞아도 로그인되지 않습니다(423 Locked). 횟수와 시간은 관리자가 [설정 > 보안] 탭에서 정합니다. 잠금 시간이 지나면 저절로 풀리고, 급하면 관리자가 [사용자 관리]에서 비밀번호를 재설정해 잠금을 풀 수 있습니다.

> **참고** 회원가입으로 만든 계정은 관리자가 승인하기 전까지 로그인할 수 없습니다. 비밀번호가 맞더라도 "관리자 승인 대기 중인 계정입니다. 승인 후 로그인할 수 있습니다."라는 안내(403)가 표시되며, 이 거부는 잠금 카운터에 포함되지 않습니다. 자세한 내용은 4.3 회원가입과 관리자 승인을 참고하세요.

4.2 초기 비밀번호 변경

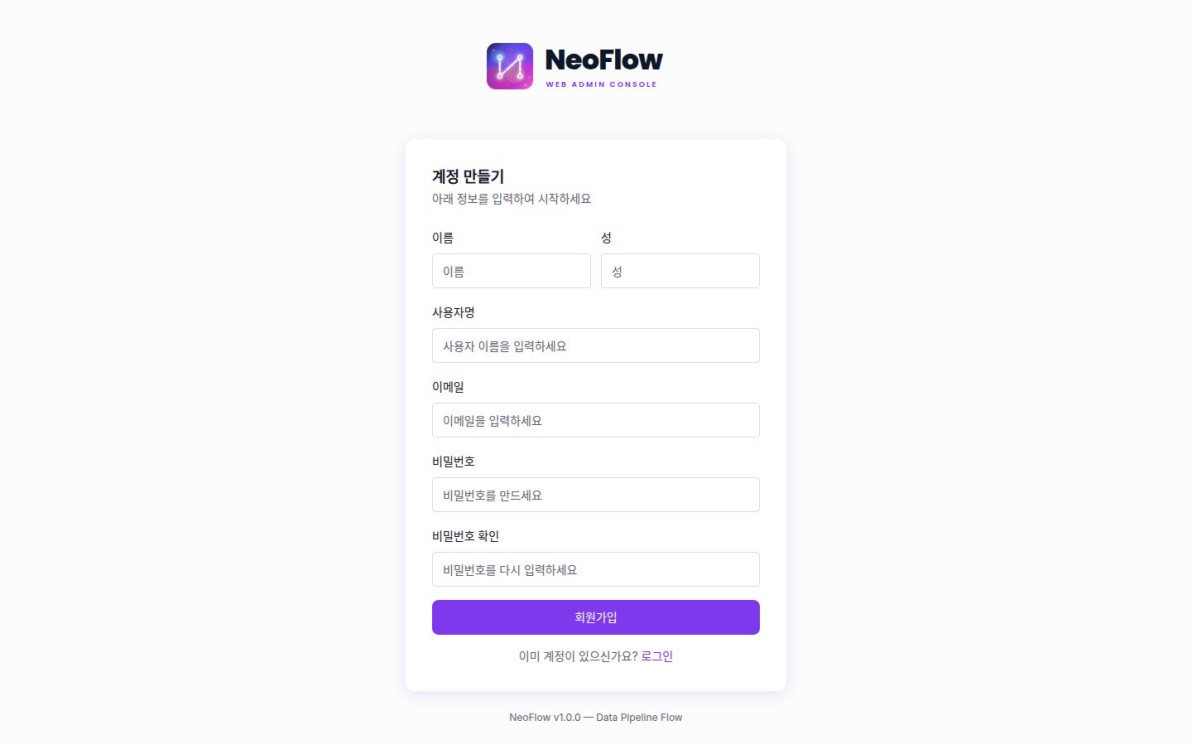
초기 관리자 계정(admin)에 설치 기본 비밀번호로 처음 로그인하거나, 관리자가 만든 계정으로 처음 로그인하거나, 관리자가 비밀번호를 재설정한 뒤 처음 로그인하면 NeoFlow 는 비밀번호를 먼저 바꾸도록 요구합니다. 이때는 다른 화면으로 진입할 수 없고(서버가 모든 요청을 403 으로 차단) '비밀번호 변경 필요' 화면으로 자동 이동합니다. 현재 비밀번호(방금 로그인한 비밀번호)와 새 비밀번호·비밀번호 확인을 입력하며,

새 비밀번호는 최소 8 자에 대문자·숫자·특수문자를 각각 하나 이상 포함해야 하고 현재 비밀번호를 포함한 최근 3 개 비밀번호와 달라야 합니다. 변경에 성공하면 모든 로그인이 끝나고 로그인 화면으로 이동해 "비밀번호가 변경되었습니다. 새 비밀번호로 다시 로그인해주세요."라는 안내가 표시되므로, 새 비밀번호로 다시 로그인합니다.

> **참고** 관리자가 [사용자 관리]에서 비밀번호를 재설정할 때 정한 새 비밀번호는 메일로 발송되지 않으므로 관리자가 직접 전달합니다. 사용자가 자발적으로 비밀번호를 바꿀 때는 [설정 > 계정]에서 변경하며(최근 3 개 비밀번호는 다시 쓸 수 없음), 이때도 이 브라우저뿐 아니라 다른 브라우저·기기의 로그인이 함께 끝납니다. API 토큰은 비밀번호 변경·재설정과 무관하게 유지됩니다.

4.3 회원가입과 관리자 승인

계정이 없는 사용자는 로그인 화면 하단의 [회원가입] 링크로 직접 가입을 신청할 수 있습니다. 이름·성·사용자명·이메일·비밀번호를 입력해 계정을 만들며, 이름·성은 각 50 자 이하, 사용자명은 영문·숫자·밑줄(_)만 쓸 수 있는 3~50 자, 비밀번호는 최소 8 자에 대문자·숫자·특수문자를 각각 하나 이상 포함해야 합니다(서버도 같은 규칙으로 다시 검사하며, 이 정책은 설정으로 바꿀 수 없습니다). 신청이 접수되면 로그인 화면으로 돌아가면서 "가입 신청이 접수되었습니다. 관리자 승인 후 로그인할 수 있습니다."라는 안내가 표시됩니다.



▲ *회원가입 화면 — 신규 계정 가입 신청*

가입만으로는 로그인할 수 없습니다. 회원가입은 인증 없이 접근할 수 있는 공개 화면이므로, 신규 계정은 비활성(INACTIVE) 상태로 만들어지고 관리자가 승인해야 로그인이 열립니다. 신청부터 실제 작업이 가능해지기까지는 다음 세 단계를 거칩니다.

단계	계정 상태	결과
가입 신청 직후	INACTIVE	로그인 시 "관리자 승인 대기 중인 계정입니다" 안내(403)
관리자 승인	ACTIVE	[사용자 관리]에서 [승인]. 로그인 가능해지며 기본 역할은 조회자(읽기 전용)
관리자 역할 변경	변동 없음	매니저 이상이어야 Job 업로드·실행·스케줄 가능

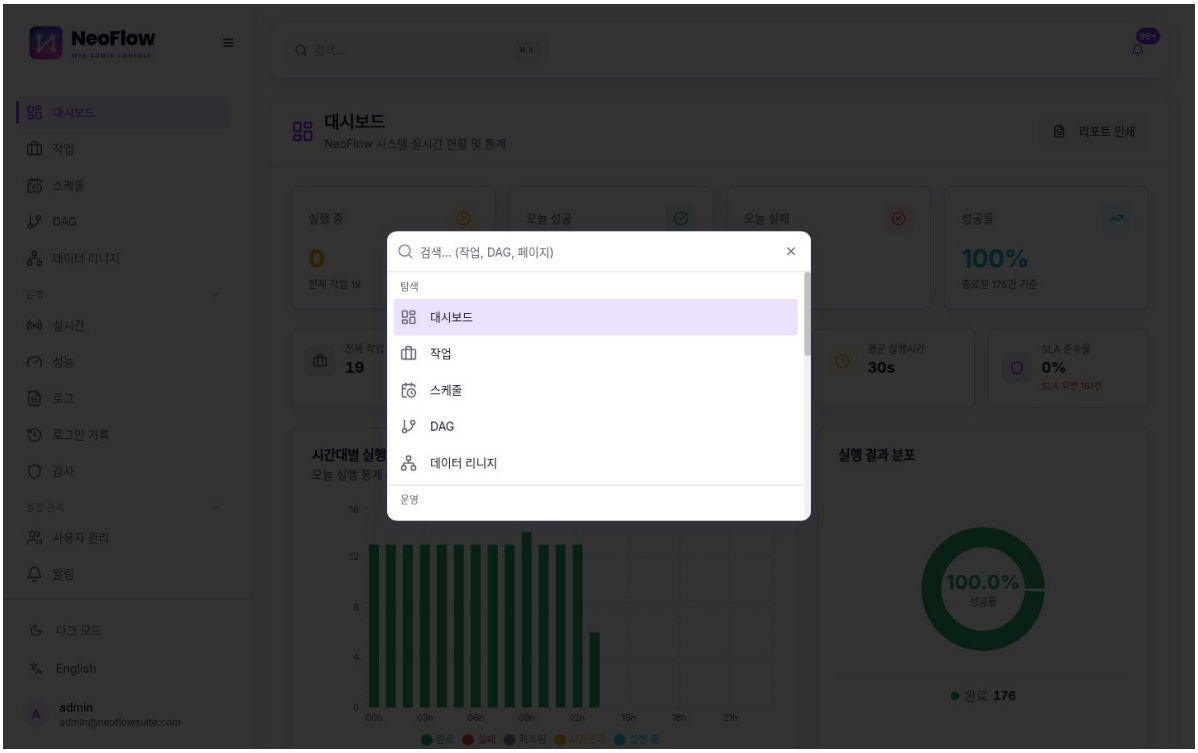
> **참고** 승인 대기 안내는 비밀번호가 맞았을 때만 표시됩니다. 비밀번호가 틀리면 일반 로그인 실패와 똑같이 처리되므로, 해당 사용자명이 존재하는지 여부는 외부에 드러나지 않습니다.

> **TIP** 관리자가 [사용자 관리] > [사용자 생성]으로 직접 만든 계정은 승인 절차 없이 처음부터 활성 상태이며, 대신 첫 로그인 때 비밀번호 변경 화면을 거칩니다. 기존에 이미 활성 상태인 계정은 이번 승인 절차의 영향을 받지 않습니다.

4.4 화면 구성 둘러보기

로그인 이후 모든 페이지는 공통 프레임을 공유합니다. 왼쪽 사이드바는 상단 기본 그룹(대시보드·작업·스케줄·DAG·데이터 리니지)과 접고 펼칠 수 있는 운영·설정 관리 그룹으로 나뉘며, ADMIN 전용 항목은 권한이 없으면 숨겨집니다. 상단 탑바에는 사이드바 토글 햄버거, 검색 버튼, 알림 벨이 있고, 사이드바 하단에는 다크 모드·언어 토글과 사용자 메뉴가 놓입니다. 사용자 메뉴 맨 아래에는 서버 버전이 표시됩니다.

가장 빠른 이동 수단은 명령 팔레트입니다. 어디서든 Ctrl + K(Mac 은 Cmd + K)를 누르거나 탑바의 검색 버튼을 누르면 검색창이 열리고, 페이지 이름과 Job·DAG 이름(2 자 이상)을 검색해 화살표와 Enter 로 바로 이동할 수 있습니다. Job 과 DAG 는 입력한 글자가 들어간 것을 각각 최대 5 건까지 보여 주며, ADMIN 전용 페이지는 권한이 없으면 목록에 나오지 않습니다.



◢ 공통 UI — 대시보드 위에서 Ctrl + K 로 연 명령 팔레트 (사이드바 · 탑바 · 알림 벨)

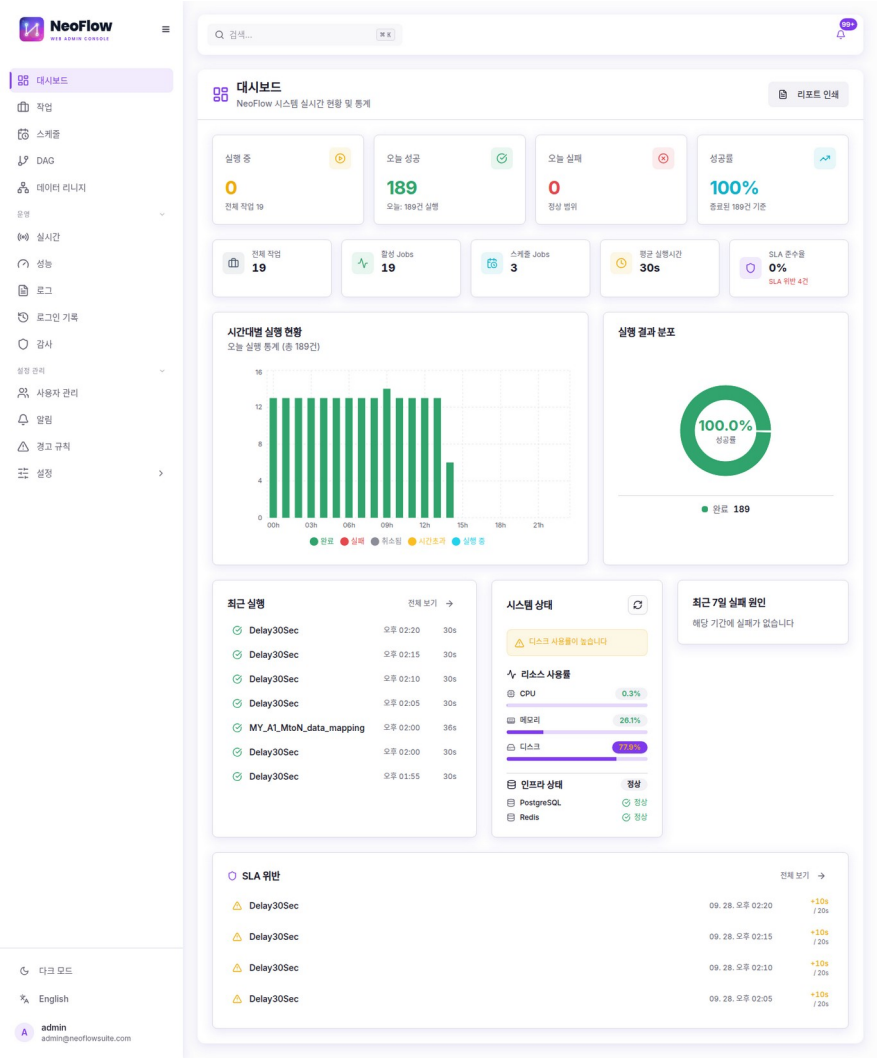
> **TIP** Job 상세의 탭(?tab=history 등)과 알림 화면의 종류 필터(?type=)는 URL 에 남으므로 그대로 북마크하거나 링크로 공유할 수 있습니다. 반면 Job 목록의 필터·정렬·페이지는 URL 에 남지 않으니, 자주 쓰는 조합은 목록의 필터 프리셋('현재 필터 저장')으로 저장하세요. 프리셋은 그 브라우저에만 저장됩니다.

5. 대시보드

대시보드는 NeoFlow 에 진입한 직후 보는 메인 화면으로, 운영자는 보통 여기서 시작해 이상 징후가 보이면 다른 화면으로 이동합니다. 맨 위 KPI 카드 네 장은 실행 중(지금 RUNNING 인 실행 수)·오늘 성공·오늘 실패·성공률이고, 그 아래 줄의 작은 카드 다섯 장은 전체 작업·활성 Jobs·스케줄 Jobs·평균 실행시간(오늘 시작해 완료된 실행의 평균)·SLA 준수율입니다. 오늘 성공·오늘 실패·성공률은 자정 이후에 시작한 실행을 셉니다. 이 카드들은 숫자만 보여 주며, 눌러도 다른 화면으로 이동하지 않습니다.

그 아래에는 오늘 0 시부터 1 시간 단위로 실행을 시작한 시각의 칸에 쌓아 보여 주는 실행 현황 막대그래프와, 오늘 시작해 끝난 실행을 완료·실패·취소됨·시간초과로 나눈 도넛 차트(가운데 성공률)가 나란히 있습니다. 다음 줄에는 시작이 가장 늦은 7 건을 보여 주는 최근 실행 표, CPU·메모리·디스크 사용률과 PostgreSQL·Redis 인프라 상태(외부 의존성)를 보여 주는 시스템 상태 카드(CPU 는 앱 프로세스의 최근 15 초 평균 사용률로, 앱이 시작한 직후 15 초 동안은 0 으로 보입니다), '최근 7 일 실패 원인' 카드가 이어지고, 맨 아래에 최근 SLA 위반을 모은 패널이 있습니다. 실패하거나 늦어진 실행을 살펴보려면 최근 실행 표나 SLA 위반 패널의 행을 눌러 실행 상세로 들어가고, SLA 위반 패널 제목 옆 [전체 보기]로는 SLA 위반 목록 화면에 들어갑니다.

'최근 7 일 실패 원인' 카드는 최근 7 일간의 실패를 장애 진단 카테고리별로 집계해 상위 5 개를 막대로 보여주고, 자동 분류가 되지 않은 미분류(UNKNOWN) 건수를 함께 표시합니다. 미분류 비중이 높다면 어떤 실패 패턴에 진단 룰을 보태야 하는지 알려 주는 신호입니다(10.3 장애 진단 참조).



대시보드 — KPI 카드 · 실행 현황 · 시스템 상태 · SLA 위반

페이지 제목 오른쪽의 [리포트 인쇄] 버튼은 대시보드 지표 8 개를 표로 담은 리포트를 브라우저 인쇄 대화상자로 엽니다(10.9 운영 리포트 참조).

> **참고** 대시보드의 '오늘 실패'는 FAILED 만 셉니다. 타임아웃 실행(TIMEOUT)까지 세는 실시간 실행 화면과 값이 다를 수 있습니다.

6. 작업 (Job)

Job 은 NeoFlow 에서 가장 핵심이 되는 실행 단위입니다. 이 장에서는 Job 목록을 살펴보고, 새 Job 을 업로드하고, Job 상세에서 설정·실행 이력·파라미터를 관리하는 방법을 다룹니다.

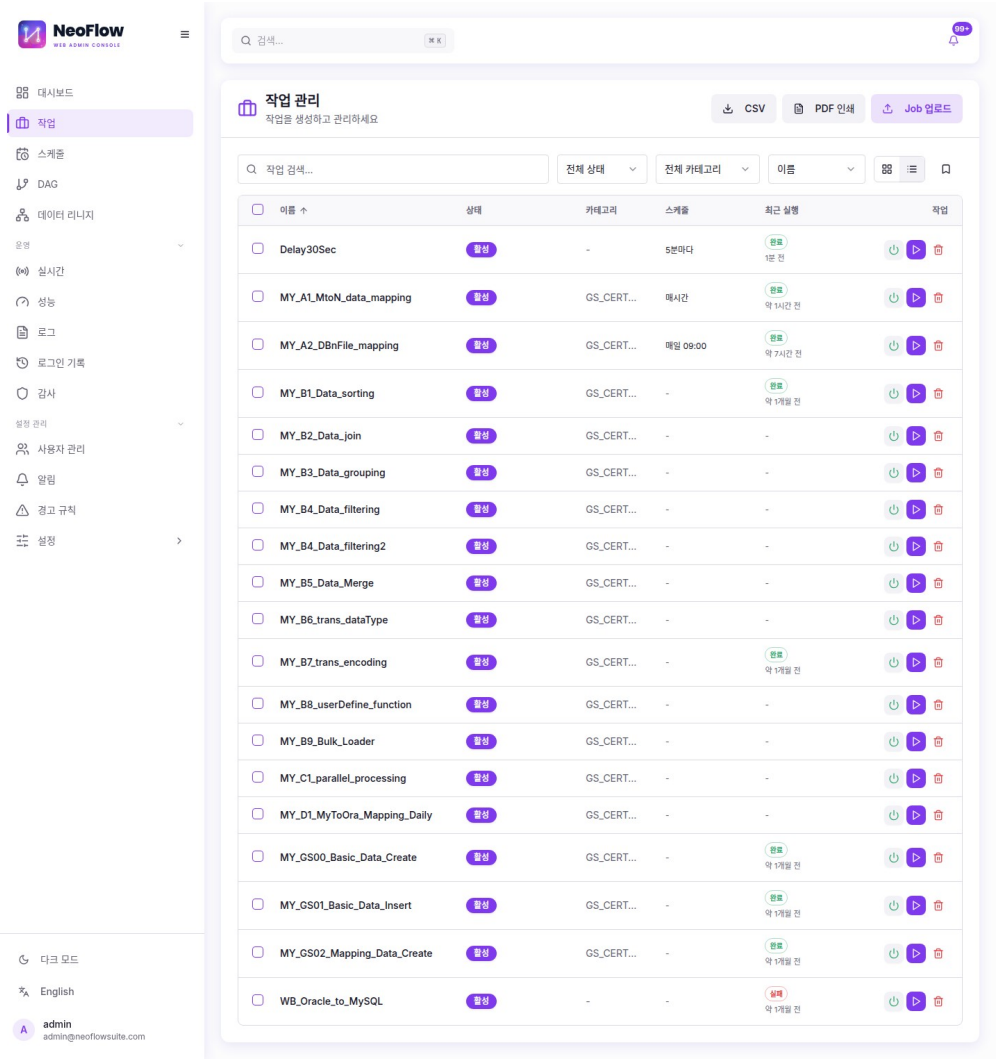
6.1 Job 목록

[작업(Jobs)] 메뉴는 등록된 모든 Job 을 한 테이블에서 보여줍니다. 상단 필터 바에서 '작업 검색'·상태(전체 상태/활성/비활성)·카테고리(하나만 선택)·정렬(최근 수정순/최신순/오래된순/이름)로 목록을 좁히고, 자주 쓰는 조합은 필터 프리셋으로 저장합니다. 테이블 컬럼은 이름(표시명 + 아래 줄에 설명)·상태·카테고리·스케줄·최근 실행·작업(행 버튼)입니다. 스케줄 컬럼은 저장된 cron 대신 '매일 09:00'처럼 말로 풀어 보여주며, 마우스를 올리면 5 필드 cron 이 보입니다. 이름·최근 실행 머리글을 누르면 정렬되고, 최근 실행 정렬은 마지막 실행 시각 순이며 한 번도 실행되지 않은 Job 은 항상 맨 뒤에 놓입니다. 페이지당 20 건으로 고정이며, 행을 클릭하면 Job 상세로 이동합니다.

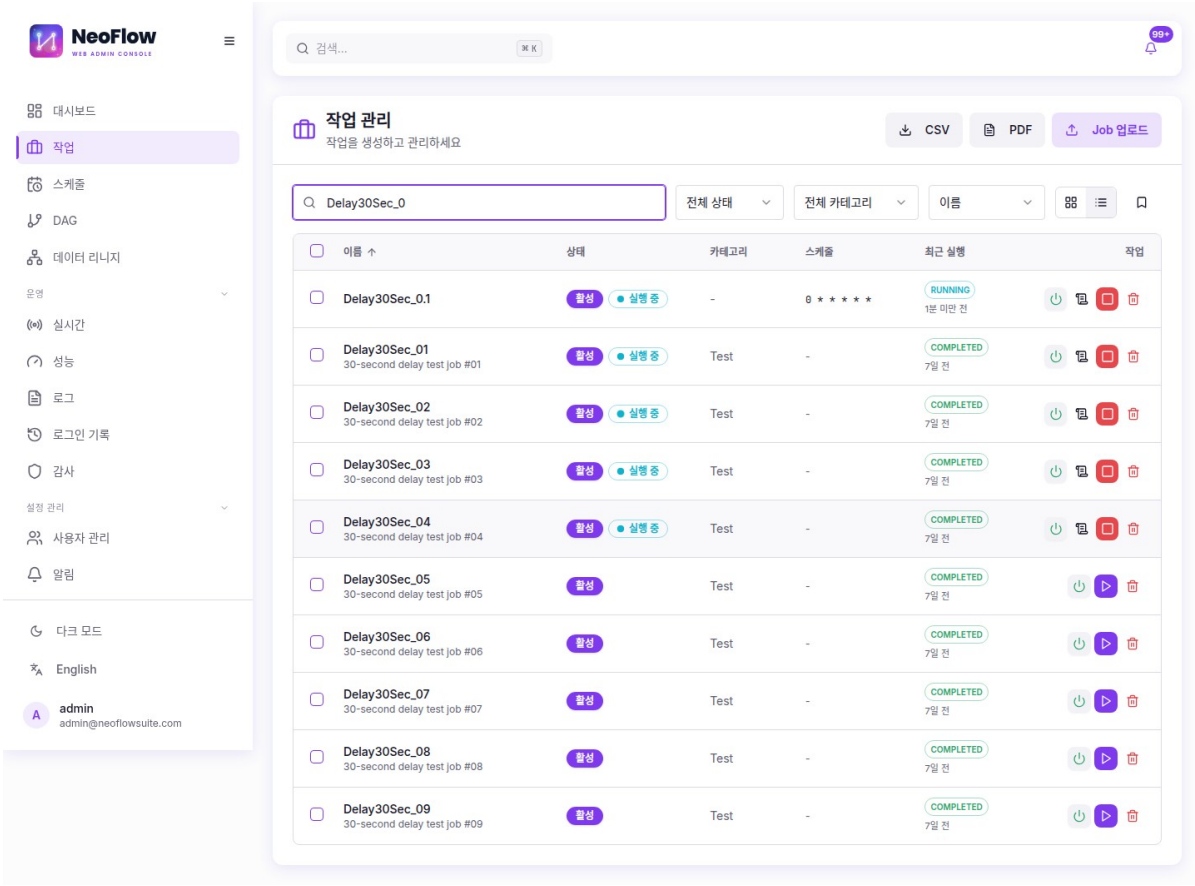
테이블 뷰에서 체크박스로 여러 Job 을 선택하면 Bulk 액션 바가 나타납니다. 선택 실행·선택 취소·선택 활성화·선택 비활성화·선택 삭제는 ADMIN 만, SLA 목표 설정·카테고리 지정은 매니저 이상이 쓸 수 있으며 조회자에게는 액션 바가 나타나지 않습니다. 선택 실행·취소·삭제는 확인 대화상자를 거치고, 선택 비활성화는 고른 Job 중 실행 중인 것이 있을 때만 확인을 묻습니다. 선택 실행은 파라미터 없이 기본값으로 Job 마다 실행을 요청하고, 선택 취소는 실행 중(RUNNING)인 실행만 취소합니다(대기 중인 실행은 행의 [실행 중지]로 취소). 상단 액션 바에서는 목록을 CSV 로 내보내거나 [PDF 인쇄]로 필터에 맞는 Job 전체(최대 10,000 건)를 인쇄하고, 매니저 이상은 [Job 업로드]로 새 Job 을 올립니다. 목록은 테이블 뷰와 카드 뷰로 전환할 수 있고(카드 뷰는 Bulk 액션 미지원), 고른 뷰는 브라우저에 기억됩니다.

목록에서 곧바로 실행할 수도 있습니다. 테이블 행 우측의 [실행] 버튼은 확인·파라미터 입력 없이 정의된 기본값으로 즉시 실행을 요청하고, 카드 뷰의 [실행] 버튼은 '작업 실행' 모달을 먼저 엽니다. 어느 쪽이든 화면이 실행 상세로 넘어가지 않고 목록에 그대로 머물며, 해당 행(카드)에 '대기 중'·'실행 중' 배지만 붙습니다. 실행 요청은 먼저 실행 큐에 등록되므로 큐가 밀려 있으면 대기 상태가 그만큼 유지되고, 대기 중이거나 실행 중일 때는 [실행] 자리에 나타나는 [실행 중지] 버튼으로 취소할 수 있으며 [실시간 로그 보기] 아이콘으로 화면을 옮기지 않고 로그만 열어 볼 수 있습니다. 행의 전원 아이콘은 활성/비활성 전환, 휴지통은 삭제입니다. [실행]은 ACTIVE 상태의 Job 에서만 눌립니다. Job 상세 화면의 [실행]도 ACTIVE Job 에서만 눌리고 그 Job 이 실행 중이면 비활성이며, 누르면 실행 상세 화면으로 이동합니다.

© 2026 주식회사 위블리스



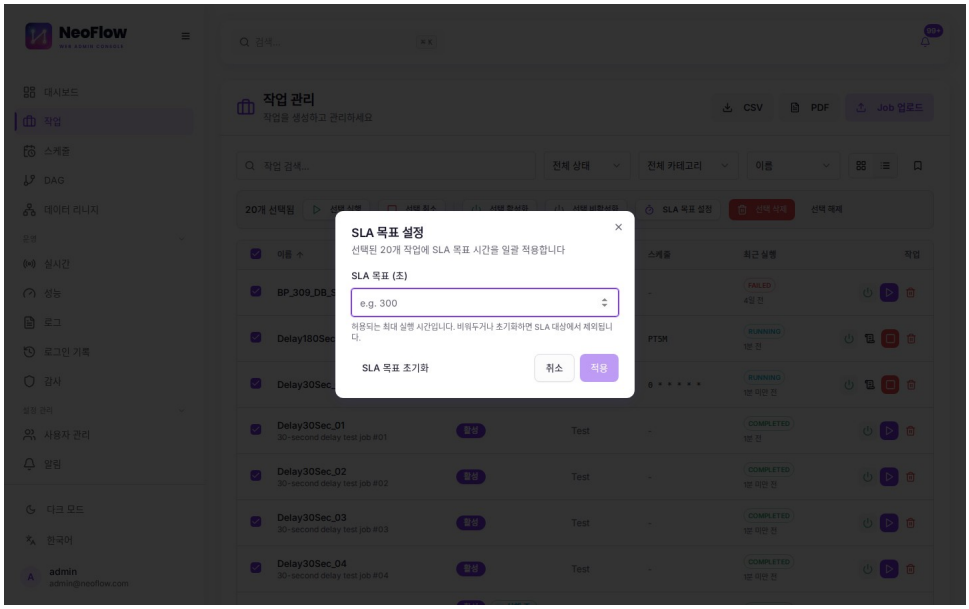
◢ *Job 목록 — 필터 바 · 스케줄 설명 · 행 버튼*



◢ *목록에서 바로 실행 — 실행 중인 행은 상태 배지가 바뀌고 [실행] 자리에 중지·실시간 로그 버튼이 나타납니다*

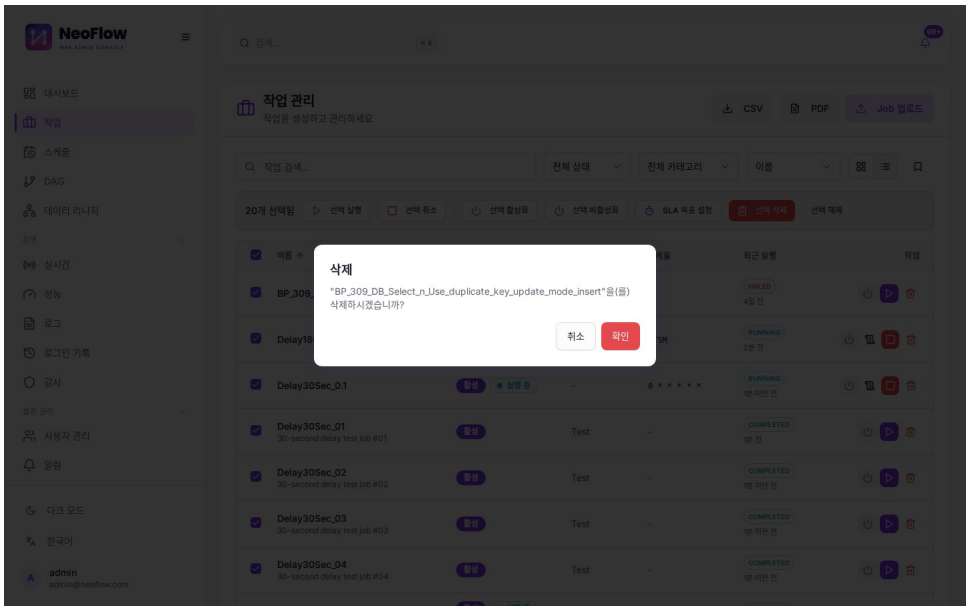
레이어팝업(모달) 체크박스로 여러 Job 을 선택하면 상단 Bulk 액션 바가 열리고, 일부 액션은 확인 모달을 거칩니다.

▸ 대량 SLA 목표 설정



- 여러 Job 을 선택한 뒤 Bulk 액션 바의 [SLA 목표 설정]으로 엽니다.
- 목표(초, 양의 정수)를 입력하면 선택한 모든 Job 에 일괄 적용됩니다(예: 300). 값이 비어 있으면 [적용]이 눌리지 않습니다.
- SLA 를 지우려면 모달의 [SLA 목표 초기화]를 누릅니다. 선택 Job 이 SLA 대상에서 빠집니다.

▸ 대량 삭제 확인

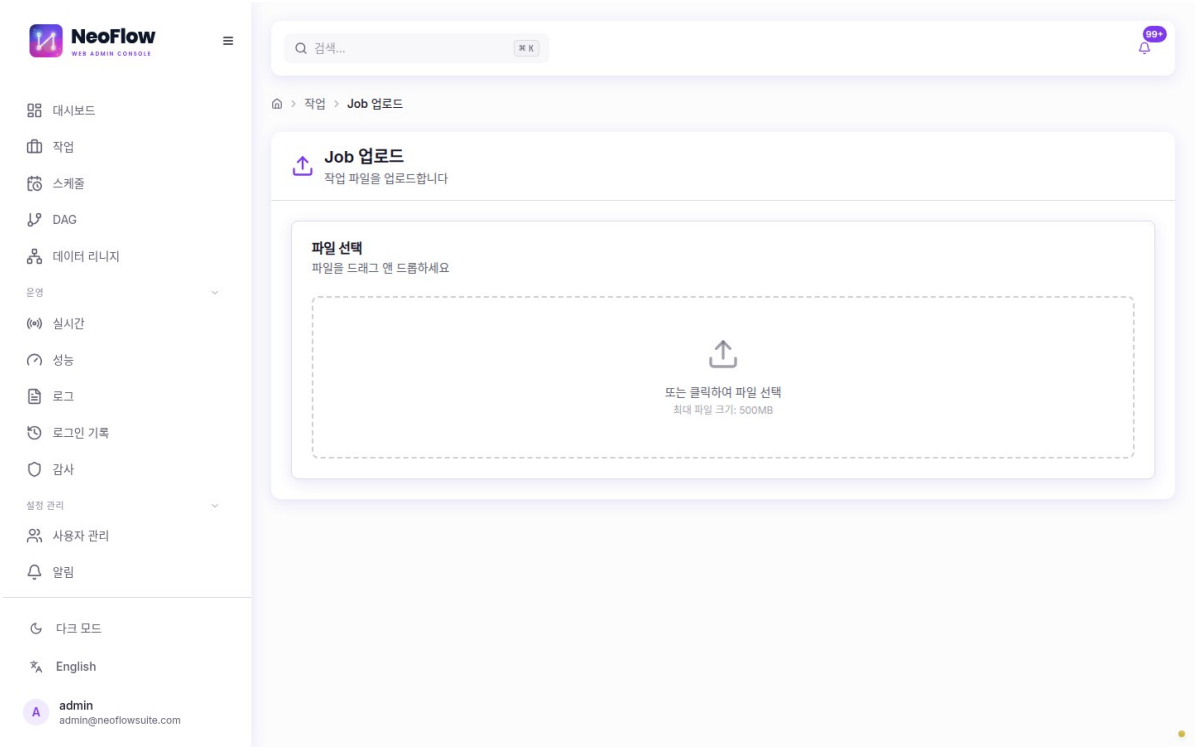


- Bulk 액션 바의 [선택 삭제](ADMIN) 클릭 시 "N 개의 작업을 삭제하시겠습니까?" 확인 모달이 뜹니다.
- 실행 중인 Job 이 섞여 있는지는 알려 주지 않으므로, 삭제 전에 상태 컬럼을 확인하세요.
- 되돌릴 수 없으므로 내용을 확인한 뒤 삭제하세요.

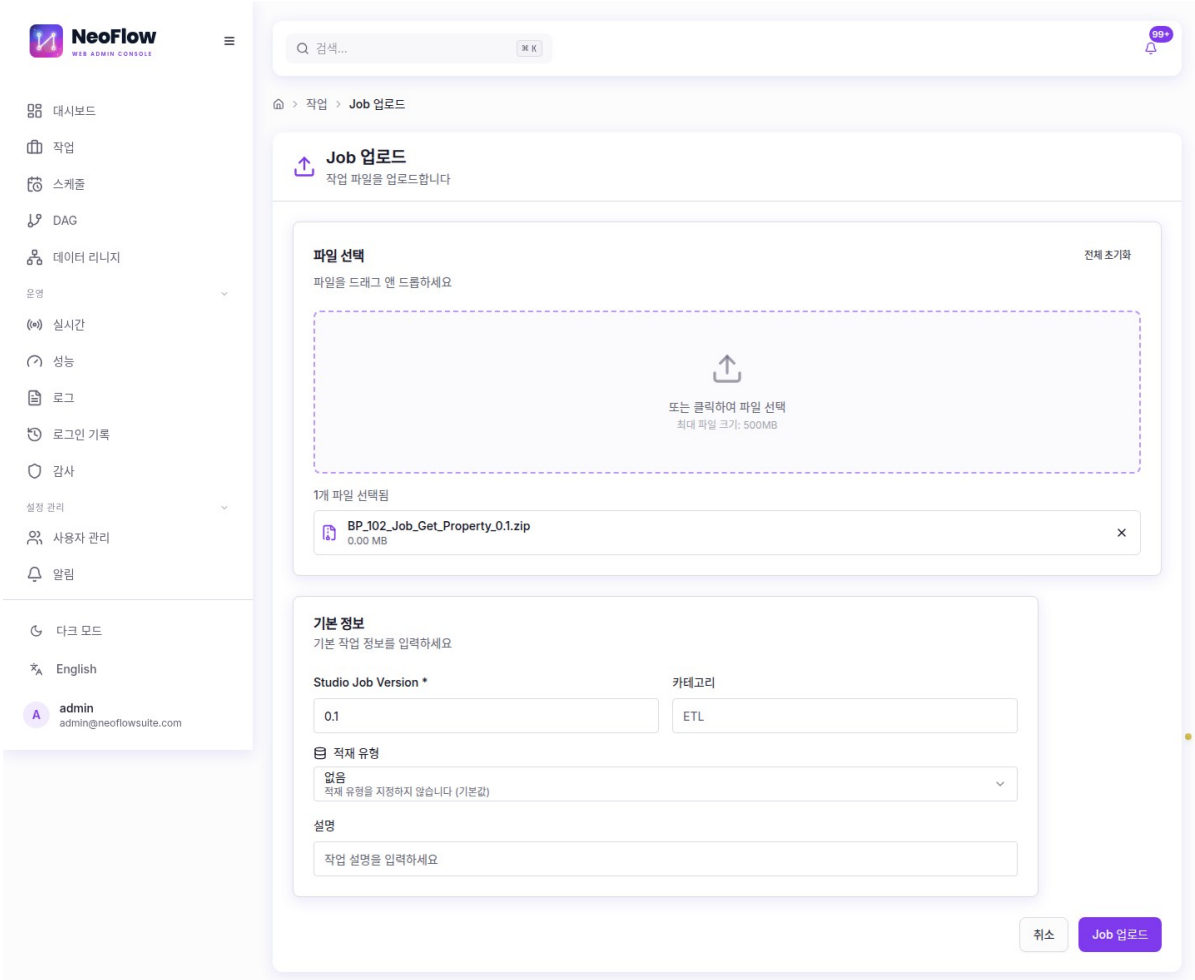
6.2 Job 업로드

새 Job 은 NeoFlow Studio 에서 export 한 패키지를 올려 등록합니다. 매니저(JOB_MANAGER) 이상 권한이 필요하며([Job 업로드] 버튼은 조회자에게 보이지 않음), 절차는 다음과 같습니다.

1. Jobs 목록에서 [Job 업로드]를 클릭합니다.
2. 파일 선택 영역에 .zip 파일을 끌어다 놓거나 클릭해 고릅니다. 파일당 최대 500MB 이며, 여러 파일을 한 번에 고를 수 있습니다. 형식이나 크기가 맞지 않는 파일은 사유와 함께 목록에서 빠집니다.
3. 파일을 고르면 선택한 파일 목록과 기본 정보 카드가 나타납니다. Studio Job Version(필수, 기본 0.1)·카테고리·적재 유형·설명을 입력합니다. 기본 정보는 이번에 고른 모든 파일에 공통으로 적용됩니다.
4. [Job 업로드](여러 개면 [N 개 파일 업로드])를 누르면 파일을 하나씩 순서대로 올립니다. 모두 성공하면 완료 토스트 후 3 초 뒤 Jobs 목록으로 이동하고, 일부가 실패하면 화면에 남아 파일별 실패 사유를 보여줍니다.



◢ *Job 업로드 — 파일 선택 영역*



◢ *파일을 고른 뒤 — 선택한 파일 목록 + 기본 정보(Studio Job Version · 카테고리 · 적재 유형 · 설명)*

화면에는 이름·표시명 입력칸이 없습니다. 파일명에서 .zip 과 끝의 _<버전>을 뗀 값이 표시명이 되고, 표시명에서 영문·숫자·_·- 외의 문자를 -로 바꾼 값이 이름이 됩니다. _<버전>은 점이 하나 이상 있어야 버전으로 인식하며, 파일명에 버전이 있으면 입력한 Studio Job Version 보다 우선합니다.

파일명	이름	표시명	버전
BP_102_Job_0.1.zip	BP_102_Job	BP_102_Job	0.1
Daily Sales_1.2.zip	Daily-Sales	Daily Sales	1.2
job_102.zip	job_102	job_102	입력값

같은 이름의 Job 이 없으면 INACTIVE 상태의 새 Job 으로 등록되고, 이미 있으면 거부되지 않고 그 Job 을 갱신합니다(표시명·설명·카테고리·적재 유형·버전 교체, 새 파일 연결, 상태 유지). 패키지는 오브젝트 저장소의 jobs/{jobId}/{버전}/job.zip 에 보관됩니다.

> **참고** 업로드한 패키지는 SHA-256 체크섬을 기록해 두고, 실행할 때마다 저장소에서 꺼낸 파일을 이 값과 대조합니다. 파일이 업로드 때와 다르면 실행을 시작하지 않고 실패로 끝내며, 실행 상세에 무결성 오류가 표시됩니다(13.4). 또한 패키지 안 컨텍스트 파일에 API 토큰 값(NEOFLOW_API_TOKEN 또는 nf_로 시작하는 값)이 들어 있으면 그 값을 비워서 저장합니다. 서버에서 실행할 때는 서버가 넣어 주는 값을 쓰므로 실행에는 영향이 없습니다.

> **주의** 패키지 .zip 안에 _run.sh 가 없으면 실행되지 않습니다. NeoFlow Studio 에서 standalone(독립 실행) 형식으로 export 했는지 확인하세요. 다시 올릴 때 카테고리·설명을 비워 두면 기존 값이 지워지므로(입력값으로 통째 교체) 유지하려면 다시 입력하세요. 파일명이 한글·공백·기호뿐이면 이름이 비어 업로드가 거부되니, 파일명에 영문이나 숫자를 넣어 export 하세요.

> **TIP** 신규 Job 은 INACTIVE 로 등록되어 스케줄 실행도, 즉시 실행도 되지 않습니다. 먼저 Job 상세 헤더의 활성화 토글이나 Jobs 목록 행의 전원 아이콘으로 ACTIVE 로 바꾸세요. 500MB 상한은 화면·게이트웨이·서버 네 곳에 같은 값으로 고정된 제품 값이라 설정 하나만 바꿔서는 늘어나지 않습니다.

적재 유형(Load Type)은 Job 의 적재 성격을 표시하는 메타데이터로, 목록 필터와 리포트에 활용됩니다. 기본값은 '없음'이며 다음 네 가지 중에서 선택합니다.

적재 유형	의미
없음 (NONE)	미지정 (기본값)
초기 적재 (INITIAL)	최초 전체 적재
증분 적재 (INCREMENTAL)	증분 적재
소급 적재 (BACKFILL)	소급 재적재

> **참고** 번들 라이브러리 자동 등록(NEO_LIBS): 업로드한 .zip 의 lib/ 폴더에 jar 가 들어 있으면 공유 라이브러리 풀(NEO_LIBS)에 자동 등록됩니다. Job 자체 산출물인 routines.jar 는 제외되고, 같은 이름의 jar 가 이미 풀에 있으면 덮어쓰지 않습니다(insert-only). 등록이 실패해도 업로드·배포는 정상 완료됩니다(best-effort). UI 업로드와 NeoFlow Studio 동기화 업로드 양쪽에 적용됩니다.

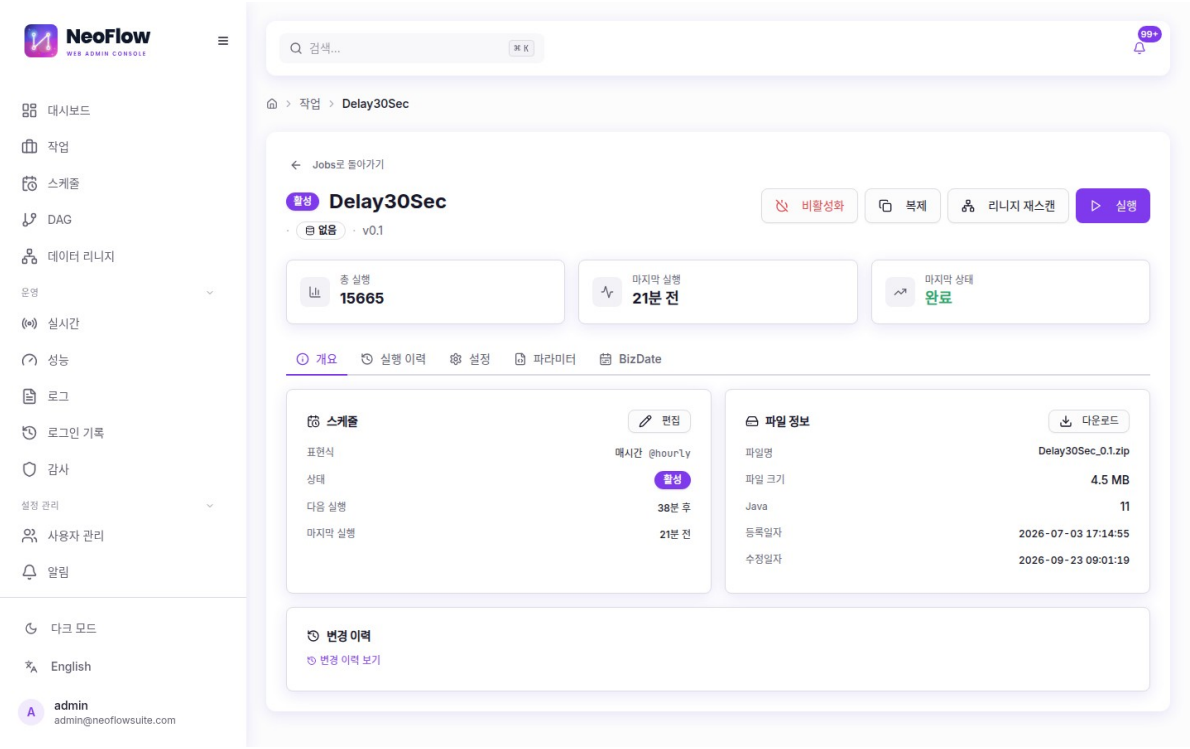
6.3 Job 상세

Job 상세 화면은 헤더(표시명·상태 배지·Studio Job Version·활성 토글·복제·리니지 재스캔(ADMIN)·실행 액션)와 KPI 카드(총 실행·마지막 실행·마지막 상태), 그리고 다섯 개의 탭으로 구성됩니다. 개요 탭은 스케줄 카드, 파일 정보 카드(파일명·크기·Java 버전·등록일자·수정일자 + 다운로드 — 다운로드는 관리자·매니저에게만 표시), 변경 이력 카드로 이뤄집니다. 등록·수정일자는 파일을 업로드·재업로드한 시각입니다. 실행 이력 탭은 이 Job 의 모든 실행을 상태·시작 시간·소요 시간·종료 코드·실행 인스턴스·로그 컬럼으로 나열하며(페이지당 20 건), 상태 필터 하나(전체 상태/완료/실패/실행 중/대기 중/요청됨/취소됨/시간초과/대기/일시정지/재시도 중)로 좁힐 수 있고, 행마다 [로그 다운로드] 버튼으로 실행 로그를 텍스트 파일로 받습니다. 설정 탭에서 편집하는 항목은 SLA 목표(목표 소요 시간, 초) 하나뿐이며, 프리셋(5 분·15 분·30 분·1 시간·2 시간)을 고르거나 초 단위로 직접 입력한 뒤 [저장]합니다(저장 전에 다른 탭으로 이동하면 입력값이 경고 없이 버려집니다). 목표를 바꾸면 그 뒤에 요청되는 실행부터 적용되며, 이미 기록된 실행의 SLA 판정은 바뀌지 않습니다(10.5 참조). 파라미터 탭에서는 이름·필수·타입·기본값·설명을 관리하고, BizDate 탭은 증분 적재 Job 의 업무 기준일을 관리합니다. 선택한 탭은 URL(?tab=)에 남습니다.

다른 항목은 설정 탭에 없습니다. 활성·비활성은 헤더 토글이나 Jobs 목록 행의 전원 아이콘으로, 표시명·설명·카테고리·상태는 Jobs 목록 카드 뷰의 [편집]으로(아래 '작업 편집' 참조), 여러 Job 의 카테고리는 Bulk 액션의 [카테고리 지정]으로, 스케줄과 변경 이력은 개요 탭에서 다룹니다. Job 타임아웃(지정하지 않으면 3600 초)과 재시도 횟수(기본 0)도 화면에서 편집할 수 없으며, Job 업로드 API 의 메타데이터(timeoutSeconds·retryCount)로 등록·재업로드할 때 지정합니다. 이 화면에는 Job 삭제 버튼이 없으므로, 삭제는 Jobs 목록의 행(카드) 버튼으로 합니다.

레이어팝업(모달) ▸ 작업 편집 (Jobs 목록 카드 뷰 [편집], 매니저 이상)

- Jobs 목록을 카드 뷰로 바꾸고 카드 아래쪽의 [편집]을 누르면 '작업 편집' 창이 열립니다. [편집] 버튼은 매니저 이상에게만 보입니다.
- 이름(표시명, 필수)·설명·카테고리·상태(활성/비활성)를 고친 뒤 창 아래의 [편집] 버튼으로 저장합니다. 저장하면 "작업이 수정되었습니다"가 표시됩니다.
- 이 창의 '이름'은 목록과 상세 헤더에 보이는 표시명입니다. 업로드 때 파일명에서 정해진 내부 이름은 바뀌지 않습니다.
- 이름은 2~100 자로 영문·한글·숫자·공백·하이픈·언더스코어만 쓸 수 있고, 설명은 500 자, 카테고리는 50 자까지입니다. 설명을 비우고 저장하면 설명이 지워집니다.
- 같은 이름의 패키지를 다시 업로드하면 표시명·설명·카테고리가 업로드 때 입력한 값으로 다시 바뀝니다(6.2 참조).



◢ *Job 상세 — 개요/실행이력/설정/파라미터/BizDate 탭*

> **참고** 리니지 재스캔(헤더, ADMIN): 이 Job 에 번들된 .item 을 다시 파싱해 데이터 리니지 가져오기 초안을 생성합니다. 실행 후 데이터 리니지 화면의 [가져오기 검토]에서 적용(Apply)하세요. 파서 개선 사항을 기존 Job 에 반영할 때 사용합니다.

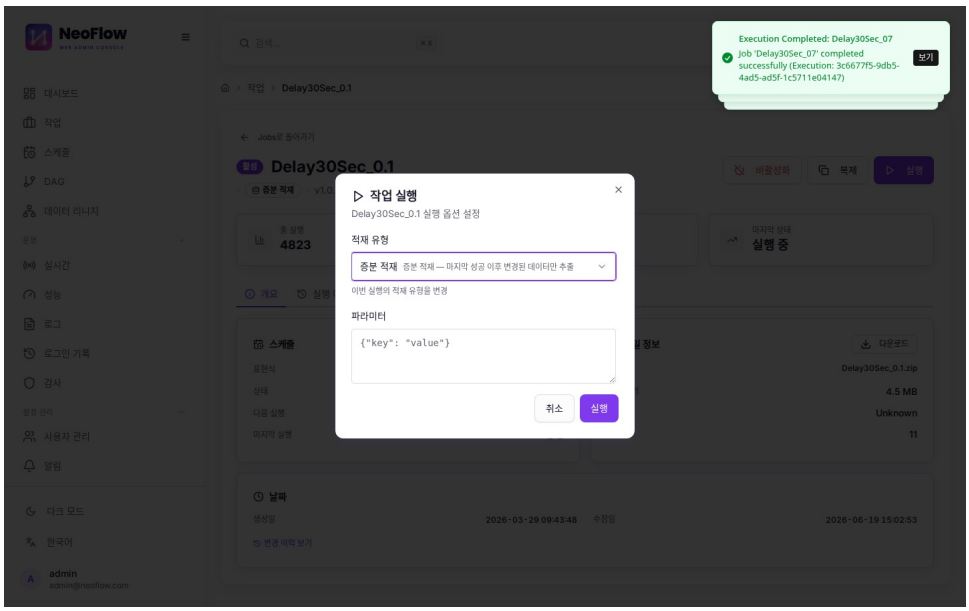
> **주의** 시크릿 평문 노출 차단: 시크릿 체크박스는 따로 없고, 키 이름에 PASSWORD·SECRET·TOKEN·KEY 가 포함된(대소문자 무관) 파라미터가 시크릿입니다. 시크릿 값은 로그와 실행 API 응답에서 ********로 마스킹되고, 실행 이력(job_executions)에는 AES 로 암호화되어 저장됩니다. 파라미터 탭 목록에서도 시크릿 기본값은 ********로 가려지고, 편집 창에서는 빈 비밀번호 입력칸('비워 두면 저장된 값을 씁니다')으로 열립니다. 비워 두고 저장하면 저장된 값이 유지되며, 기본값을 없애려면 파라미터를 삭제하고 다시 만드세요. 운영 배포 시 암호화 키 NEOFLOW_ENCRYPTION_KEY 를 반드시 교체하세요(기본 키는 개발용).

개요 탭 스케줄 카드는 스케줄을 '매일 09:00' 같은 설명과 5 필드 cron 으로 함께 보여주며, 편집(연필) 버튼은 스케줄 편집 모달을 엽니다. 저장하거나 삭제하면 모달이 닫히고, 카드 내용이 길어져도 모달 안에서 세로로 스크롤되므로 버튼이 화면 밖으로 밀리지 않습니다. [스케줄 삭제]는 확인 대화상자를 한 번 거치며, 확인하면 Cron 표현식 자체가 지워집니다. 잠시 멈추는 것이 목적이라면 삭제가 아니라 활성화 토글을 끄십시오(표현식은 그대로 남습니다).

변경 이력 카드(ADMIN 에게만 표시)의 [변경 이력 보기]는 20 건씩 끊어 보여주고 [더 보기]로 다음 장을 불러옵니다(한 번에 최대 100 건). 스케줄러와 실행기가 실행할 때마다 되쓰는 값 — 마지막 실행 시각·다음 실행 시각·마지막 실행 상태·수정 시각·버전 — 은 런타임 기록으로 분류되어 이 목록에서 제외되므로, 남는 것은 사람이 바꾼 항목뿐입니다. 변경 이력이 예상보다 적게 보이는 것은 정상입니다. 기록 자체가 지워지는 것은 아니며, 런타임 기록까지 포함한 전체 추적은 감사 로그 화면(10.8)에서 확인합니다.

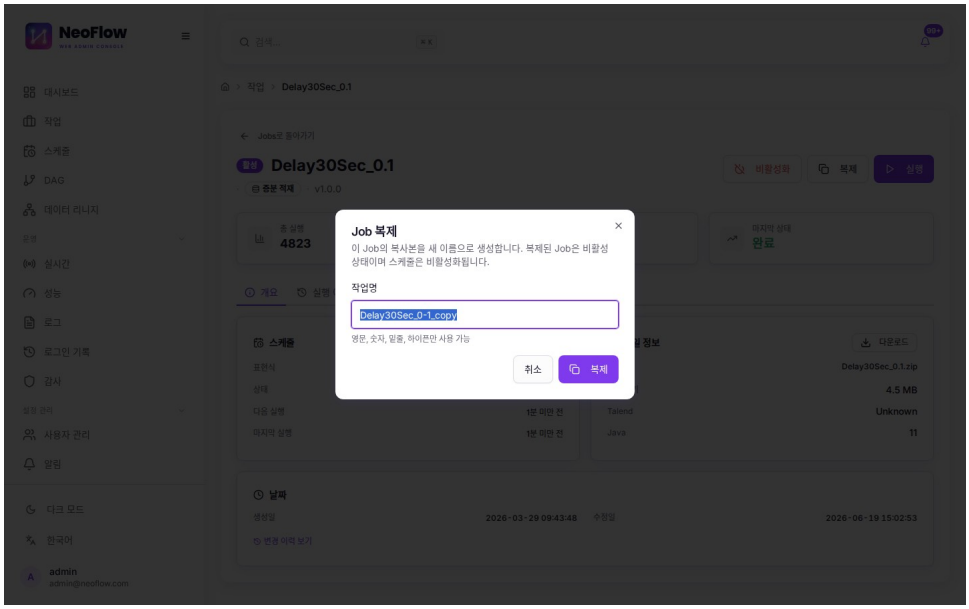
레이어팝업(모달) Job 상세에서는 실행·복제·파라미터·BizDate·변경 이력을 모달로 다룹니다.

▸ 작업 실행



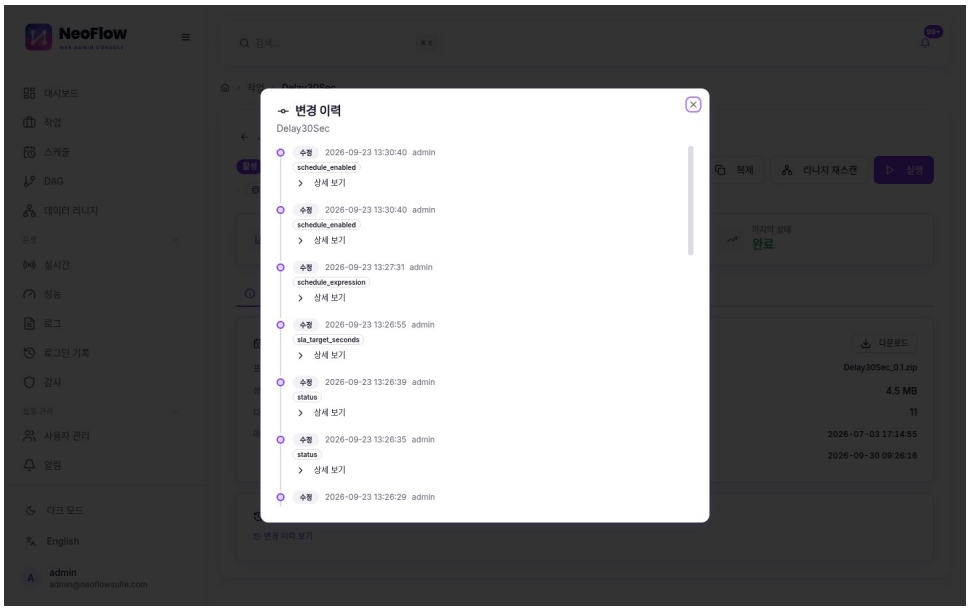
- 헤더 [실행] 버튼으로 엽니다.
- 이번 실행에만 쓸 적재 유형(없음·초기 적재·증분 적재·소급 적재)을 고르며, 소급 적재는 시작·종료일을 입력합니다.
- 파라미터는 정의된 폼(기본값이 채워짐) 또는 JSON 으로 입력합니다. 필수 항목이 비어 있으면 [실행]이 눌리지 않습니다.
- 시크릿 파라미터는 기본값을 채우지 않은 비밀번호 입력칸으로 나오며, 비워 두면 저장된 값으로 실행합니다.
- [실행] 시 즉시 큐에 등록된 뒤 실행 상세로 이동합니다.

▸ 작업 복제



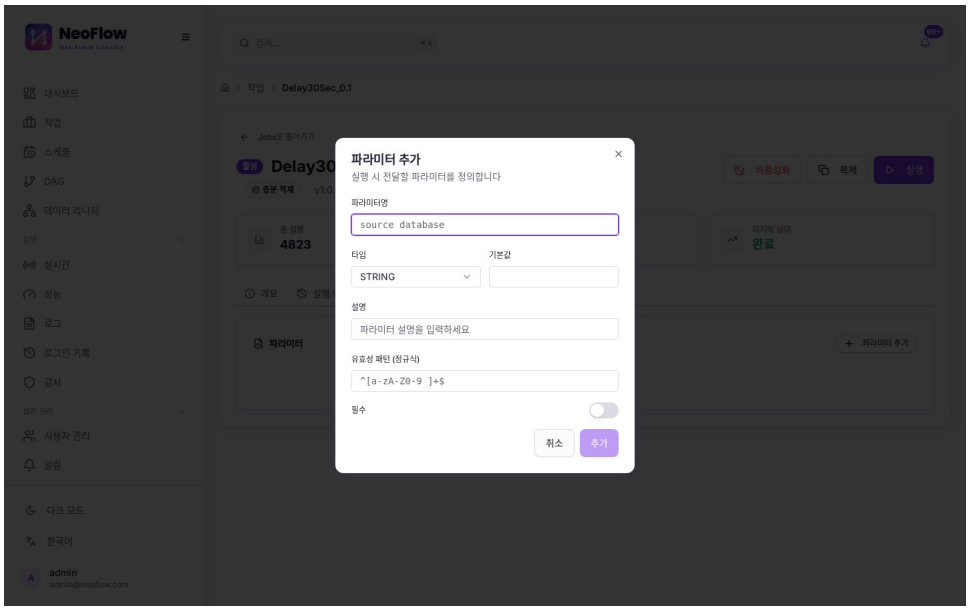
- 헤더 [복제] 버튼으로 엽니다.
- 새 작업 이름을 정하면(기본값 원본_copy) 메타·파라미터까지 복제되어 복제본 상세로 이동합니다.

▸ 변경 이력



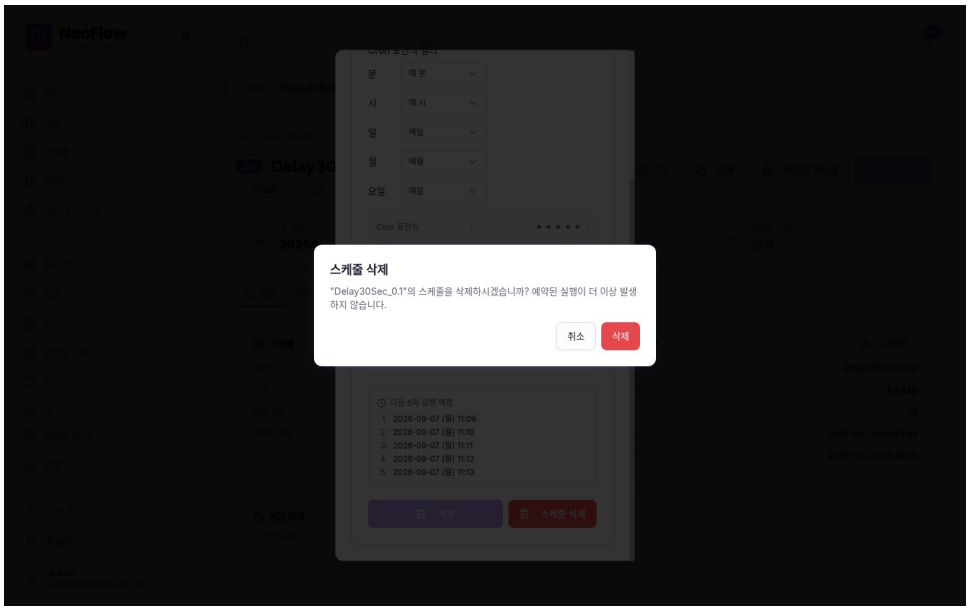
- 개요 탭의 [변경 이력 보기] 링크로 엽니다. 변경 이력 카드와 이 창은 ADMIN 에게만 표시됩니다.
- 시간순 타임라인에 작업 배지(추가/수정/삭제)·시각·변경자·변경 필드가 표시됩니다.
- 각 항목을 펼치면 이전/이후 값을 비교할 수 있습니다.
- 20 건씩 끊어 보여주며 [더 보기]로 다음 장을 불러옵니다(한 번에 최대 100 건).
- 실행 시각 되쓰기 같은 런타임 기록은 제외되어 사람이 바꾼 항목만 남습니다.

▸ 파라미터 추가·편집



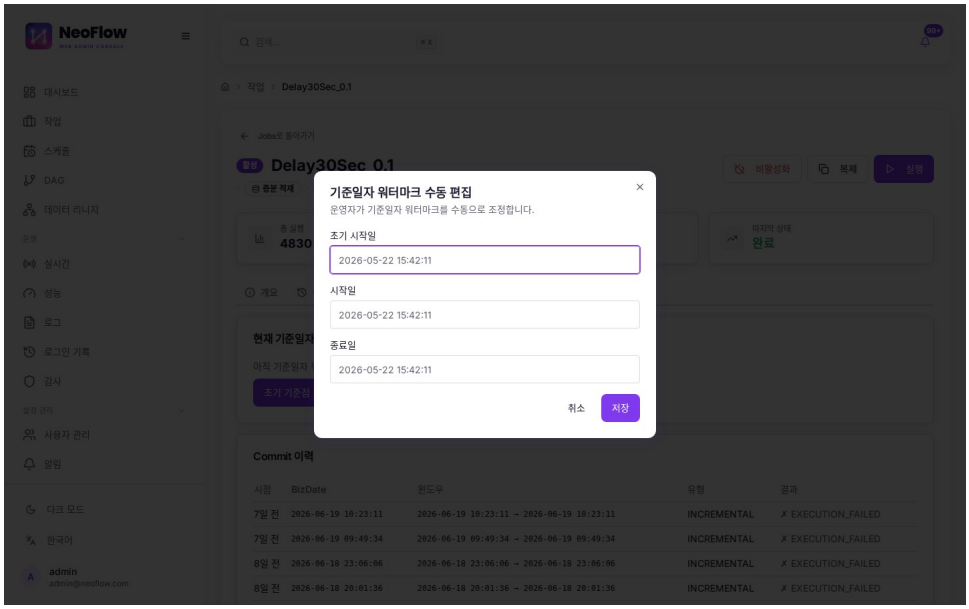
- 파라미터 탭의 [파라미터 추가] 또는 행 편집으로 엽니다.
- 이름·타입(STRING/INTEGER/BOOLEAN/DATE/DECIMAL/JSON)·기본값·필수·설명을 정의합니다.
- 여기서 정한 정의로 실행 시 입력 폼이 자동 생성됩니다.
- 이름에 PASSWORD·SECRET·TOKEN·KEY 가 들어간 파라미터는 시크릿이며, 편집 창의 기본값이 빈 비밀번호 입력칸으로 열립니다. 비워 두고 저장하면 저장된 값이 유지됩니다.

▸ 스케줄 삭제 확인



- 스케줄 편집 모달의 [스케줄 삭제] 버튼으로 열리며, 편집 모달 위에 겹쳐 뜹니다.
- 확인하면 Cron 표현식까지 지워져 예약된 실행이 더 이상 발생하지 않습니다.
- 잠시 멈추는 것이 목적이면 삭제 대신 활성화 토글을 끄세요(표현식 보존).

▸ BizDate 워터마크 편집



- BizDate 탭에서 초기 기준점을 설정·편집합니다(ADMIN·JOB_MANAGER).
- 초기 시작일·현재 시작일·종료일(yyyy-MM-dd HH:mm:ss)을 지정합니다.
- 필드를 비우면 해당 워터마크가 제거되며, 증분 적재의 다음 실행 범위를 결정합니다.

6.4 Job 파라미터

파라미터는 같은 Job 을 날짜·대상 시스템만 바꿔 재사용하기 위한 장치입니다. Job 상세의 파라미터 탭에서 이름·타입·기본값·필수 여부·설명을 정의하며(이름에 PASSWORD·SECRET·TOKEN·KEY 가 들어가면 시크릿), 저장하면 다음 실행부터 적용됩니다. 이 절에서는 여러 곳에 흩어진 값 중 무엇이 실제로 쓰이는지, 그리고 그 값이 Job 안으로 어떻게 들어가는지를 정리합니다.

한 파라미터에 여러 출처의 값이 있으면 다음 순서로 결정됩니다. 위쪽이 우선입니다.

순위	출처	설정 위치
1	실행 시 입력값	즉시 실행 모달 · 실행 API 요청 본문 · DAG 수동 실행의 Config Override
2	Job 파라미터 기본값	Job 상세 > 파라미터 탭
3	공통 설정 (Global Properties)	설정 관리 > 공통 설정 (프로파일별)

> **참고** 공통 설정이 Job 기본값을 덮어쓴다고 오해하기 쉬우나 반대입니다. 같은 키가 양쪽에 있으면 Job 파라미터 기본값이 이깁니다. 공통 설정은 Job 이 값을 따로 지정하지 않았을 때 쓰이는 환경별 기본값 역할입니다.

이렇게 확정된 값은 두 경로로 Job 프로세스에 전달됩니다.

경로	형식	전달 대상
실행 인자	--context_param <키>=<값>	비-시크릿 파라미터 (공통 설정의 비밀 속성도 제외)
환경변수	PARAM_<대문자 키>=<값>	전체 파라미터 (시크릿 포함)

시크릿을 실행 인자로 넘기지 않는 이유는 커맨드라인이 같은 호스트의 다른 프로세스에서 /proc/<pid>/cmdline 으로 읽히기 때문입니다. 시크릿은 환경변수 경로로만 전달됩니다.

공통 설정에서 비밀로 지정한 키도 키 이름과 관계없이 시크릿으로 다룹니다. 실행 인자로 넘기지 않고, 실행 로그와 실행 API 응답에서 ********로 가립니다. 따라서 이런 키(예: DB_CONN_STR)를 context 변수로 쓰는 Job 은 아래 NeoFlowContext routine 이나 System.getenv 로 받아야 합니다(11.5 참조).

Job 쪽에서 이 값을 받는 권장 방법은 NeoFlow Studio 에 시스템 routine 으로 포함된 NeoFlowContext 를 쓰는 것입니다. Job 의 tPrejob 다음 tJava 에 아래 한 줄을 둡니다.

```
NeoFlowContext.apply(context);
```

그 뒤로는 context.TARGET_DATE 처럼 평소대로 쓰면 되고, 파라미터가 늘어나도 Job 을 고칠 필요가 없습니다. routine 은 PARAM_* 환경변수를 같은 이름의 context 변수에 대소문자를 무시하고 매핑하되, NeoFlow 가 실제로 주입한 변수만 덮어씁니다. NeoFlow Studio 에서 로컬로 실행할 때는 PARAM_*가 없으므로 Context 탭 기본값이 그대로 유지되어, 로컬용·서버용 분기 코드를 둘 필요가 없습니다. 시크릿 파라미터도 이 경로로 도달합니다.

파라미터 키와 Job 의 context 변수명이 어떻게 맞물리는지는 다음과 같습니다.

상황	결과
키와 context 변수명이 정확히 일치	실행 인자가 직접 바인딩 (routine 없이도 동작)
대소문자만 다름 (gsOutputDir ↔ GS_OUTPUT_DIR)	routine 이 대소문자 무시 매칭으로 처리
Date 타입 context 변수	routine 은 건드리지 않고 실행 인자에 위임 (포맷 패턴을 알 수 없음)

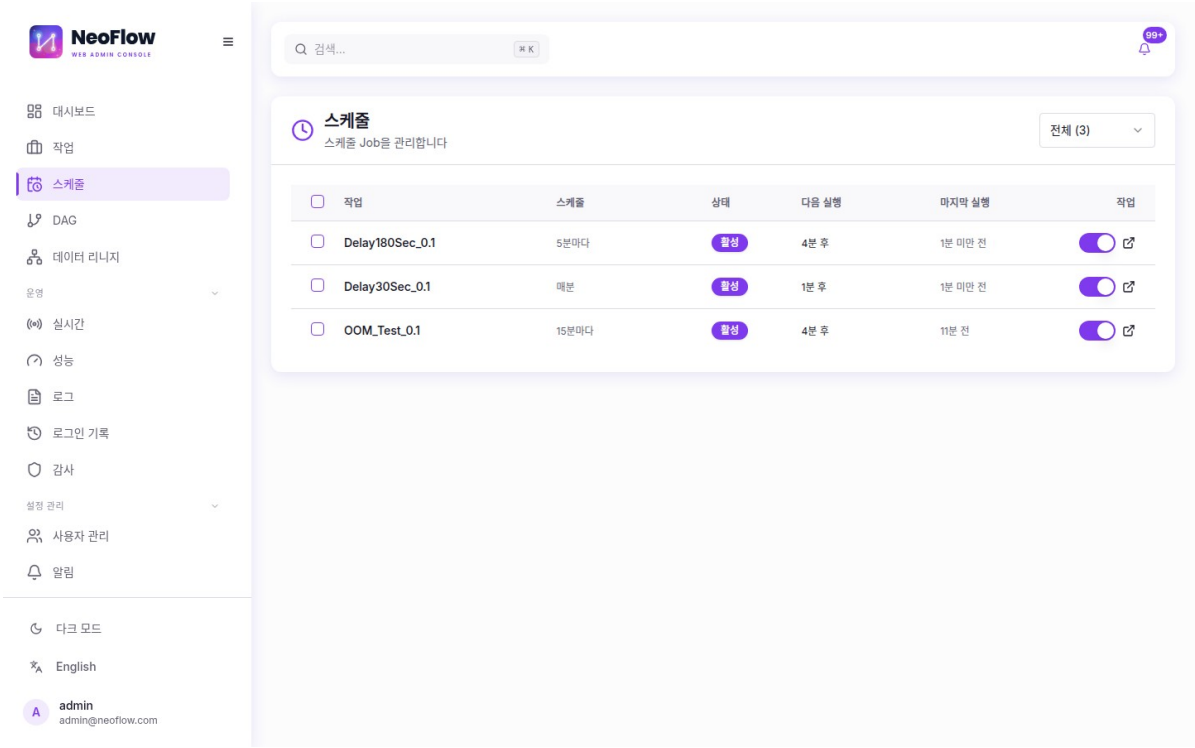
주의 이름 자체가 다르면 어느 경로로도 값이 전달되지 않습니다. routine 이 흡수해 주는 것은 대소문자 차이뿐입니다. 또한 파라미터 값을 System.out.println 으로 찍지 마세요. Job 의 표준출력은 실행 로그로 그대로 적재되며, 이 경로에는 서버측 시크릿 마스킹이 적용되지 않습니다.

참고 NeoFlowContext 는 NeoFlow Studio 에 시스템 routine 으로 포함되어 있어 따로 등록하지 않습니다. 같은 이름의 사용자 routine 을 만들지 마세요. routine 을 쓰지 않는 Job 은 System.getenv("PARAM_TARGET_DATE") 처럼 환경변수를 직접 읽어도 됩니다. 실행 인자 주입은 서버 설정 execution.context-param.enabled(기본 true)로 끌 수 있고, 꺼도 환경변수 경로는 유지되므로 routine 을 쓰는 Job 은 영향을 받지 않습니다.

© 2026 주식회사 위블리스

7. 스케줄

[스케줄] 화면은 스케줄이 걸린 모든 Job 을 한 테이블에서 보여주고, 각 행에서 즉시 ON/OFF 할 수 있게 합니다. 스케줄 컬럼은 Cron 표현식을 '5 분마다', '매시간'처럼 사람이 읽기 쉬운 문구로 풀어 표시하고, 다음 실행은 '4 분 후'와 같은 상대시간으로 안내합니다. 우상단 상태 필터는 전체 / 활성 / 비활성 세 가지이며 괄호 안에 각 상태의 건수가 표시됩니다. 활성화 토글을 끄면 스케줄이 비활성 상태가 되어 다음 Cron 시각이 와도 실행되지 않고, 다시 켜면 켠 시점부터 다음 실행 시각을 다시 계산합니다. 토글을 꺼도 Cron 표현식은 그대로 보존됩니다.



◢ *스케줄 — Cron 사람친화 표기 + 활성화 토글*

스케줄 편집 화면의 Cron 빌더와 입력칸은 Linux cron 과 같은 5 필드(분 시 일 월 요일)를 받고, Job 은 저장할 때 앞에 초 0 을 붙인 6 필드(초 분 시 일 월 요일)로 저장합니다. 그래서 개요 탭의 스케줄 카드와 API 응답에는 6 필드로 보이며, REST API 로 직접 넣을 때는 6 필드를 써야 합니다. 프리셋은 없음 / 매시간 / 매일 00:00 / 매일 09:00 / 매주 월요일 09:00 / 매월 1 일 00:00 / 5·15·30 분마다 / 커스텀입니다. 자주 쓰는 표현은 부록의 Cron 치트시트를 참고하세요.

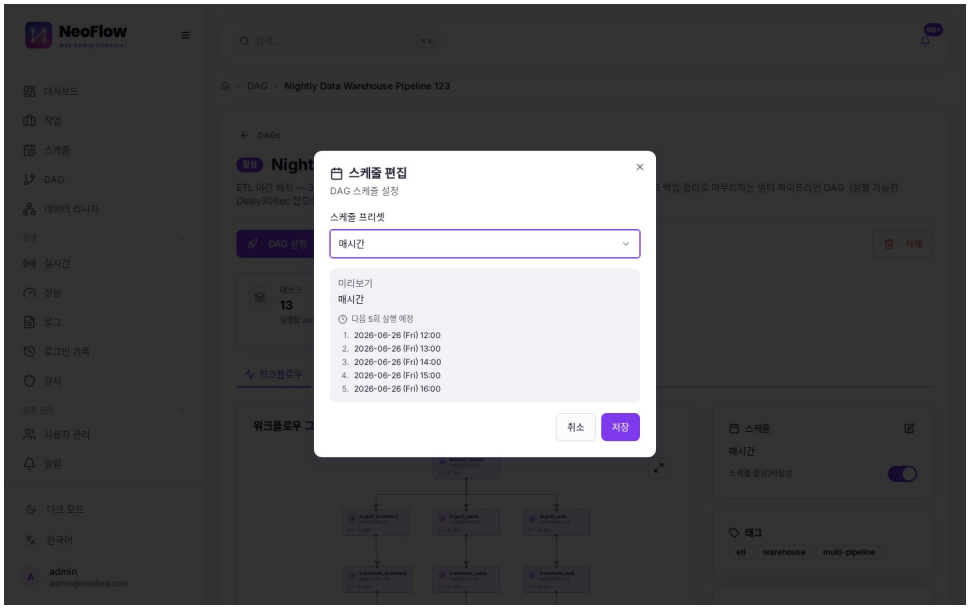
서버는 1 분마다 실행할 Job 을 확인하므로, 예약 시각이 지나고 최대 약 1 분 늦게 시작할 수 있습니다. HA(Active-Active) 구성에서는 모든 앱 노드가 각자 확인하지만 예약분을 먼저 선점한 노드만 실행하므로 같은 예약분이 두 번 실행되지 않습니다. 스케줄은 앱 서버 시간대(배포본은 Asia/Seoul 고정) 기준입니다.

> **참고** 비활성인 동안 지나간 실행은 catch-up(소급 실행)하지 않습니다. 비활성인 동안에는 다음 실행 시각이 갱신되지 않아 지난 시각이 보일 수 있으며, 다시 켜면 그 시점 이후의 첫 예약 시각부터 실행됩니다. 예약 실행이 거부된 경우(비활성 Job, 라이선스 한도 등)에도 따라잡지 않고 다음 예약 시각을 기다립니다.

> **TIP** 스케줄 상태는 활성 / 비활성 두 가지뿐이며 '일시정지' 상태는 없습니다. 잠시 멈출 때는 활성화 토글을 끄고(Cron 표현식 보존), 스케줄 자체를 없앨 때는 Job 상세 → 스케줄 카드의 [스케줄 삭제]를 씁니다. 삭제는 확인 대화상자를 거치며 Cron 표현식까지 지우므로, 되돌리려면 표현식을 다시 입력해야 합니다. '일시정지'는 실행(Execution)과 실시간 로그 스트리밍처럼 실제로 멈췄다 이어지는 대상에만 쓰는 표현입니다.

레이어팝업(모달) 스케줄은 전용 편집 모달에서 프리셋 또는 Cron 빌더로 설정합니다.

▸ 스케줄 편집



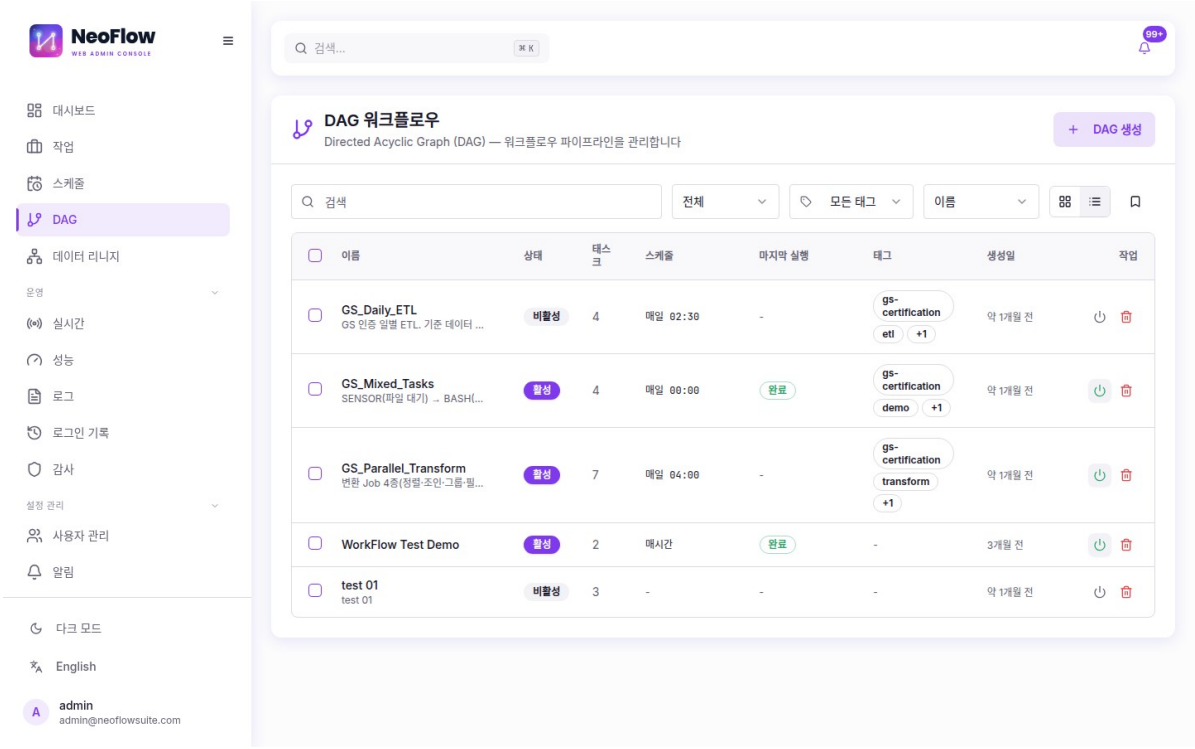
- 스케줄 카드의 편집(연필) 버튼으로 엽니다.
- 프리셋(없음/매시간/매일 00:00/매일 09:00/매주 월요일 09:00/매월 1 일 00:00/5·15·30 분마다/커스텀)을 고르고, 커스텀은 Cron 빌더(분·시·일·월·요일)나 직접 입력칸으로 구성합니다.
- 빌더와 입력칸은 초 없는 5 필드 cron 이며, Job 은 저장할 때 앞에 초 0 을 붙여 6 필드로 저장합니다.
- 미리보기에서 해석된 Cron 설명과 다음 5 회 실행 시각을 확인한 뒤 저장합니다. 미리보기는 서버와 같은 방식으로 계산합니다.
- 저장·삭제가 끝나면 모달이 닫히고, 내용이 길면 모달 안에서 세로로 스크롤됩니다.
- [스케줄 삭제]는 확인 대화상자를 거치며 Cron 표현식까지 지웁니다(토글 OFF 는 표현식 보존).

8. DAG 워크플로우

DAG(Directed Acyclic Graph)는 여러 Job/Task 를 의존성으로 연결한 워크플로우입니다. 이 장에서는 DAG 목록에서 시작해 새 DAG 를 만들고, 상세에서 실행하고, DAG Run 의 진행 상황을 모니터링하는 흐름을 다룹니다.

8.1 DAG 목록

[DAG] 메뉴는 등록된 워크플로우를 검색·필터링하고 활성/비활성을 토글하거나 상세로 진입하는 화면입니다. 테이블에는 이름·상태·태스크 수·스케줄·마지막 실행 배지·태그·생성일이 표시되며, 태그는 다중 선택 시 AND 조건으로 필터링됩니다. 우상단의 [+ DAG 생성] 버튼은 JOB_MANAGER 이상에게만 보이며, 우측 끝에는 그리드/리스트 뷰 토글과 자주 쓰는 필터를 저장하는 북마크 아이콘이 있습니다.



◢ *DAG 목록 — 검색 · 상태/태그 필터 · 태스크 수 · 마지막 실행*

리스트(테이블) 뷰에서 체크박스로 여러 DAG 를 고르면 목록 위에 ‘N 개 선택됨’과 일괄 작업 버튼이 나타납니다. 매니저 이상은 [활성화]·[비활성화]·[실행 취소]를, ADMIN 은 [삭제]까지 쓸 수 있으며, 그리드 뷰와 조회자에게는 일괄 작업 바가 나타나지 않습니다. [선택 해제]를 누르면 선택이 풀립니다.

일괄 작업	동작
활성화	고른 DAG 를 활성 상태로 바꿉니다. 확인 없이 바로 적용됩니다.
비활성화	고른 DAG 를 비활성으로 바꾸고, 진행 중인 DAG Run 이 있으면 함께 취소합니다. 진행 중인 Run 이 있을 때만 “실행 중인 N 개 DAGRun 이 중지됩니다. 계속할까요?”라고 묻습니다.
실행 취소	고른 DAG 의 대기·실행 중인 DAG Run 을 모두 취소합니다. “선택된 N 개 DAG 의 활성 실행을 모두 취소하시겠습니까?” 확인 대화상자를 거칩니다.
삭제 (ADMIN)	고른 DAG 를 삭제합니다. “이 DAG 를 삭제하시겠습니까?” 확인 대화상자를 거치며 되돌릴 수 없습니다.

8.2 DAG 생성

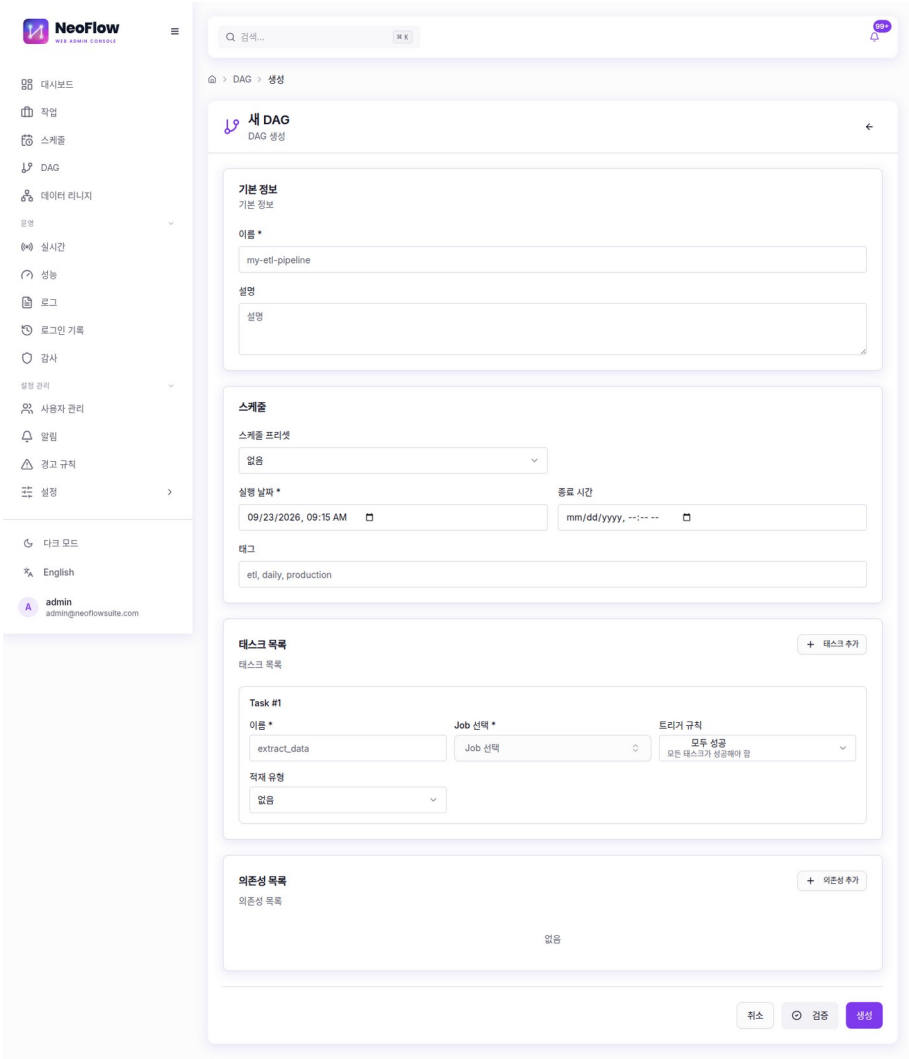
DAG 폼은 기본 정보·스케줄·태스크·의존성을 한 화면에서 위에서 아래로 입력하도록 구성됩니다. 기본 정보에 이름(영문 kebab-case)과 설명을 넣고, 스케줄 카드에서 프리셋·실행 날짜·종료 시간(선택)과 분류용 태그(쉼표 구분)를 정합니다. [+ 태스크 추가]로 각 단계를 Task 카드로 추가하며, 각 카드에는 이름·실행할 Job(필수, ACTIVE Job 전체가 목록에 나옴)·트리거 규칙·적재 유형(없음/초기/증분/소급 적재)을 지정합니다. [+ 의존성 추가]로 Task 간 선후 관계를 정의합니다.

트리거 규칙은 부모 태스크들이 끝난 상태에 따라 이 태스크를 실행할지 정하는 조건으로, 다음 7 가지 중에서 고릅니다. 선택 목록에는 이름과 한 줄 설명이 함께 표시되며 기본값은 '모두 성공'입니다.

화면 이름	코드	화면 설명
모두 성공	ALL_SUCCESS	모든 태스크가 성공해야 함 (기본값)
모두 실패	ALL_FAILED	모든 태스크가 실패해야 함
모두 완료	ALL_DONE	모든 태스크 완료 (상태 무관)
하나 성공	ONE_SUCCESS	하나 이상의 태스크가 성공
하나 실패	ONE_FAILED	하나 이상의 태스크가 실패
실패 없음	NONE_FAILED	실패한 태스크가 없어야 함 (건너뜀 허용)
건너뜀 없음	NONE_SKIPPED	건너뛴 태스크가 없어야 함

DAG 스케줄은 5 필드·6 필드 cron 과 모든 프리셋(5·15·30 분마다 포함)·Cron 빌더 값을 그대로 실행하며, @reboot·7 필드 cron·1 분 미만 간격처럼 실행할 수 없는 형식은 저장할 때 거부됩니다. 새로 만든 DAG 는 비활성 상태로 저장되고 스케줄 자동 실행 스위치도 꺼져 있습니다. 실행하려면 DAG 상세에서 [활성화]를 누르고, 예약 실행을 쓰려면 스케줄 카드의 '스케줄 활성/비활성' 스위치도 켭니다.

> **참고** DAG 와 Task 에는 재시도·타임아웃 입력이 없습니다. Task 는 선택한 Job 을 일반 Job 실행으로 돌리므로 그 Job 의 타임아웃(기본 3600 초)과 재시도 횟수(기본 0)가 그대로 적용됩니다.



▲ DAG 폼 — 기본 정보 · 태스크 카드 · 의존성 정의

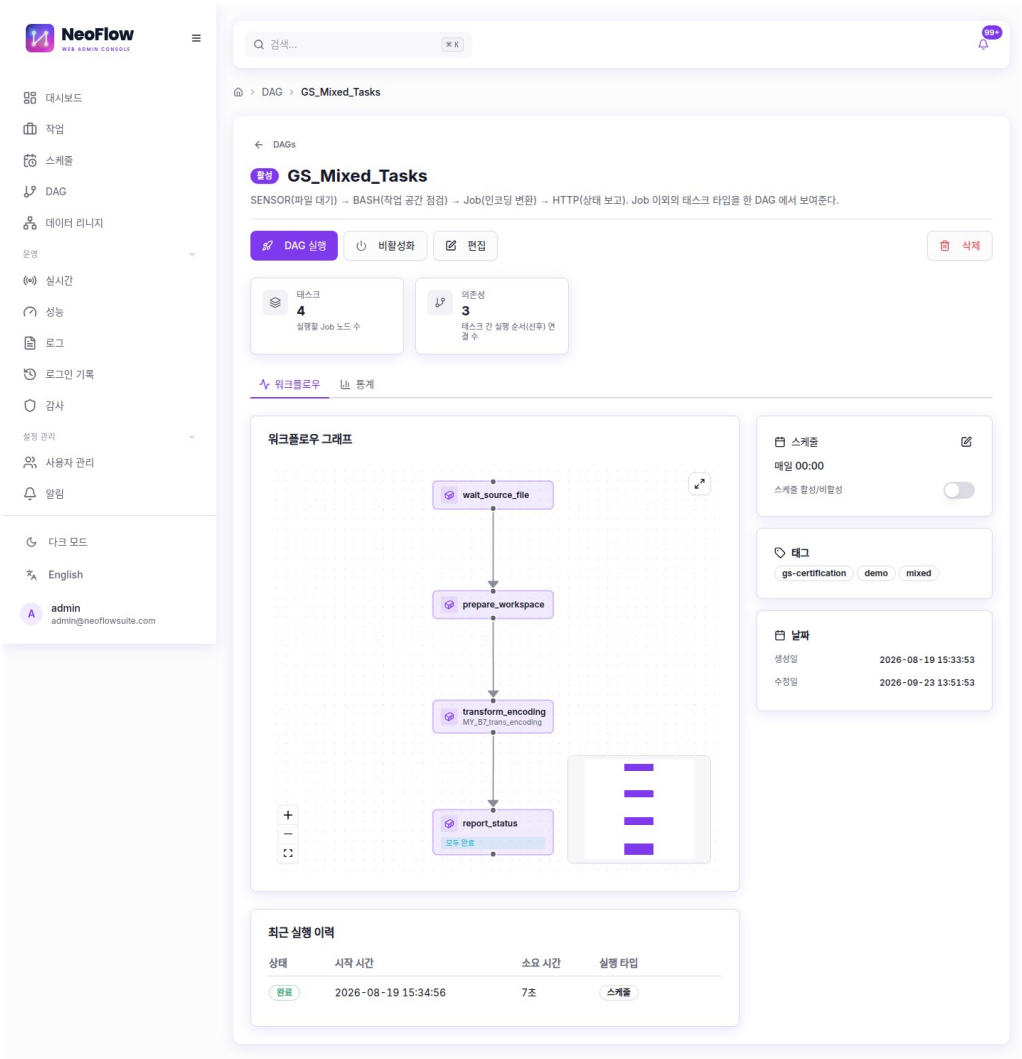
폼 아래쪽의 [검증] 버튼을 누르면 서버가 태스크 이름 중복, 각 태스크에 지정한 Job 의 존재 여부와 ACTIVE 상태, 의존성이 가리키는 태스크의 존재, 순환 의존성, 스케줄 형식을 검사하고, 결과를 의존성 카드 아래의 검증 결과 패널에 표시합니다. 통과하면 'DAG 검증 통과'가, 문제가 있으면 'DAG 검증 실패'와 함께 항목별 오류(예: "중복된 Task 이름: …", "Job 이 ACTIVE 상태가 아닙니다: …")가 목록으로 나옵니다. 저장할 때도 서버가 같은 이름의 DAG 존재 여부, 스케줄 형식, Job 존재·ACTIVE 상태, 의존성 참조, 순환을 다시 검사하며, 문제가 있으면 저장하지 않고 "DAG 저장에 실패했습니다: <사유>" 토스트로 알립니다.

8.3 DAG 상세와 실행

DAG 상세는 워크플로우와 통계 두 탭으로 나뉩니다. 워크플로우 탭은 메타 정보, 읽기 전용 워크플로우 그래프, 최근 실행 이력을 한 화면에 모으고, 통계 탭에서는 이 DAG 의 실행 추이를 확인할 수 있습니다. 그래프에서는 노드별 Job 을 확인할 수 있고, 우측 사이드 카드(스케줄·태그·날짜)에서 스케줄을 토글하거나 편집합니다. 상단 액션 바의 [DAG 실행]을 누르면 새 DAG Run 이 생성되고, 자동으로 DAG Run 상세로 이동해 진행 상황을 추적합니다. 비활성 DAG 는 실행이 거부되므로 먼저 [활성화]를 누릅니다. 최근 실행 이력에서 대기·실행 중인 Run 은 행 끝의 취소 버튼으로 멈출 수 있습니다. 액션 바(실행·활성 토글·편집·삭제)와 취소 버튼은 매니저 이상에게만 보이며, 조회자에게는 스케줄 자동 실행 스위치가 비활성으로 표시됩니다.

수동 실행 시 입력하는 Config Override(JSON)는 모든 Task 의 Job 에 파라미터로 전달되며, Job 실행 대화상자에 입력한 값과 같은 최우선 순위를 가집니다(6.4 참조). load_type 을 넣으면 Task 마다 지정한 적재

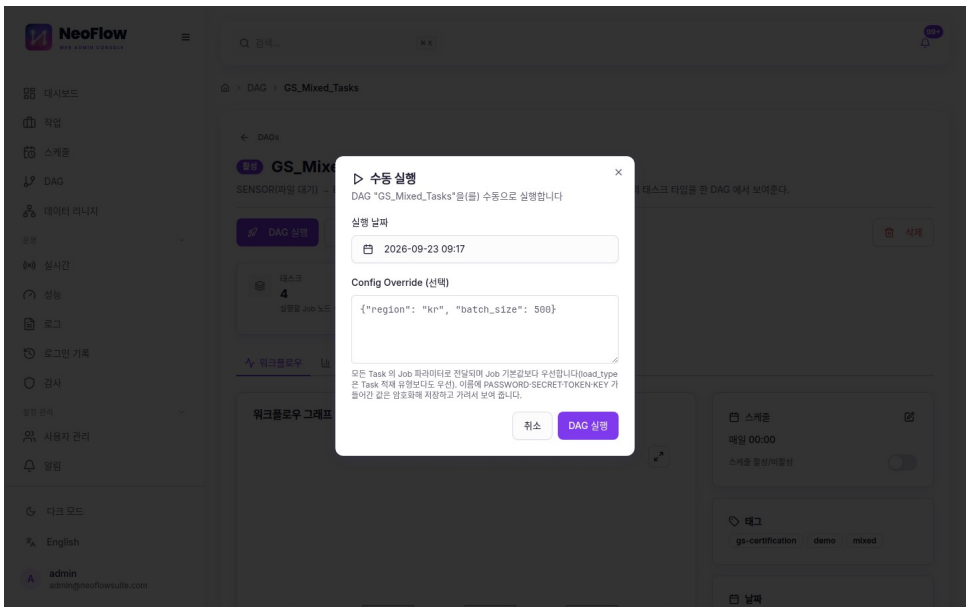
유형보다 우선하고, 이름에 PASSWORD·SECRET·TOKEN·KEY 가 들어간 값은 암호화해 저장되어 응답에서 ********로 보입니다.



▲ DAG 상세 — 워크플로우 그래프 + 실행/편집 액션

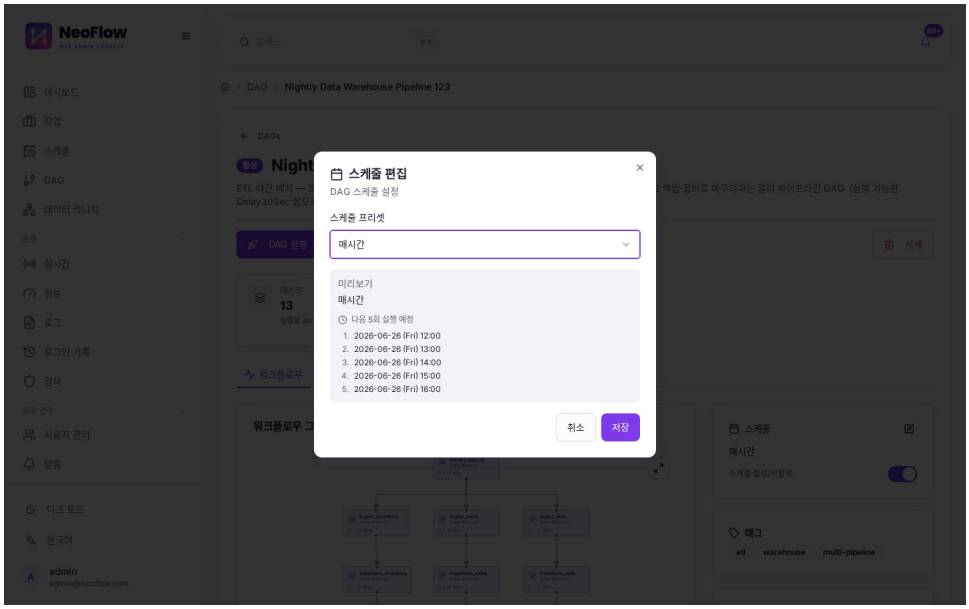
레이어팝업(모달) DAG 상세에서는 수동 실행·스케줄 편집·그래프 전체화면을 레이어로 엽니다.

▸ 수동 실행



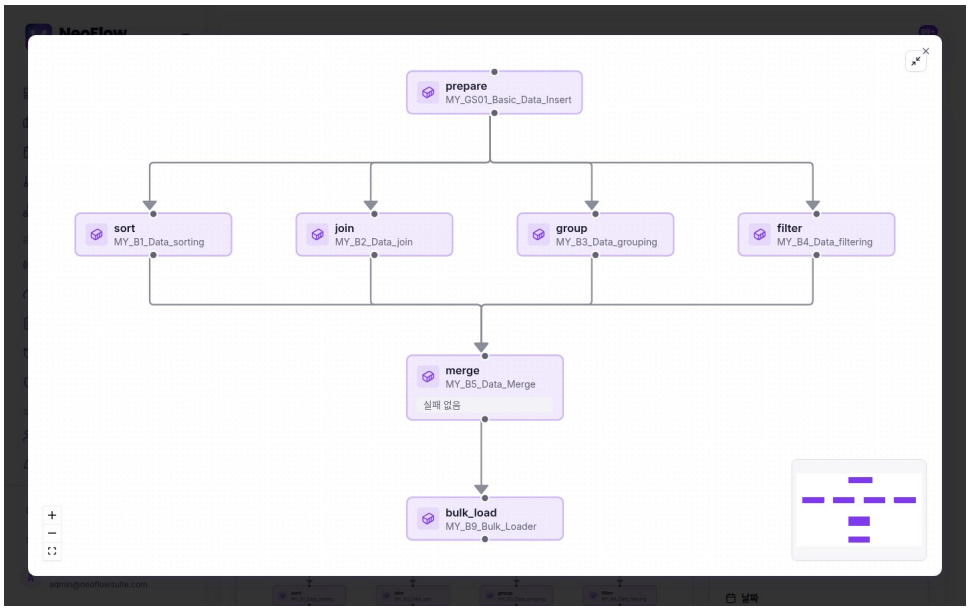
- 헤더 [DAG 실행](로켓) 버튼으로 엽니다.
- 실행 날짜와 Config Override(JSON, 실시간 유효성 검사)를 입력합니다.
- Config Override 의 모든 항목은 모든 Task 의 Job 에 파라미터로 전달되며, Job 기본값·공통 설정·Task 적재 유형보다 우선합니다. 값은 문자열·숫자·true/false 만 받습니다.
- [실행] 시 DAG Run 이 생성되고 실행 상세로 이동합니다.

▸ 스케줄 편집



- 스케줄 카드의 편집(연필) 버튼으로 엽니다.
- 프리셋 또는 Cron 빌더로 구성하고, 미리보기로 향후 실행 시각을 확인합니다.

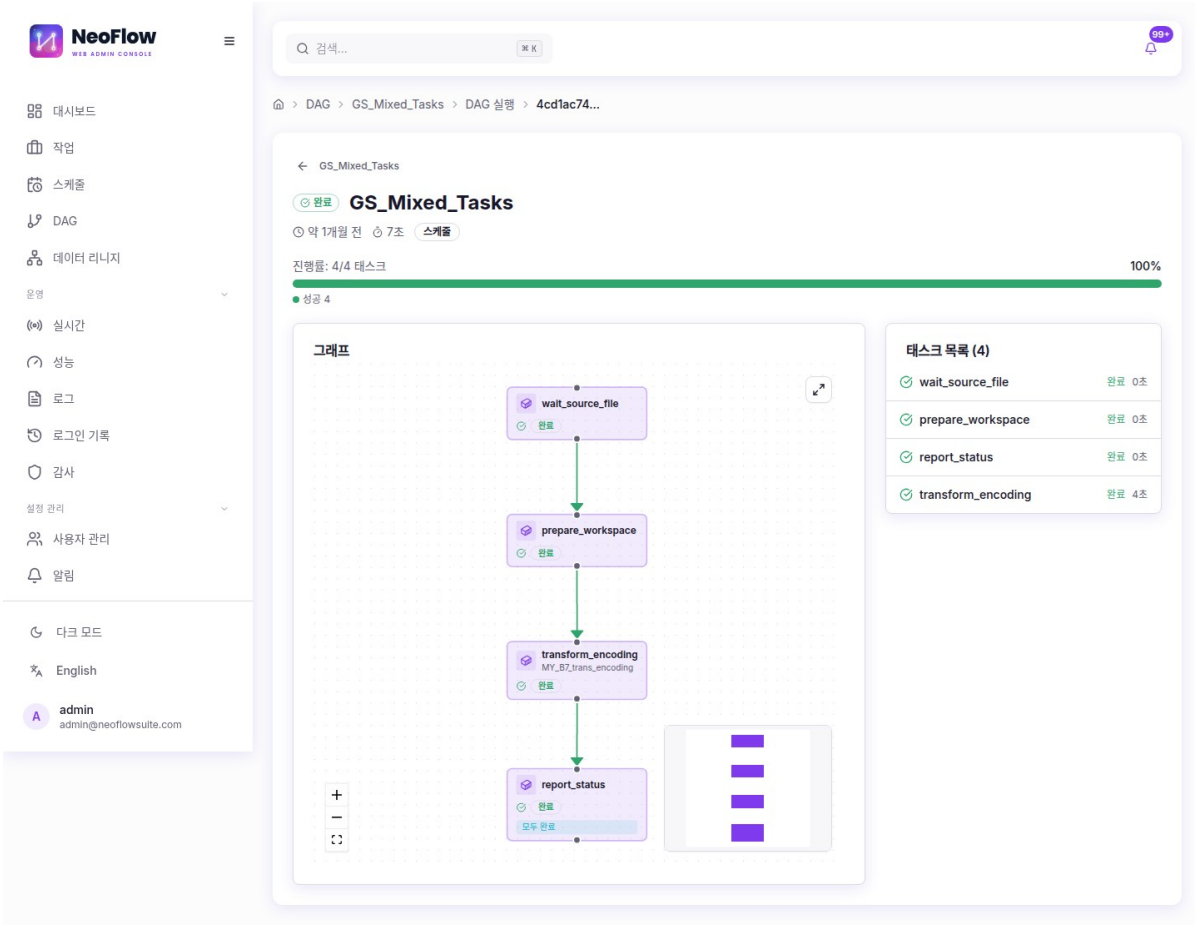
▸ 그래프 전체화면



- 그래프 우상단 [확대]로 95vw×90vh 전체화면 다이얼로그가 열립니다.
- 미니맵·줌/팬 컨트롤로 큰 DAG 를 탐색합니다.
- 노드를 클릭하면 닫히며 해당 Task 가 선택되고, [축소] 또는 ESC 로 닫습니다.

8.4 DAG Run 모니터링

DAG Run 상세는 한 번의 DAG 실행이 어떻게 진행되는지를 그래프와 태스크 목록으로 추적합니다. 상단 진행률 바가 '진행률: 7/7 태스크'처럼 끝난 태스크 수와 비율을 보여주고(완료·실패·건너뜀·업스트림 실패를 모두 끝난 것으로 세므로 모든 태스크가 끝나면 결과와 관계없이 100%), 막대를 상태별 색으로 나누며 그 아래에 상태별 개수를 표시합니다. 태스크 상태가 바뀔 때마다 갱신됩니다. 그래프의 각 노드 안 상태 배지 색으로 태스크 상태를 한눈에 파악할 수 있고, 우측 태스크 목록의 행을 클릭하면 시작·완료 시각과 실행 상세로 가는 [상세 보기] 링크가 펼쳐집니다. Job 이 비활성이어서 실행 요청이 거부된 태스크는 Job 실행이 만들어지지 않으므로 [상세 보기]가 없고, 행 아래에 거부 사유가 빨간 글씨로 표시됩니다. 헤더 배지가 '실시간'이면 WebSocket 으로 갱신 중이고, 'Polling'이면 연결이 끊겨 5 초마다 다시 조회하는 중입니다. '실시간' 상태에서도 10 초마다 서버에서 상태를 다시 읽으므로, 이중화 구성에서 다른 인스턴스가 진행하는 실행도 최대 약 10 초 안에 화면에 반영되고, 실행이 끝나면 갱신을 멈춥니다(12.8 참조).



▲ DAG Run 상세 — 진행률 + 상태 배지 그래프

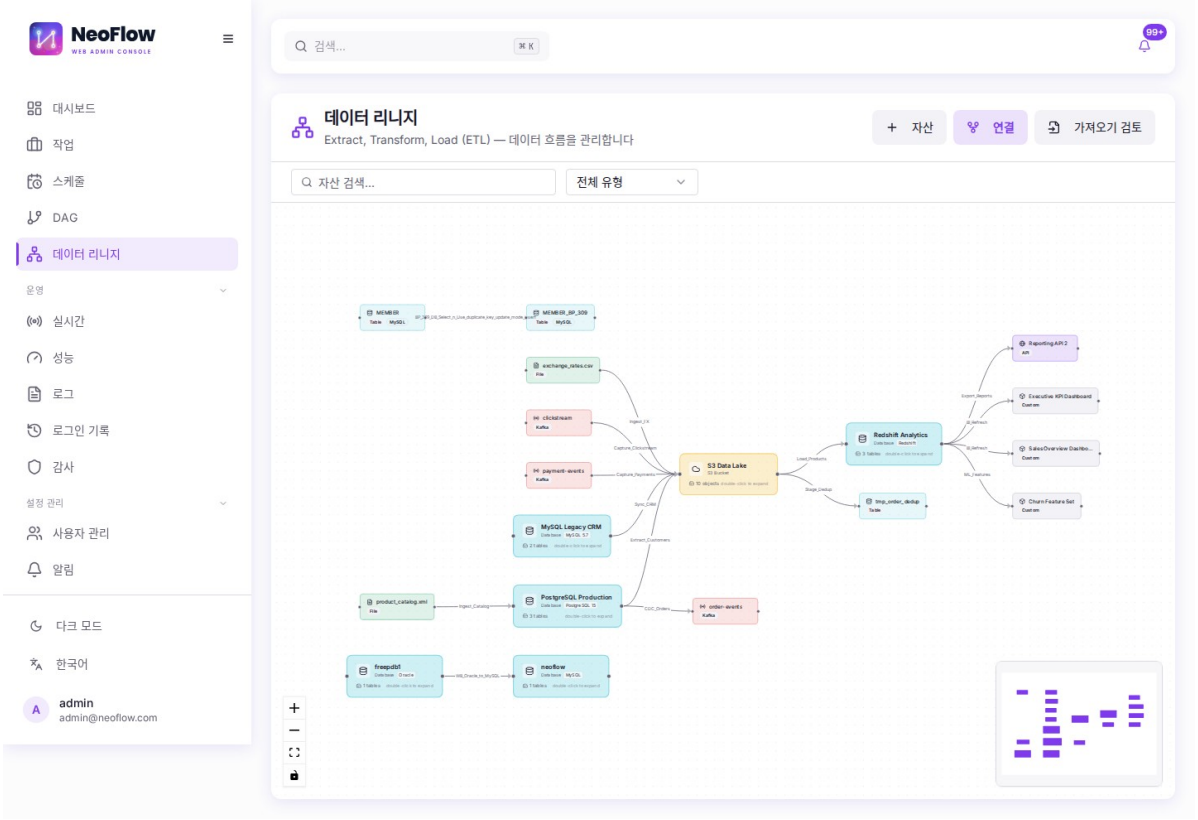
노드 상태	상태 배지 색
QUEUED (대기)	회색
RUNNING (실행 중)	청록 (회전 아이콘, 노드 테두리 강조)
SUCCESS (성공)	초록
FAILED (실패)	빨강
SKIPPED (건너뜀)	회색
UPSTREAM_FAILED (업스트림 실패)	노랑

DAG Run 의 최종 상태는 모든 태스크가 끝난 뒤 정해집니다. FAILED·UPSTREAM_FAILED 태스크가 하나도 없으면 SUCCESS(건너뜀은 무관), 하나라도 있으면 FAILED 이며, 알림용으로 둔 '하나 실패'·'모두 완료' 태스크가 성공해도 마찬가지입니다. 사용자가 취소하면 CANCELLED 가 되고, 실행 중이던 노드는 FAILED, 대기 노드는 SKIPPED 로 끝납니다. 대기·실행 중인 Run 은 매니저 이상이 취소할 수 있습니다.

> **참고** 실패한 DAG Run 을 이 화면에서 다시 돌리는 기능(부분 재실행·전체 재실행)은 없습니다. 원인을 고친 뒤 DAG 상세의 [DAG 실행]으로 새 DAG Run 을 시작하면 모든 태스크가 처음부터 실행됩니다.

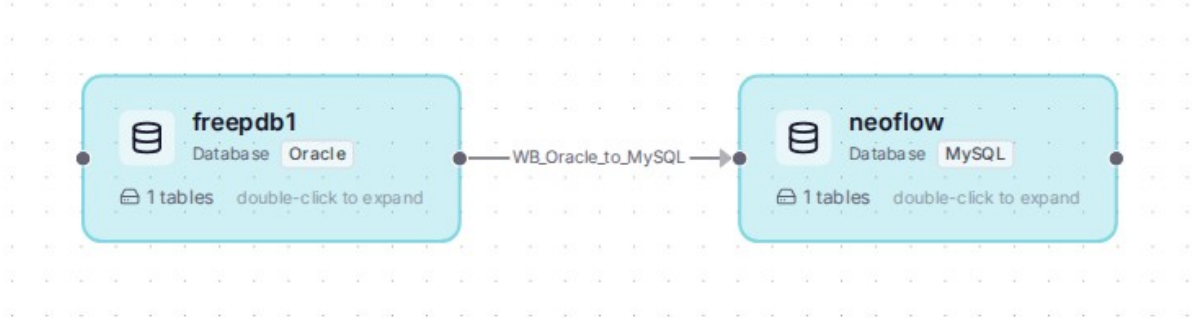
9. 데이터 리니지

[데이터 리니지] 화면은 Extract·Transform·Load 데이터 흐름을 그래프로 시각화합니다. 데이터베이스 시스템·테이블·파일 같은 자산(Asset) 노드를 엣지로 연결해, 어떤 데이터가 어디서 와서 어디로 흘러가는지 한눈에 보여줍니다. 노드 유형마다 색과 아이콘이 다르며, 캔버스를 드래그해 이동하고 휠로 확대/축소합니다. 좌하단 줌 컨트롤과 우하단 미니맵으로 큰 그래프도 쉽게 탐색할 수 있습니다. 상단에서 자산 이름으로 검색하고 '전체 유형' 드롭다운(데이터베이스·데이터베이스 테이블·S3 버킷·S3 객체·파일·API·Kafka 토픽·사용자 정의)으로 표시할 자산 종류를 좁힐 수 있습니다. 조회는 모든 사용자가 할 수 있고, 자산·연결 편집과 가져오기 승인은 ADMIN 만 가능합니다.



▲ 데이터 리니지 — 자산 노드 + 의존성 엣지 그래프

데이터베이스 시스템·테이블 노드에는 NeoFlow Studio 컴포넌트에서 추출한 DB 엔진이 벤더 배지(Oracle·MySQL·PostgreSQL·SQL Server)로 함께 표시됩니다. 벤더가 특정되지 않는 제네릭 컴포넌트(tDBInput·tJDBCInput)에는 배지가 붙지 않습니다.



▲ DB 엔진 배지 — Oracle 소스 → MySQL 타겟

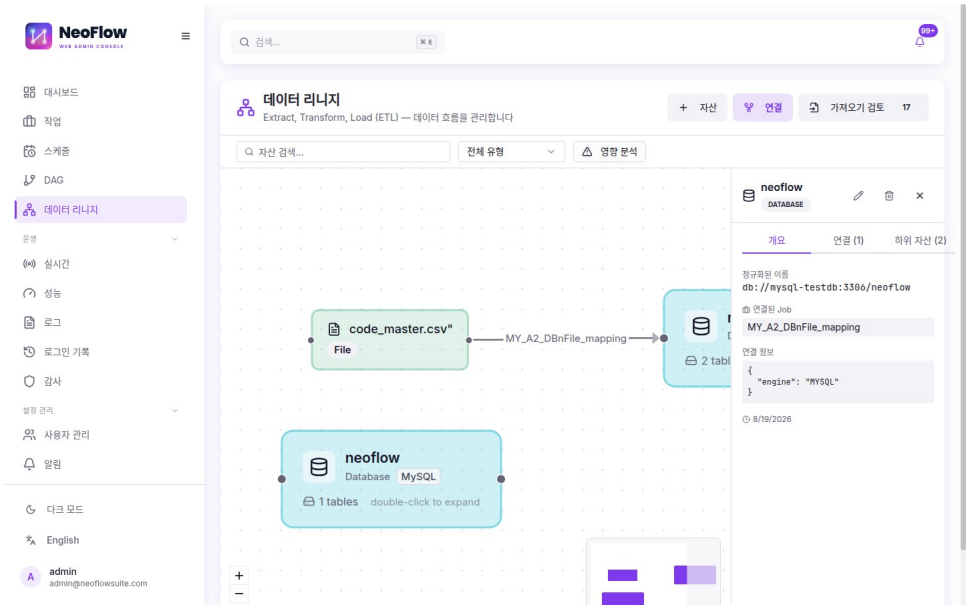
> **참고** 가져오기 재스캔: 리니지 초안은 Job 업로드 시 자동 생성됩니다. 이미 업로드된 Job 을 다시 파싱하려면 Job 상세의 [리니지 재스캔](ADMIN) 또는 가져오기 검토 패널의 [다시 스캔](↻)을 사용합니다. 재스캔은 기존

초안을 SUPERSEDED 처리하고 새 PENDING 초안을 만들어, 파서 개선(접속 정보 파라미터 치환, DB 엔진 추출 등)을 기존 Job 에 반영할 때 유용합니다.

주의 파라미터화된 접속 정보: 호스트·DB 를 Job 파라미터로 주입한 경우 해당 Job 파라미터 값으로 자동 치환되어 자산이 식별됩니다. 파라미터가 정의돼 있지 않으면 'unresolved:'로 표시되므로, Job 파라미터를 먼저 등록하세요.

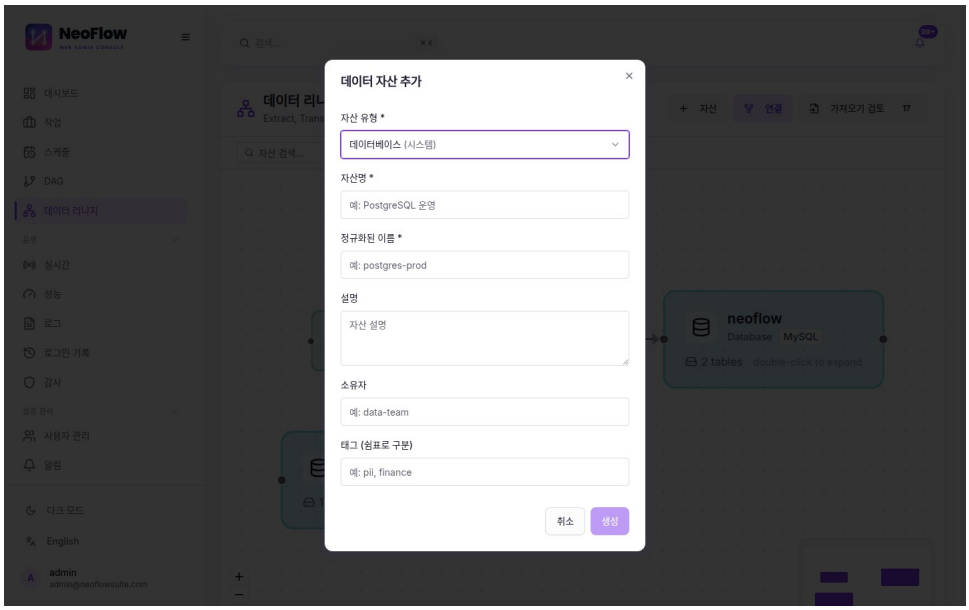
레이어팝업(모달) 그래프의 노드·엣지를 클릭하면 우측 상세 패널이 열리고, 자산·연결 편집과 NeoFlow Studio 가져오기 검토는 모달/드로어로 처리합니다.

▸ 자산 상세 패널



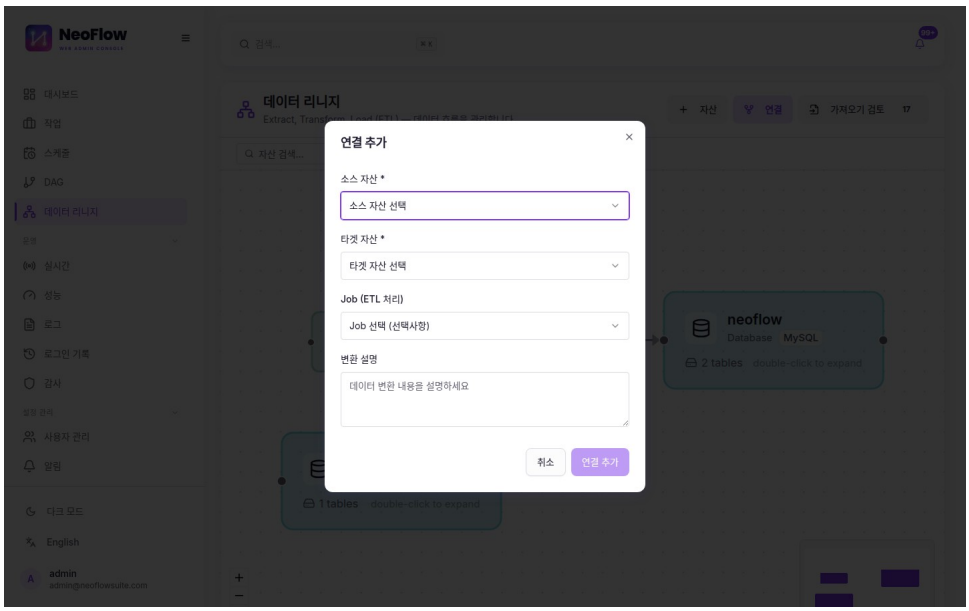
- 그래프 노드를 클릭하면 우측 패널이 열립니다.
- 개요(정규화된 이름·설명·소유자·태그·연결된 Job·생성일)·연결(업/다운스트림)·하위 자산을 보여줍니다.
- ADMIN 은 패널에서 자산·연결을 편집·삭제할 수 있습니다.

▸ 자산 추가·편집



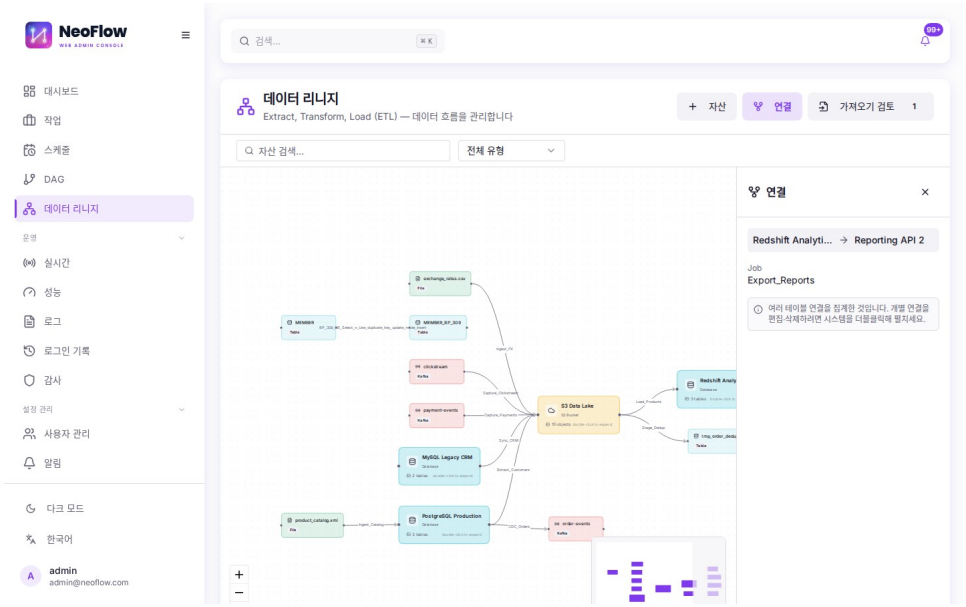
- 헤더 [자산] 추가 또는 상세 패널 편집으로 엽니다.
- 유형·상위 시스템(테이블·객체일 때)·자산명·정규화된 이름·설명·소유자·태그(쉼표로 구분)를 입력합니다.

▸ 연결 추가



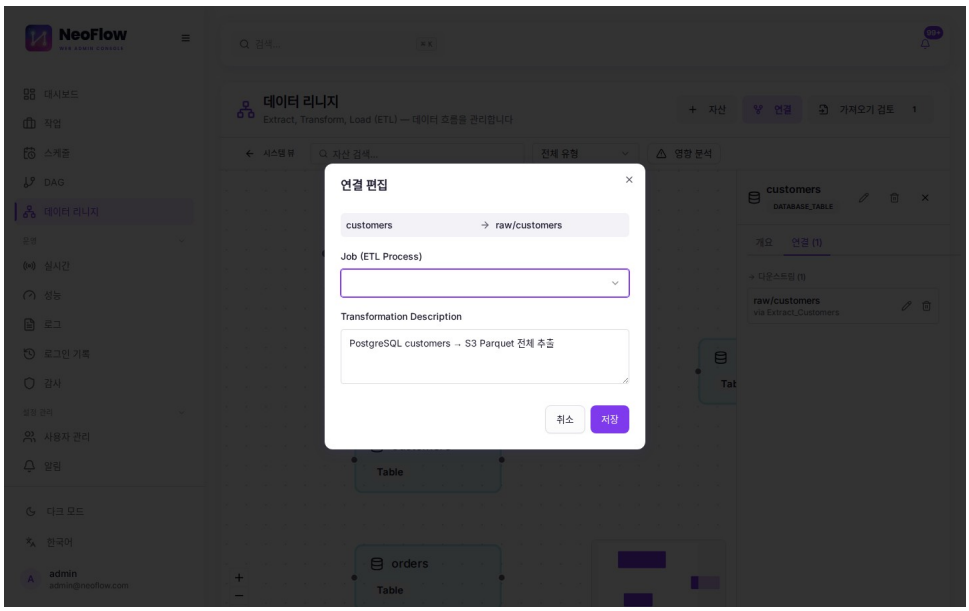
- 헤더 [연결] 버튼으로 엽니다.
- 소스 자산·타겟 자산·Job (ETL 처리)(선택)·변환 설명으로 데이터 흐름(엣지)을 만듭니다.
- Job 선택 목록에는 등록된 모든 Job 이 나옵니다.

▸ 연결 상세 패널



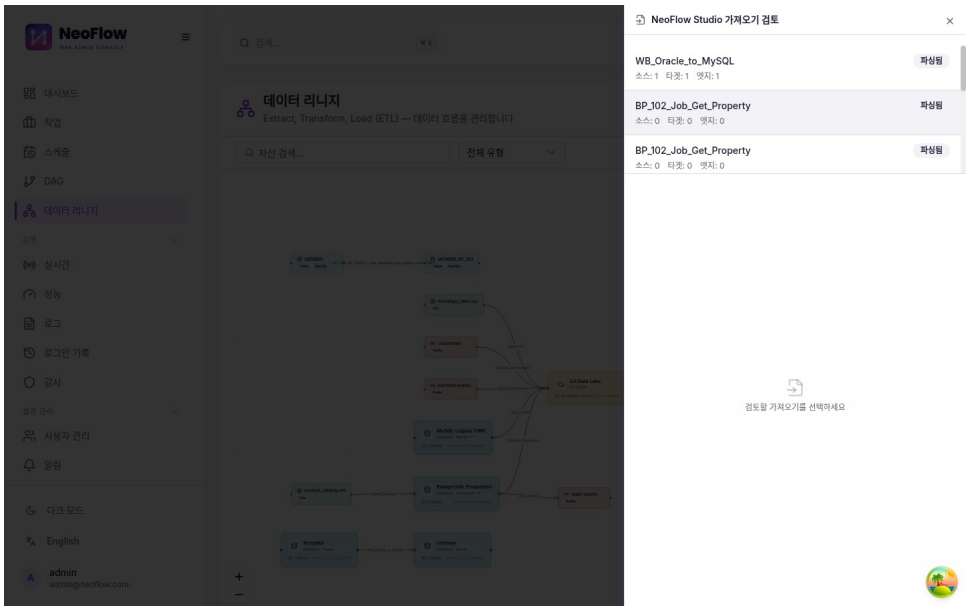
- 그래프 엣지를 클릭하면 우측 패널이 열립니다.
- 소스→타겟 흐름·연결 Job·변환 설명을 보여줍니다(시스템 뷰 집계 엣지는 안내만, 상세 엣지는 ADMIN 편집·삭제).

▸ 연결 편집



- 엣지 상세 또는 자산 연결 행의 [편집](ADMIN)으로 엽니다.
- 소스→타겟은 읽기 전용이며, Job 과 변환 설명만 수정합니다.

▸ NeoFlow Studio 가져오기 검토



- 헤더 [가져오기 검토](ADMIN, 대기 건수 배지)로 우측 드로어가 열립니다.
- 좌측 가져오기 목록(파싱됨/건너뜀)과 우측 상세(Sources·Targets·Edges·Unmapped)를 확인합니다.
- 상세 패널 헤더의 [다시 스캔](↻)으로 같은 Job 을 재파싱해 새 초안을 만들 수 있습니다.
- Apply(적용)/Reject(거부)로 초안 엣지를 확정합니다.

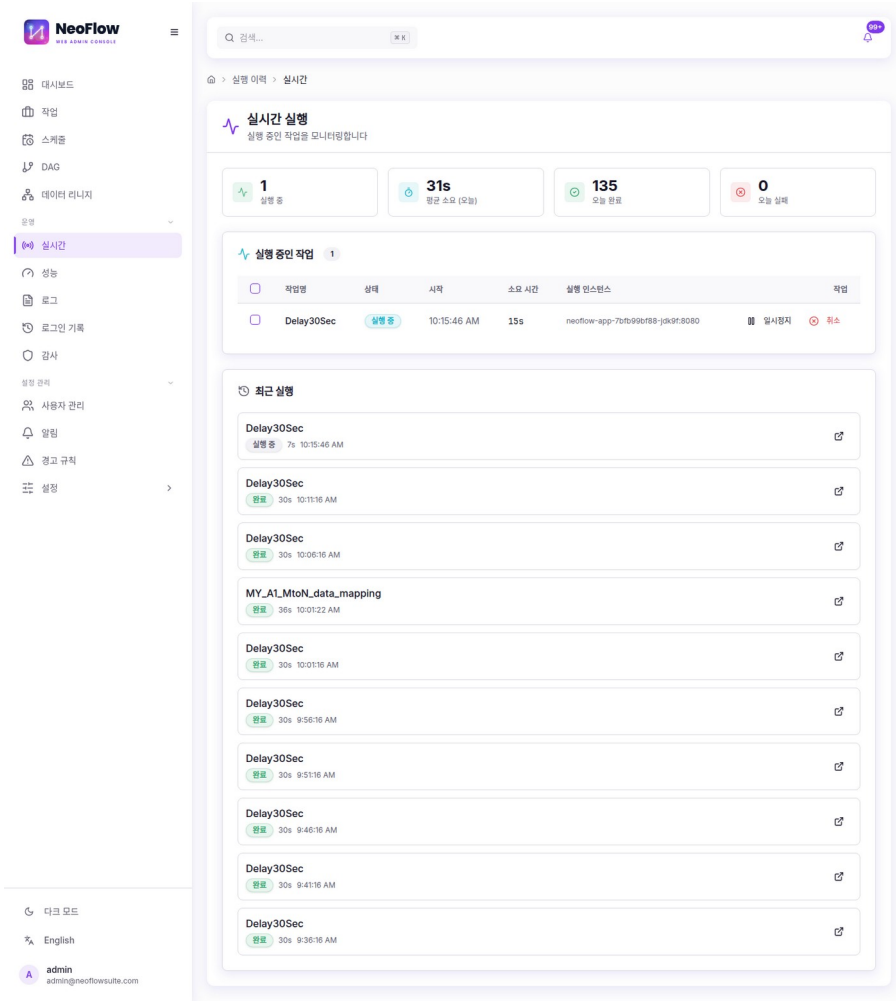
10. 운영

운영 장에서는 사이드바 '운영' 그룹의 화면 — 실시간 실행 모니터링, 실패 원인을 자동 분류하는 장애 진단, 성능, 로그, 로그인 기록, 감사 — 과 운영 리포트를 다룹니다.

10.1 실시간 실행

[운영 > 실시간] 화면은 지금 실행 중인 작업을 실시간으로 모니터링합니다. 상단에 실행 중·오늘 평균 소요·오늘 완료·오늘 실패 KPI 카드 4 개가 있습니다. '오늘'은 서버 시간 00:00 이후 시작한 실행이며, 오늘 실패는 FAILED 와 TIMEOUT 을 함께 셉니다. 실행 슬롯을 기다리는 대기(QUEUED) 실행이 있으면 그 아래에 '대기 중인 작업' 목록이 나타나 Job 이름과 우선순위 배지(긴급/높음/보통/낮음)를 실행될 순서(우선순위가 높은 것부터, 같으면 먼저 대기한 것부터)대로 보여주고, 행 우측 ⊗ 아이콘으로 취소할 수 있으며, 누르면 "정말로 실행을 취소하시겠습니까?" 확인 대화상자를 거칩니다.

'실행 중인 작업' 테이블에는 작업명·상태·시작 시각·소요 시간·실행 인스턴스(host:port)가 표시되며, 행마다 [일시정지]/[취소] 버튼이 있습니다. [일시정지]는 확인 없이 바로 요청되고, [취소]는 "정말로 실행을 취소하시겠습니까?" 확인 대화상자를 거칩니다. 일시정지한 실행은 표에 남아 '일시정지' 배지와 [재개] 버튼으로 바뀌고, 체크박스로 여러 행을 고르면 [선택 취소]로 한꺼번에 취소합니다("선택한 N 개 실행을 취소하시겠습니까?" 확인). 하단의 '최근 실행' 카드 목록은 최근 요청된 10 건을 상태 배지·소요 시간과 함께 보여주며, FAILED·TIMEOUT 카드의 ↻ 재시도 아이콘은 확인 없이 원래 실행과 같은 설정으로 새 실행을 만듭니다. 이 조작 버튼들은 매니저 이상에게만 보입니다.



◢ *실시간 실행 — KPI + 실행 중 작업 + 최근 실행*

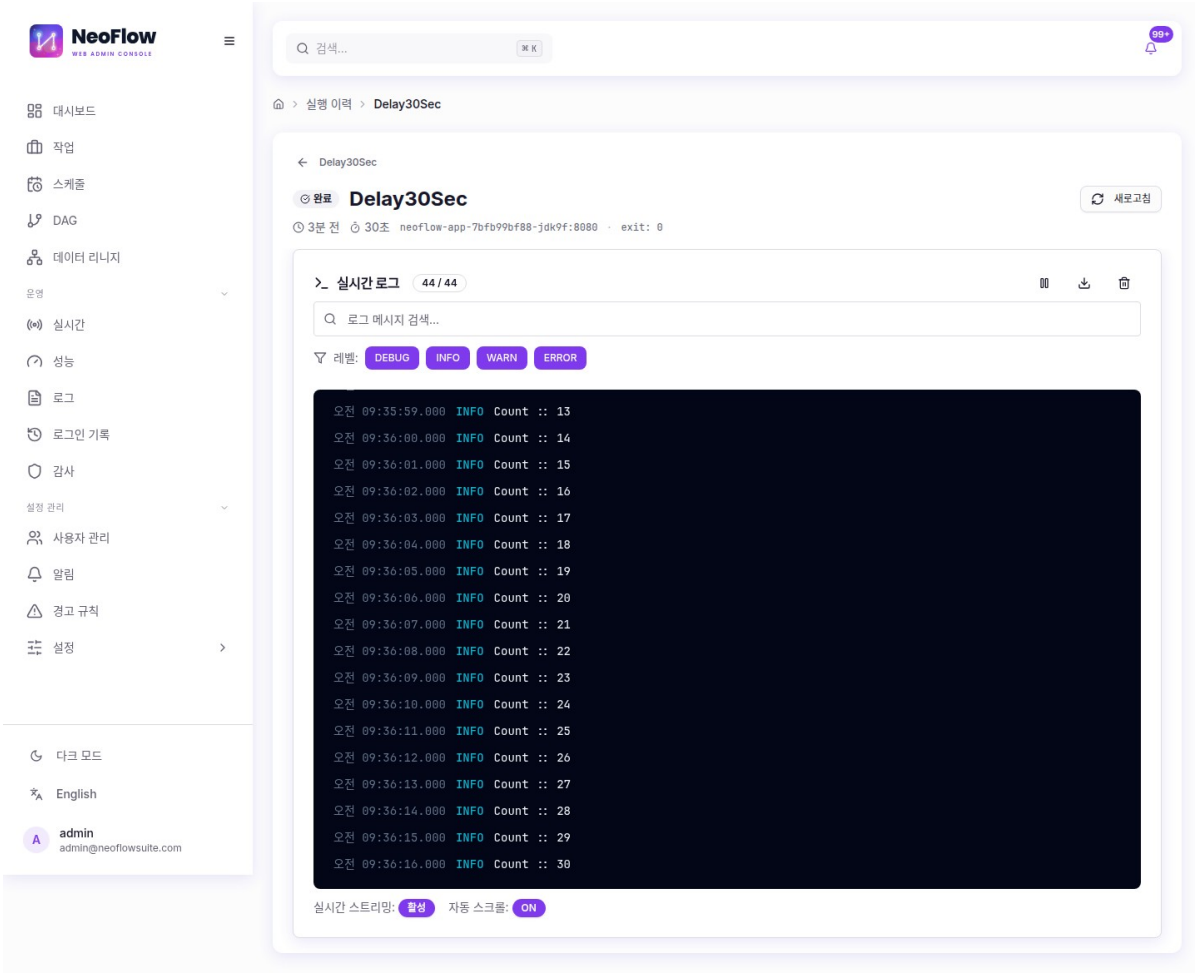
> **참고** 재시도는 이 화면에서만 합니다(실행 상세·Job 상세에는 재시도 버튼이 없음). 재시도는 끝난 실행을 되살리지 않고 새 실행(새 ID)을 만들며, 원래 실행의 결과·로그·진단은 그대로 남습니다. 새 실행은 원래 실행의 Job 버전·파일·파라미터·타임아웃·재시도 횟수를 그대로 쓰므로, 그 뒤에 Job 을 다시 업로드했어도 원래 버전으로 돌아갑니다. 새 버전으로 돌리려면 Job 에서 새로 실행하세요. Job 이 INACTIVE 이거나 삭제됐거나 라이선스 동시 실행 한도에 이르렀으면 거부됩니다.

> **참고** '실행 인스턴스'는 그 실행을 실제로 집행한 앱 인스턴스를 host:port 로 나타냅니다(예: HOONY:8080). NeoFlow 는 별도의 워커 노드 클러스터를 두지 않고 앱 인스턴스 내부의 실행 큐로 동시 실행 수를 제어하므로, 이 값은 클러스터의 워커가 아니라 HA 구성에서 어느 인스턴스가 그 실행을 맡았는지를 알려 줍니다. 같은 값이 Job 상세의 실행 이력 탭과 실행 상세에도 표시됩니다.

10.2 실행 상세

실행 행을 클릭하면 실행 상세로 들어갑니다. 헤더 좌상단의 '← Job 이름'으로 Job 상세에 돌아갈 수 있고, 상태 배지와 함께 시작 시각·소요 시간(진행 중이면 1 초마다 늘어나는 경과 시간)·실행 인스턴스·exit code 가 인라인으로 표시됩니다. 우측 액션은 [새로고침]·[일시정지](RUNNING)·[재개](PAUSED)·[취소]이며, 재시도 버튼은 없습니다. [취소]는 아직 끝나지 않은 모든 상태(요청됨·대기 중·대기·실행 중·일시정지·재시도 중)에서 나타나고 "정말로 실행을 취소하시겠습니까?" 확인 대화상자를 거칩니다. 실행에 오류 메시지가 있으면 헤더 아래 빨간 배너로 보여주고, FAILED 이면 로그 패널 위에 장애 진단 카드가 붙습니다.

로그 패널은 실행 중에는 WebSocket 스트림과 3 초 주기 재조회로 채워지고(연결이 끊겨도 로그가 빠지지 않음), 종료된 실행은 DB(job_execution_logs)에 저장된 로그를 보여줍니다. 로그 컨트롤로 스트리밍 일시정지·재개, 로그 다운로드(화면에 보이는 줄을 .txt 로), 로그 클리어(화면에서만 지움), 메시지 검색(일치 부분 강조), 레벨 필터(DEBUG·INFO·WARN·ERROR)를 쓸 수 있습니다. Job 이 찍은 줄의 레벨은 줄 모양으로 정합니다(예외·스택 줄은 ERROR, 평문 메시지는 INFO — 10.6 참고). 이 화면은 실행 파라미터를 표시하지 않습니다(API 응답에서 확인하며 시크릿은 ********).



▲ 실행 상세 — 헤더 · 오류 배너 · 실시간 로그

상태	동작
REQUESTED (요청됨)	실행 요청이 접수된 직후의 상태. 곧바로 실행 큐에 들어가 QUEUED 가 됨. [취소] 가능
QUEUED (대기 중)	실행 슬롯 대기. 이 화면의 [취소](확인 대화상자)로 취소할 수 있고, 실시간 실행의 '대기 중인 작업' ⊗나 Jobs 목록의 [실행 중지]로도 취소
PENDING (대기)	처리 대기를 뜻하는 예비 상태 값. 현재 실행 흐름에서는 기록되지 않으며, 표시되면 [취소] 가능
RUNNING (실행 중)	WebSocket 으로 로그 push, 회전 아이콘 배지. [일시정지]·[취소] 가능
PAUSED (일시정지)	프로세스가 멈춘 상태. [재개]·[취소] 가능. 경과 시간은 계속 늘어남
RETRY (재시도 중)	재시도 횟수가 남은 Job 이 실패해 다음 시도를 기다리는 상태. 다시 큐에 들어가 실행되며 [취소] 가능
COMPLETED (완료)	소요시간·exit code 표시. DB 에 저장된 로그 조회
FAILED (실패)	빨간 배지. 오류 배너 + 장애 진단 카드
CANCELLED (취소됨)	오류 배너에 취소 사유 표시
TIMEOUT (시간초과)	Job 타임아웃을 넘겨 강제 종료(exit code 124). 장애 진단 카드는 붙지 않음

[취소]를 확인하면 상태가 즉시 CANCELLED 로 바뀌고, Job 프로세스와 그 자식 프로세스(Talend java 포함)를 곧바로 강제 종료합니다. HA 에서 다른 인스턴스가 실행 중이던 Job 이면 그 인스턴스가 5 초 안에 취소

요청을 받아 종료하므로, 취소 뒤 로그가 조금 더 찍힐 수 있습니다. [일시정지]는 Job 프로세스 트리를 멈추고 (PAUSED), [재개]는 다시 움직입니다. 일시정지·재개는 Linux 호스트에서만 동작하며, 요청이 그 Job 을 실행 중인 인스턴스에 닿아야 합니다. 거부되면 토스트에 서버가 알려 준 이유가 함께 표시됩니다.

10.3 장애 진단

실행이 실패(FAILED)하면 실행 상세의 로그 패널 위에 장애 진단 카드가 자동으로 붙습니다. 실패 로그를 진단 룰셋과 대조해 원인 카테고리·요약·조치 가이드를 채워 주므로, 로그 전체를 처음부터 읽지 않고도 어디를 봐야 하는지 먼저 좁힐 수 있습니다. 분류 엔진은 결정론적인 룰 기반이며 모델 추론이 아닙니다. 같은 로그를 같은 룰셋으로 다시 평가하면 언제나 같은 결과가 나옵니다. 진단 요약은 실패 알림(인앱 알림·웹훅·Slack·실패 메일)에도 함께 붙습니다. 타임아웃으로 끝난 실행(TIMEOUT)과 취소된 실행에는 진단이 붙지 않습니다.

카드 머리에는 카테고리와 신뢰도(HIGH/MEDIUM/LOW) 배지, 매칭된 룰 id, 룰셋 버전이 표시되고 그 아래로 한 줄 요약·근거·조치 가이드·부가 매칭(접힘)이 이어집니다. 카드 하단에는 [로그 해당 위치로 이동]·[재분석] 버튼과 조치 코멘트 입력란이 있습니다.



▲ 장애 진단 카드 — 카테고리·신뢰도·룰 id·룰셋 버전 + 근거 · 조치 가이드 · 조치 코멘트

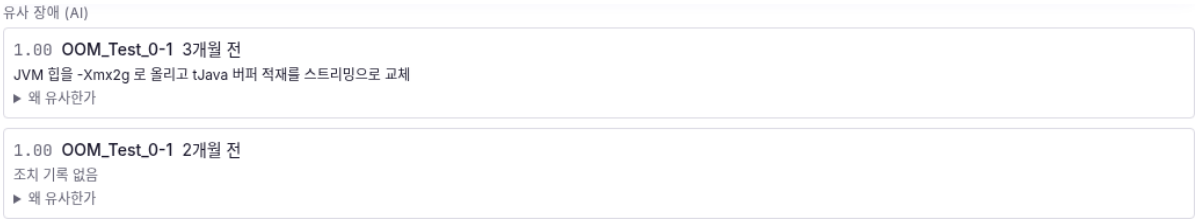
항목	의미
카테고리	ENV · DB_NETWORK · DB_AUTH · DB_SCHEMA · DB_DATA · FILE · RESOURCE · API · SAP · UNKNOWN 중 하나 (시드 룰 26 종의 분류 체계)
신뢰도	HIGH / MEDIUM / LOW. 통계로 계산한 확률이 아니라 룰 작성 시점의 판단값
룰 id	매칭된 룰의 식별자(예: db-conn-timeout). 커스텀 룰이 시드 룰을 덮어써도 id 는 그대로 유지
룰셋 버전	seed-2026.08 형식. 커스텀 룰이 있으면 파일 해시가 붙음(seed-2026.08+c3a1). 진단 레코드에 함께 저장되어 그 시점의 판단을 재현할 수 있음
근거	매칭된 로그 행 번호 + 마스킹 후 원문 인용. exit code 만으로 판단하는 룰은 행 번호가 없을 수 있음
조치 가이드	확인할 절차를 순서대로 제시하는 텍스트. 카드가 자동으로 수행하는 조치는 없음

> **참고** UNKNOWN 은 오류가 아닙니다. 룰셋이 로그를 설명하지 못하면 카드는 회색으로 바뀌고 카테고리에 UNKNOWN 이 표시되며, 근거 행 번호나 조치 가이드를 지어내지 않고 로그 마지막 구간만 인용합니다. 근거 없는 추정으로 그럴듯한 원인을 만들지 않기 위한 의도된 동작이며, 미분류가 쌓이는 것 자체가 다음 룰을 어디에

> 보태야 하는지 알려 주는 데이터입니다.

조치 코멘트는 이 실패를 어떻게 해결했는지 남기는 입력란으로, 매니저 이상만 저장할 수 있습니다. 같은 원인으로 다시 실패했을 때 다음 담당자가 처음부터 조사하지 않아도 되고, 아래의 유사 실패 목록이 이 코멘트를 그대로 읽어 가므로 남겨 두는 편이 좋습니다.

유사 장애 (AI)



▲ *유사 장애(AI) — 유사도 점수 · Job · 실패 시각 · 그때 남긴 조치 코멘트*

진단 카드 아래에는 과거의 비슷한 실패가 [유사 장애 (AI)]라는 이름으로 최대 3 건까지 붙습니다. 각 줄은 유사도 점수·Job 이름·실패 시각과 그때 남긴 조치 코멘트를 함께 보여주고, '왜 유사한가'를 펼치면 점수를 계산한 시그니처 원문을 확인할 수 있습니다. 유사도 하한은 0.90 입니다. 진단 시그니처가 모두 같은 틀을 공유해 서로 무관한 실패끼리도 0.87 까지 나오기 때문에 실측으로 정한 값이며, 엉뚱한 선례를 보고 엉뚱한 조치를 하는 것을 막기 위해 높게 잡았습니다.

이 블록은 임베딩 모델이 있을 때만 나타납니다. 모델 경로(기본 ./conf/models/bge-m3, 환경변수 NEOFLOW_EMBEDDING_MODEL_PATH)에 모델이 없으면 '유사한 실패 없음' 같은 문구조차 띄우지 않고 블록 자체가 표시되지 않습니다. 룰 기반 진단은 모델 유무와 무관하게 그대로 동작합니다. 모델을 새로 넣었다면 과거 실패에는 아직 임베딩이 없으므로, ADMIN 이 임베딩 소급 생성 API(POST /api/diagnosis/embeddings/backfill)를 호출해 채웁니다. 이중화 설치에서는 검색할 때마다 DB 에서 새로 저장된 사례를 더 읽으므로, 다른 노드가 진단한 실패도 바로 유사 사례로 나옵니다.

커스텀 룰

고객사 환경에만 있는 실패 패턴(사내 전용 컴포넌트의 에러 문자열, 사내 담당 부서 연락처 등)은 커스텀 룰로 보완합니다. 룰 파일은 앱 실행 위치 기준 ./conf/rules/diagnosis/*.yaml 에 두며, 배포 환경에서는 환경변수 NEOFLOW_DIAGNOSIS_RULES_DIR 로 경로를 바꿀 수 있습니다. 시드 룰과 같은 id 를 쓰면 새 진단이 추가되는 것이 아니라 그 시드 룰이 커스텀 내용으로 통째로 대체됩니다. 문법이 잘못된 파일은 그 파일만 건너뛰고 경고 로그를 남기므로, 커스텀 룰 하나의 오타가 서버 기동을 막지는 않습니다.

룰 파일은 서버 기동 시 자동으로 로드됩니다. 서버를 재기동하지 않고 반영하려면 ADMIN 권한으로 룰셋을 다시 불러온 뒤, 해당 실행의 카드에서 [재분석]을 누릅니다. 반영되면 룰셋 버전에 파일 해시가 붙으므로 카드의 버전 표기로 확인할 수 있습니다. 룰셋 재로드를 위한 전용 관리 화면은 아직 없습니다.

```
POST /api/diagnosis/rules/reload        # 룰셋 재로드 (ADMIN)
```

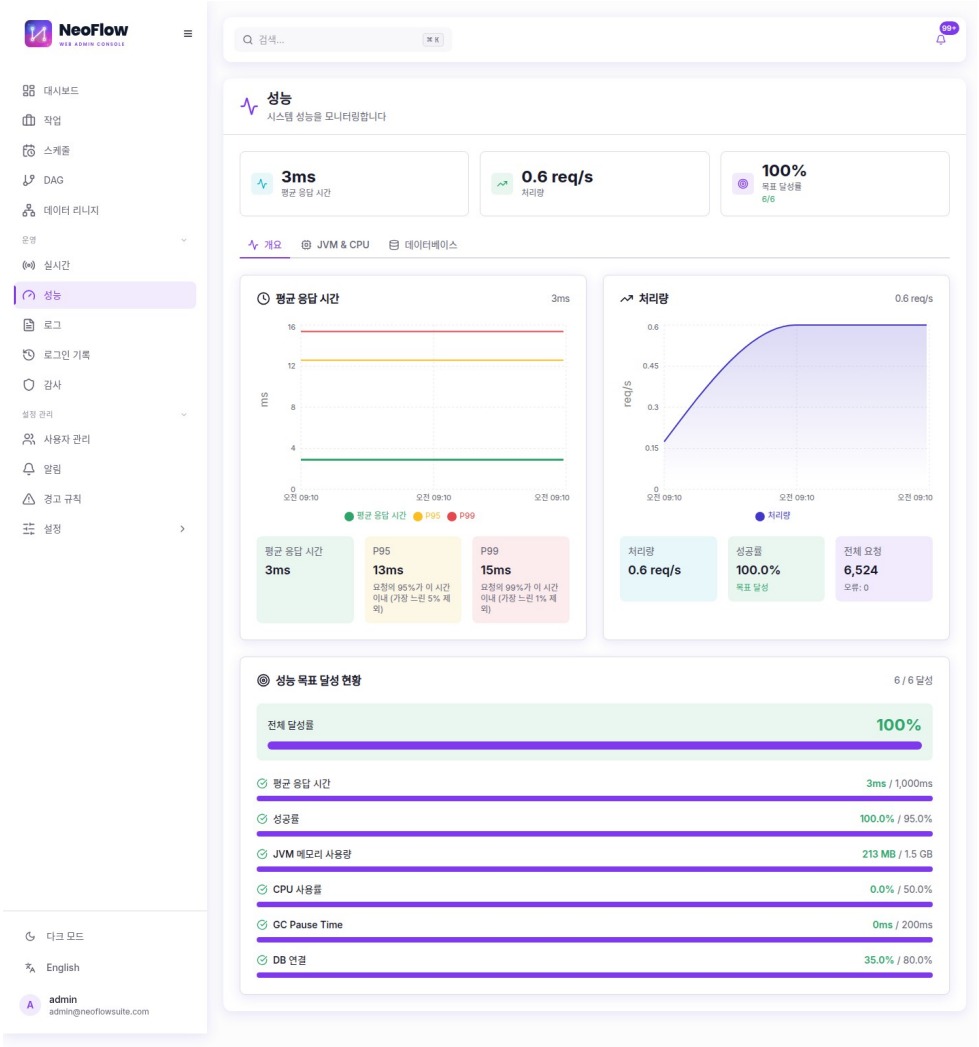
역할별로 할 수 있는 동작은 다음과 같습니다.

동작	조회자	매니저	관리자
카드 조회 (근거 · 조치 가이드 포함)	≣	≣	≣
로그 해당 위치로 이동	≣	≣	≣
재분석	×	≣	≣
조치 코멘트 입력 · 저장	×	≣	≣
룰셋 재로드	×	×	≣

> **주의** 자주 빠지는 함정 — ① [로그 해당 위치로 이동]이 안내만 띄우고 이동하지 않으면 실행이 끝나 실시간 로그 버퍼가 비워진 경우입니다. 로그를 내려받아 확인하세요. ② 커스텀 룰 파일을 고친 뒤에는 룰셋을 먼저 다시 불러와야 [재분석] 결과가 바뀝니다. 재분석은 현재 로드된 룰셋으로 재평가할 뿐 파일을 다시 읽지 않습니다. ③ 조치 코멘트 입력란이 보이지 않으면 조회자(읽기 전용) 권한입니다. ④ 근거 인용의 일부가 가려진 것은 토큰·자격증명·IP 마지막 옥텟을 마스킹한 결과로, 의도된 동작입니다.

10.4 성능 (ADMIN)

[성능(Performance)] 화면은 시스템 성능을 모니터링합니다. 사이드바의 [성능] 메뉴와 명령 팔레트의 성능 항목은 ADMIN 에게만 보이며, 다른 역할이 주소로 직접 들어가면 대시보드로 이동합니다. 상단 KPI 카드로 평균 응답 시간·처리량·목표 달성률을 요약하고, 개요·JVM & CPU·데이터베이스 탭으로 나누어 응답 시간(평균/P95/P99) 추이, 초당 요청 수, Heap·GC·CPU, HikariCP 커넥션 풀 등을 보여줍니다. 하단의 성능 목표 달성 현황은 항목별 현재값과 목표값을 진행 막대로 한눈에 비교합니다.



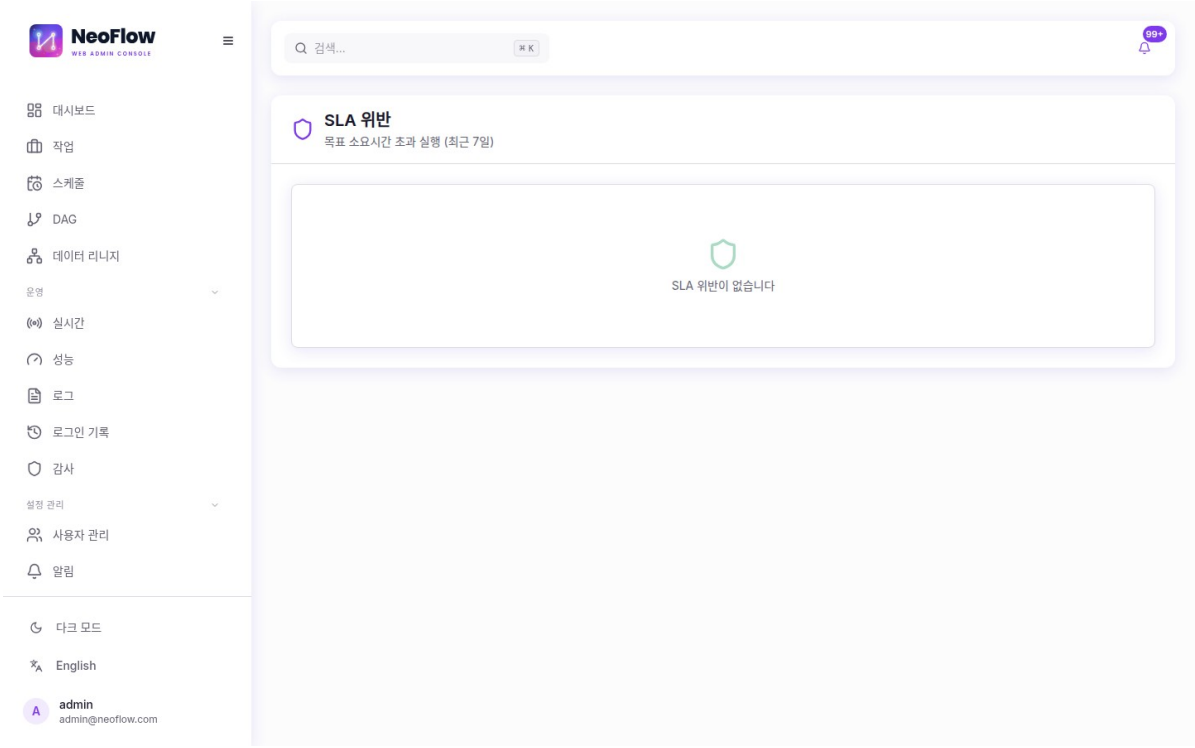
▲ 성능 — 응답 시간 · 처리량 · 목표 달성 현황

10.5 SLA 위반

[SLA 위반] 화면은 목표 소요시간(SLA target)을 넘겨 완료된 실행을 최근 7 일(시작 시각 기준)로 모아 보여줍니다. 사이드바 메뉴는 없고, 대시보드 SLA 위반 패널의 [전체 보기]로 들어갑니다. 위반이 있으면

작업명·시작 시간·소요 시간·SLA 목표·초과분이 최근에 끝난 순으로 최대 200 건 나열되고(기간·Job 필터와 정렬은 없음), 행을 클릭하면 그 실행의 상세 화면으로 이동합니다. 위반이 없으면 'SLA 위반이 없습니다'라는 빈 상태가 표시되며, 이는 오류가 아니라 정상 운영을 의미합니다. 목록은 60 초마다 다시 불러옵니다.

> **참고** 위반 여부는 실행을 요청할 때 그 실행에 기록해 둔 Job 의 SLA 목표로 판정합니다. Job 상세 설정 탭에서 목표를 바꾸면 그 뒤에 요청되는 실행부터 새 목표가 적용되고, 이미 기록된 실행의 판정은 바뀌지 않습니다. 목표가 없을 때 요청된 실행은 판정 대상이 아니며, 실시간 실행 화면의 재시도로 만든 새 실행은 재시도한 시점의 목표를 따릅니다.



▲ SLA 위반 — 최근 7 일 목표 초과 실행 (빈 상태 = 정상)

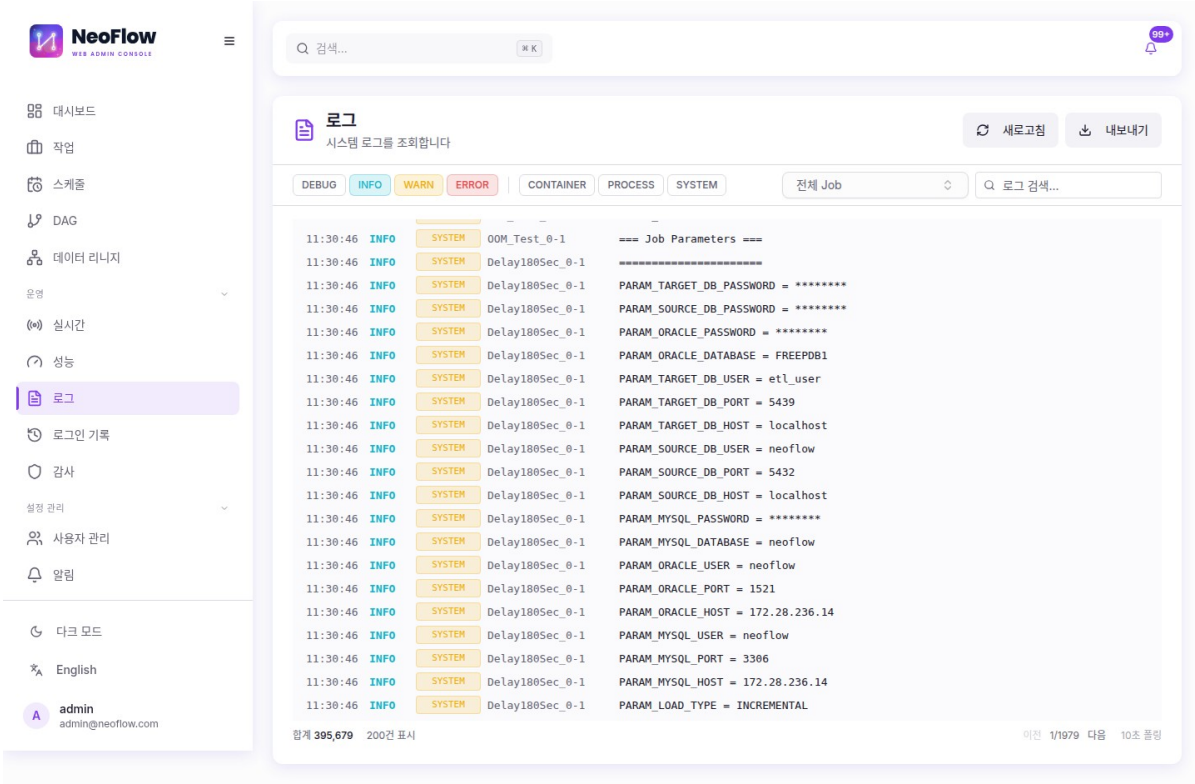
10.6 로그

[로그(Logs)] 화면은 저장된 Job 실행 로그를 조건으로 검색하는 화면으로, 모든 사용자가 쓸 수 있습니다. 레벨(DEBUG/INFO/WARN/ERROR)과 소스(아래 표)를 조합해 필터링하고, 특정 Job 만 골라 보거나 메시지 키워드(대소문자 무시, 부분 일치)로 검색합니다. 테이블은 고정폭 시각·레벨·메시지·Job 이름을 시간 역순으로 보여주며, DEBUG 는 기본으로 숨겨져 있습니다. Job 이 찍은 줄의 레벨은 줄 모양으로 정합니다. Java 예외(Exception in component..., Caused by:, 스택 줄, 예외 클래스로 시작하는 줄)와 SEVERE:는 ERROR, WARNING:은 WARN, log4j 로 빌드한 Job 의 [ERROR]·[WARN]·[INFO]·[DEBUG] 접두어는 그대로 쓰고, 나머지 평문 줄은 INFO 입니다. 평문으로 찍은 오류 문구(예: tDie 메시지)도 INFO 로 저장되므로, 오류를 빠짐없이 찾으려면 INFO 까지 켜고 메시지로 검색합니다. 실행 중인 Job 의 실시간 로그는 이 화면이 아니라 실행 상세에서 봅니다. 서버 애플리케이션 로그(Logback)는 이 화면에 나오지 않으므로 서버 로그 파일이나 컨테이너 로그로 확인합니다.

우상단 [내보내기]는 지금 걸린 레벨·소스·Job·검색어 필터에 맞는 로그를 최신 순으로 전부 CSV(logs-YYYY-MM-DD.csv, UTF-8 BOM)로 받습니다. 행이 많으면 오래 걸리고 끝날 때까지 버튼이 비활성(회전 표시)이며, 실패하면 “CSV 내보내기에 실패했습니다”가 뜹니다.

소스	기록되는 줄
PROCESS	Job 프로세스가 표준 출력·표준 오류로 찍은 줄. 실행 로그의 대부분
SYSTEM	서버가 실행을 시작할 때 남기는 파라미터 목록(=== Job Parameters === 블록, 시크릿 값은 ********)
CONTAINER	컨테이너 실행 모드용 소스. 제공하는 배포 구성은 모두 프로세스 실행이라 나타나지 않음

소스 선택 칩에는 저장된 로그에 실제로 있는 소스만 나타납니다.



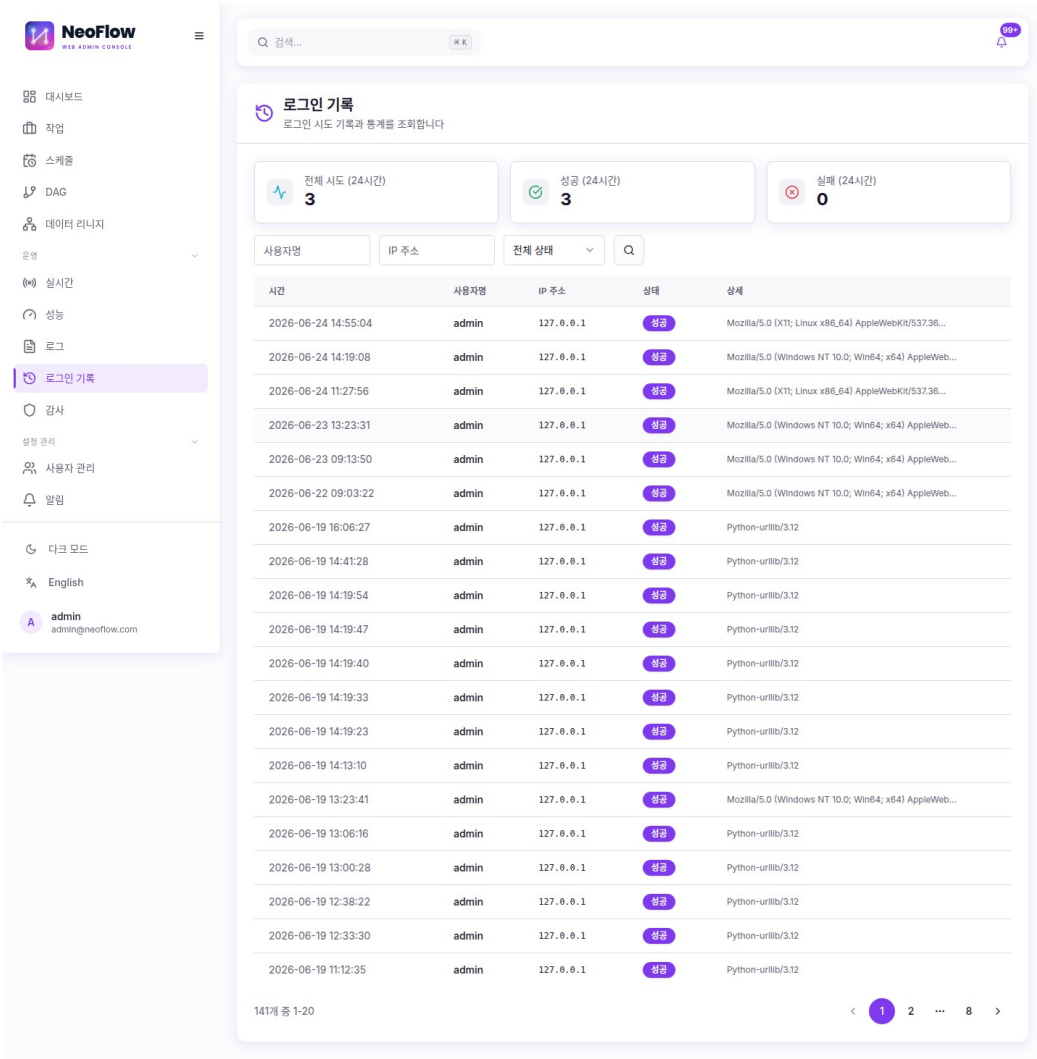
▲ *로그 — 레벨/소스 필터 + 키워드 검색*

> **참고** 이 화면의 로그(DB log_entries)는 실행 상세의 로그(job_execution_logs)와 같은 줄을 따로 한 벌 더 저장한 것입니다. 앱이 매일 03:30 에 보관 기간(기본 90 일, 서버 설정 LOG_ENTRIES_RETENTION_DAYS)이 지난 행을 자동으로 지웁니다. 실행 상세의 실행 로그는 설정 > 스토리지 탭의 실행 이력 보관 기간(기본 90 일)에 따라 실행 이력과 함께 정리됩니다.

10.7 로그인 기록 (ADMIN)

[로그인 기록(Login History)]은 모든 로그인 시도를 성공·실패와 함께 시각·사용자·IP 로 추적하는 ADMIN 전용 보안 화면입니다. 사용자명·IP·상태로 필터링할 수 있고, 상단 KPI 카드에 전체 시도·성공·실패 건수를 요약합니다. 이 화면에는 잠금 해제 버튼이 없습니다. 잠긴 계정은 잠금 시간이 지나면 저절로 풀리고, 급하면 [사용자 관리]의 비밀번호 재설정으로 풀 수 있습니다.

기록되는 IP 는 실제로 접속한 클라이언트의 주소입니다. 로드밸런서나 프록시를 거치는 구성에서는 서버가 신뢰하도록 지정한 프록시(환경변수 TRUSTED_PROXIES)가 전달한 X-Forwarded-For 값만 사용하므로, 클라이언트가 헤더를 조작해 다른 IP 로 기록되게 할 수 없습니다. 감사 로그와 요청 한도도 같은 규칙으로 IP 를 정합니다.

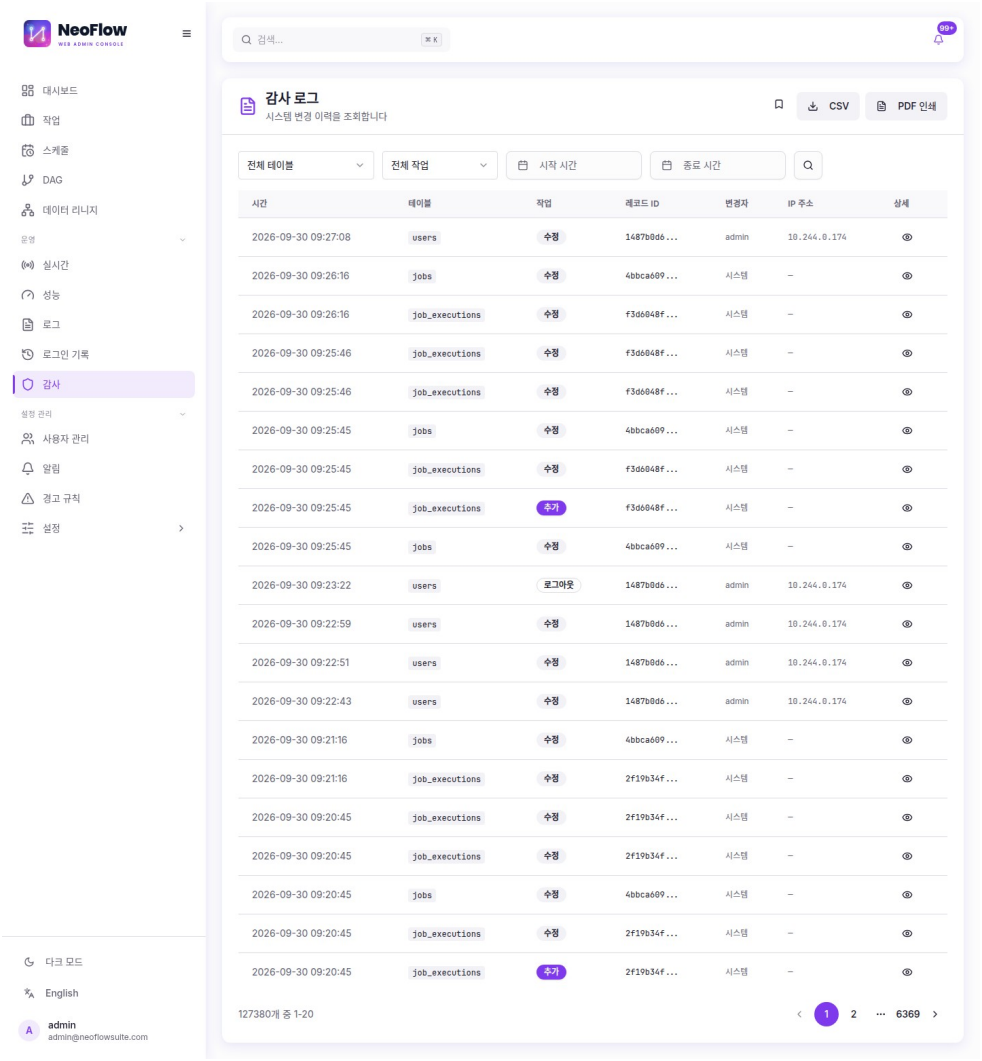


▲ 로그인 기록 — 성공/실패 시도 추적

로그인 기록은 90 일 동안 보관됩니다. 앱이 매일 02:00 에 보관 기간이 지난 기록을 자동으로 삭제합니다. 보관 기간을 정하는 항목은 설정 화면에 없으며, 서버 환경변수 LOGIN_HISTORY_RETENTION_DAYS(기본 90, 단위 일)로만 바꿀 수 있습니다. Docker 설치는 compose/.env 에 값을 적고 docker compose up -d 로 앱 컨테이너를 다시 만들어야 적용됩니다(restart 만으로는 .env 를 다시 읽지 않음).

10.8 감사 로그 (ADMIN)

[감사 로그(Audit)]는 어떤 테이블의 어떤 레코드가 누구에 의해 언제 추가·수정·삭제되었는지와 로그아웃·계정 잠금·설정 변경 같은 보안 작업을 시간순으로 추적하고, 행을 열면 변경 전/후 차이를 보여주는 ADMIN 전용 화면입니다. 테이블·작업(추가/수정/삭제와 아래 보안 작업)·기간(시작/종료 시각, 고른 시각 그대로 조회)으로 필터링하고 자주 쓰는 필터 조합을 북마크(프리셋)로 저장할 수 있습니다. [CSV]는 필터에 맞는 로그를 최신 순으로 전부 내려받고(행이 많으면 오래 걸리며 끝날 때까지 버튼 비활성), [PDF 인쇄]는 최신 순 최대 1,000 건을 인쇄 대화상자로 엽니다. 1,000 건을 넘으면 인쇄물 제목 아래에 “필터에 맞는 N 건 중 최근 1000 건만 인쇄했습니다”가 붙으므로, 기간을 나눠 인쇄하거나 CSV 를 쓰세요.



◢ *감사 로그 — 변경 이력 + 전/후 Diff*

작업	기록 내용
추가 · 수정 · 삭제	사용자·Job·실행·DAG·리니지·경고 규칙·웹훅·공통 설정·Job 파라미터·API 토큰(발급·폐기) 등의 데이터 변경. 변경 전/후 값
로그아웃	로그인 중인 사용자가 로그아웃한 시각과 IP
계정 잠금	로그인 실패가 한도에 닿아 계정이 잠긴 사실(사용자명·실패 횟수·잠금 시간). 행위자는 시스템, IP 는 실패한 요청의 IP
계정 잠금 해제	관리자가 잠금을 해제했거나, 잠겨 있던 계정의 비밀번호를 관리자가 재설정한 경우
설정 변경	시스템 설정(리포트·보안 정책·경고·보관 기간)의 바뀐 항목과 이전→이후 값, SMTP 설정의 바뀐 항목(비밀번호는 바뀌었는지 여부만), 공통 설정의 활성 프로필 변경
라이선스 설치	설치한 라이선스의 고객·에디션·유형·발급일·만료일·하드웨어 바인딩 여부(파일 내용·서명은 남기지 않음)

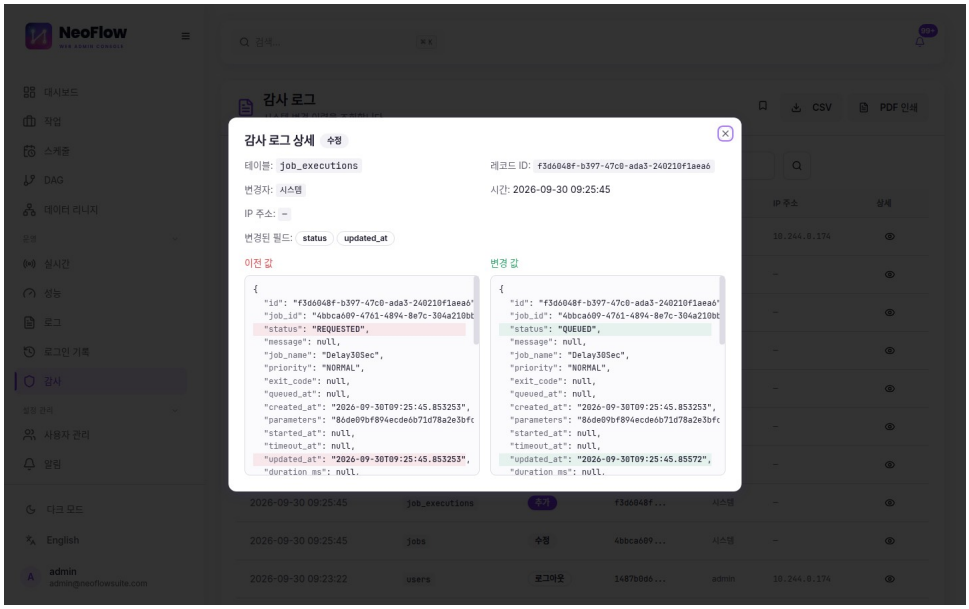
모든 기록에는 작업한 사용자와 클라이언트 IP 가 남고, 목록과 상세에서 'IP 주소'로 확인할 수 있습니다. 비밀번호 해시·API 토큰 해시·웹훅 URL 과 서명 키·비밀로 지정한 공통 설정 값과 Job 파라미터 기본값은 저장할 때 ********로 가려지며, 화면·CSV·인쇄에도 가려진 값만 나옵니다. CSV 에는 변경한 사용자 이름(Changed By Name) 열이 포함됩니다.

스케줄러와 실행기는 실행이 끝날 때마다 마지막 실행 시각·다음 실행 시각·마지막 실행 상태를 되씁니다. 이 UPDATE 도 감사 트리거를 타기 때문에, 분 단위로 도는 Job 하나가 감사 기록의 대부분을 차지하는 일이 생깁니다. 이런 항목은 런타임 기록으로 분류만 될 뿐 목록에서 빠지지 않으며, 이 화면과 CSV 내보내기·PDF 인쇄는 런타임 기록까지 전부 보여줍니다. 감사 추적의 완전성이 우선이기 때문입니다. 특정 Job 이 언제 무엇이 바뀌었는지만 보려면 런타임 기록을 걸러 주는 Job 상세의 [변경 이력 보기]를 쓰는 편이 빠릅니다(6.3 참조).

감사 로그는 365 일 동안 보관됩니다. 앱이 매일 02:15 에 보관 기간이 지난 기록을 자동으로 삭제합니다. 보관 기간을 정하는 항목은 설정 화면에 없으며, 서버 환경변수 AUDIT_LOG_RETENTION_DAYS(기본 365, 단위 일)로만 바꿀 수 있습니다. 적용 방법은 로그인 기록과 같습니다(10.7 참조).

레이어팝업(모달) 각 감사 로그 행의 상세는 변경 전후 값을 비교하는 모달로 확인합니다.

▸ **감사 로그 상세**



- 행의 눈 아이콘으로 엽니다.
- 테이블·레코드 ID·변경자·시각·액션 배지를 보여줍니다.
- 이전 값/변경 값을 2 열로 비교하며 변경된 필드를 강조합니다.

10.9 운영 리포트 (PDF 인쇄 · 주간 리포트)

NeoFlow 는 운영 현황을 브라우저 인쇄로 PDF 를 만들거나, 주간 리포트 메일로 정기 발송할 수 있습니다. 인쇄 버튼은 세 곳에 있으며, 누르면 브라우저 인쇄 대화상자가 열리고 대상에서 'PDF 로 저장'을 고르면 한글이 제대로 보이는 PDF 가 만들어집니다(대상이 프린터로 잡혀 있으면 종이로 인쇄되므로 주의).

화면	버튼	담기는 내용
대시보드	리포트 인쇄	대시보드 지표 8 개(실행 중, 오늘 성공/실패, 성공률, 전체·활성·스케줄 Job 수, 평균 실행시간). 차트·Top 10·SLA 목록은 없음
Job 목록	PDF 인쇄	지금 걸린 필터에 맞는 Job 전체(최대 10,000 건). 이름·표시명·카테고리·상태·Studio Job Version·최근 실행
감사 (ADMIN)	PDF 인쇄	지금 걸린 필터에 맞는 로그를 최신 순 최대 1,000 건

인쇄물 제목·열 이름은 화면 언어를 따르며, 기간이나 포함 항목을 고르는 옵션은 없습니다(대시보드 리포트는 오늘 기준 값).

주간 리포트는 최근 7 일의 실행 요약을 메일 본문(HTML)으로 보내며 PDF 첨부는 없습니다. 받는 사람은 관리자가 [설정 > 리포트] 탭의 수신자에 적은 주소뿐이고, 사용자가 스스로 구독하는 기능은 없습니다.
관리자는 이 탭에서 활성화 스위치·스케줄(Cron)·수신자(쉼표 구분)를 저장하며, 스케줄은 프리셋(월 09:00 / 금 18:00 / 매일 09:00)이나 초를 포함한 6 필드 cron(예: 0 0 9 * * MON)으로 정합니다. 저장한 스케줄은 1 분 안에 적용되고, [지금 발송]은 지금 수신자 입력란의 주소로 곧바로 보냅니다. 리포트 탭은 ADMIN 이 아니면 읽기 전용이며, 메일이 오지 않으면 SMTP 설정(11.4)과 수신자 주소를 확인하세요.

11. 설정 관리

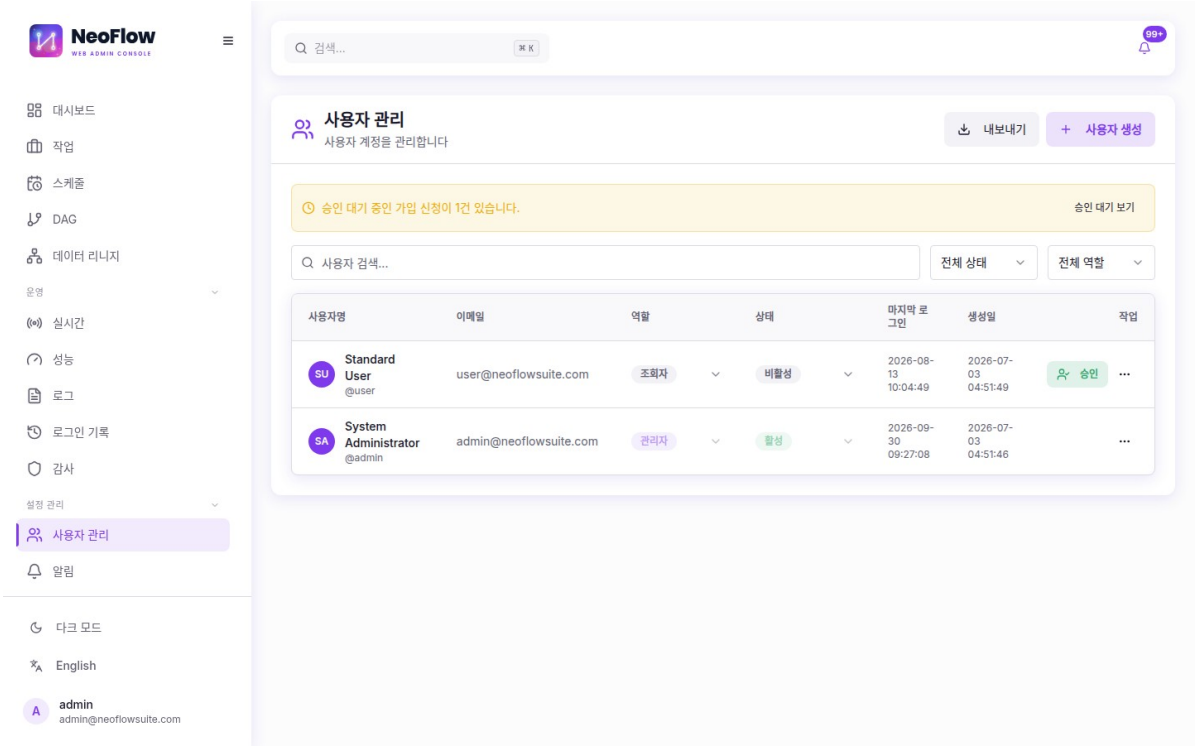
설정 관리 장에서는 사이드바 '설정 관리' 그룹의 화면 — 사용자 관리, 알림, 경고 규칙, 설정, 공통 설정, 웹훅, API 토큰 — 을 다룹니다. 알림·경고 규칙(조회)·설정·공통 설정(조회)·API 토큰은 모든 사용자가 쓰고, 사용자 관리·웹훅은 ADMIN 전용입니다.

11.1 사용자 관리 (ADMIN)

[사용자 관리(Users)]는 계정을 등록·수정·삭제하고 역할(관리자/매니저/조회자)과 상태(활성/비활성/정지)를 테이블에서 인라인으로 바꾸는 ADMIN 전용 화면입니다. 사용자 API 와 CSV 내보내기에는 역할이 ADMIN/MANAGER/USER, 상태가 ACTIVE/INACTIVE/SUSPENDED 저장 코드로 나옵니다. 역할·상태 배지 옆 드롭다운으로 모달 없이 즉시 변경할 수 있고, 이름·이메일 검색과 역할·상태 필터로 사용자를 빠르게 찾을 수 있습니다. username 은 변경할 수 없습니다. 우상단 [내보내기]로 현재 사용자 목록을 CSV 로 내려받을 수 있습니다.

사용자가 [설정 > 계정]에서 스스로 계정을 삭제하면 목록에 '삭제됨'으로 표시되고(상태 필터로 조회 가능), 이 계정의 상태는 바꿀 수 없습니다. 삭제된 계정은 라이선스의 사용자 수에 포함되지 않지만 비활성·정지 계정은 포함됩니다. 정지된 계정으로 올바른 비밀번호를 넣어 로그인하면 "정지된 계정입니다. 관리자에게 문의하세요."가 표시되며, 이 시도는 로그인 실패 횟수에 들어가지 않습니다.

역할을 바꾸거나 상태를 비활성·정지로 바꾸면 대상 사용자의 로그인이 즉시 모두 끝나고 API 토큰도 인증되지 않으며, 다시 로그인하면 새 권한이 적용됩니다. 행 ⋯ 메뉴의 [비밀번호 재설정]은 관리자가 새 비밀번호를 정해 주는 기능으로, 대상 사용자는 모든 곳에서 로그아웃되고 로그인 잠금이 풀리며 다음 로그인 때 비밀번호 변경 화면을 거칩니다(메일은 발송되지 않으므로 새 비밀번호는 직접 전달). 로그인한 ADMIN 본인 행은 역할·상태 드롭다운이 비활성이고 비밀번호 재설정·삭제 메뉴가 없으며, 활성 ADMIN 이 0 명이 되는 변경(마지막 ADMIN 의 강등·비활성화·삭제)은 거부됩니다.



◢ *사용자 관리 — 승인 대기 배너 · [승인] 버튼 · 인라인 역할/상태 변경*

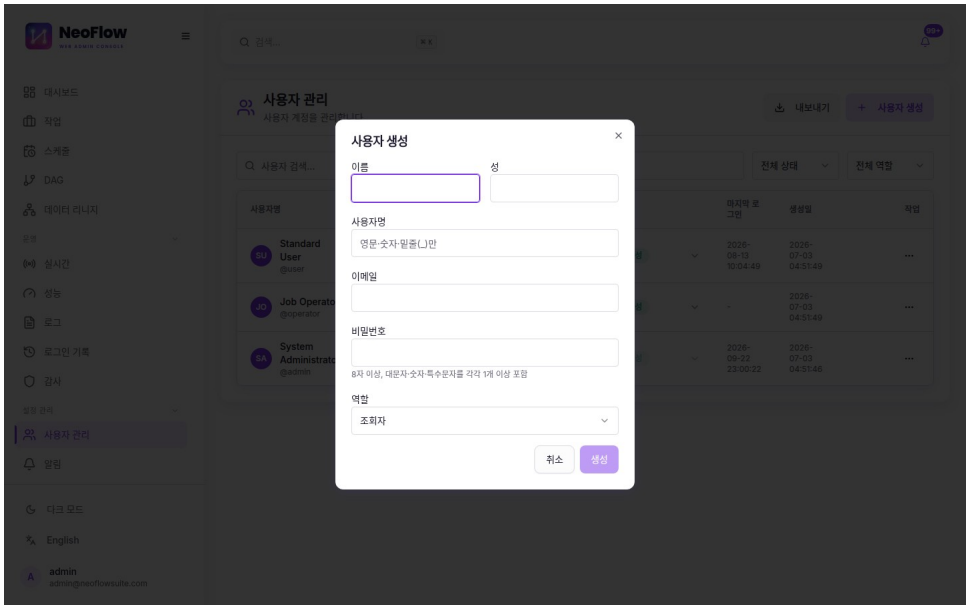
회원가입으로 들어온 계정은 이 화면에 INACTIVE 상태로 쌓입니다. 승인을 기다리는 계정이 있으면 목록 위에 대기 건수를 알리는 노란 배너가 나타나고, 배너의 [승인 대기 보기]를 누르면 목록이 대기 계정만으로

좁혀집니다. INACTIVE 행에는 초록색 [승인] 버튼이 함께 표시되며, 이 버튼을 누르면 상태가 ACTIVE 로 바뀌어 해당 사용자의 로그인이 열립니다. 승인은 상태를 ACTIVE 로 바꾸는 것과 같은 동작이므로, 상태 배지 옆 드롭다운에서 직접 ACTIVE 를 선택해도 결과는 같습니다.

> **참고** INACTIVE 는 '아직 승인하지 않은 가입 신청'과 '관리자가 비활성화한 기존 계정' 두 가지를 함께 나타내며 목록에서는 구분되지 않습니다. 승인 전에는 가입일·마지막 로그인 열을 함께 확인해 어느 쪽인지 판단하세요. 승인 직후 역할은 조회자(읽기 전용)이므로, Job 을 다뤄야 하는 사용자는 역할도 매니저 이상으로 함께 올려야 합니다.

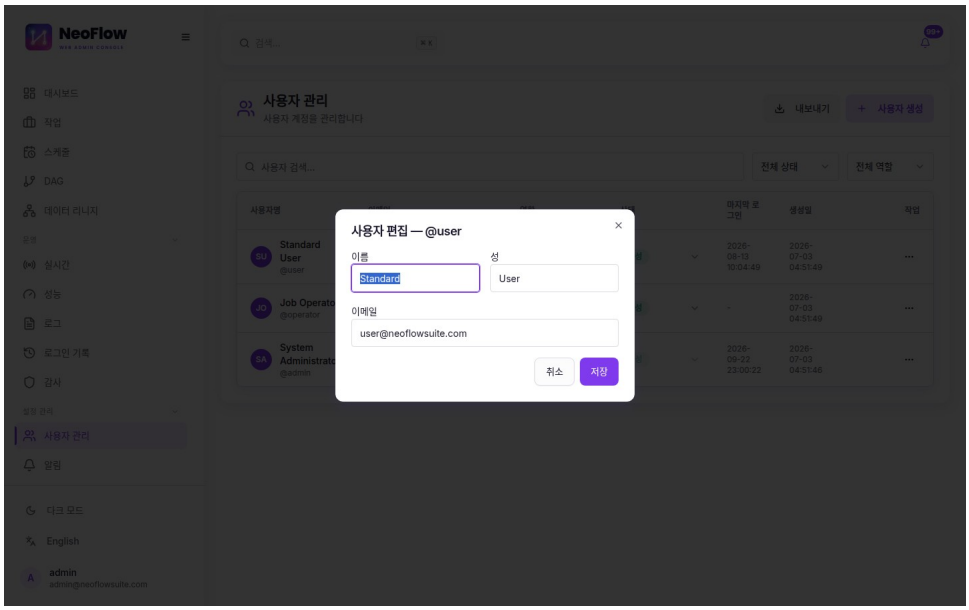
레이어팝업(모달) 사용자 생성·편집·비밀번호 재설정·삭제는 모두 모달에서 처리합니다(ADMIN 전용).

▸ 사용자 생성



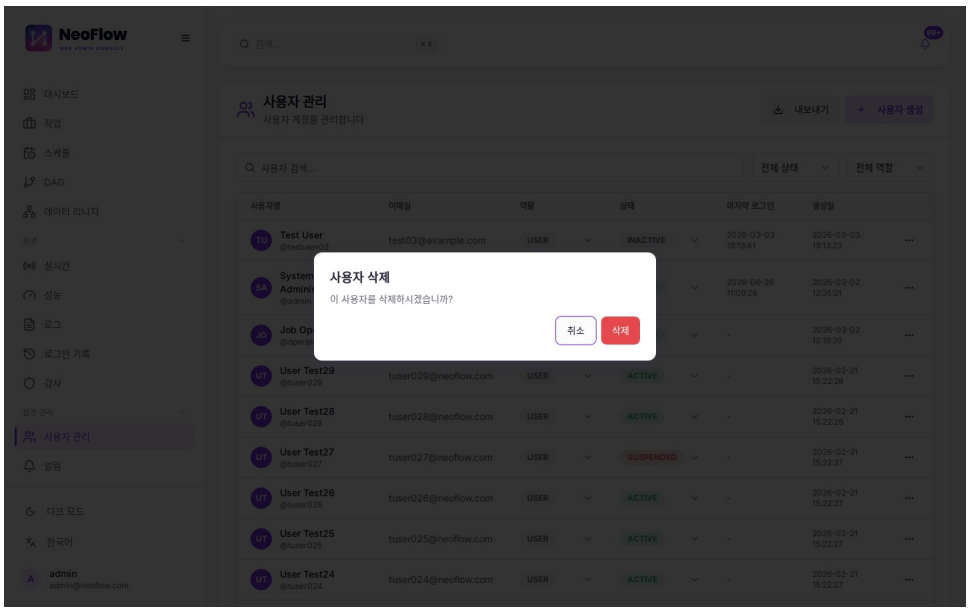
- 헤더 [+ 사용자 생성] 버튼으로 엽니다.
- 이름·성·사용자명(영문·숫자·밑줄만)·이메일·비밀번호·역할(관리자/매니저/조회자)을 입력합니다.
- 비밀번호는 8 자 이상에 대문자·숫자·특수문자를 각 1 개 이상 포함해야 하며, 정책을 만족해야 [생성] 버튼이 켜집니다.
- 관리자가 만든 계정은 승인 절차 없이 바로 활성 상태로 추가되고, 첫 로그인 때 비밀번호 변경 화면을 거칩니다.

▸ 사용자 편집



- 행 우측 ··· 메뉴 → [편집]으로 엽니다.
- 이름·성·이메일을 수정합니다(사용자명은 변경 불가).

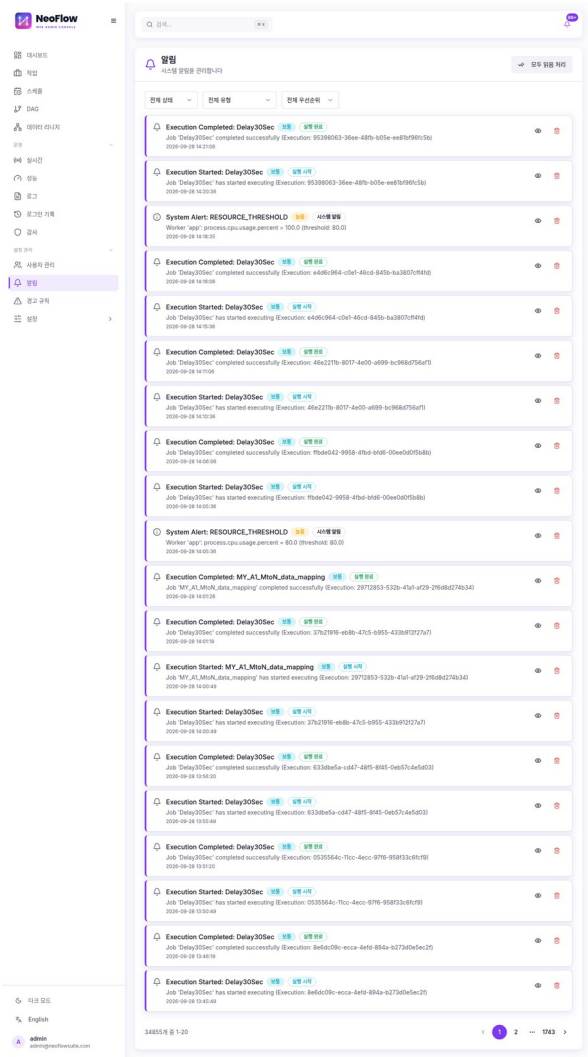
▸ 사용자 삭제



- 행 우측 ··· 메뉴 → [삭제]로 엽니다(로그인한 ADMIN 본인 행에는 없음).
- 확인 후 영구 삭제되며 되돌릴 수 없습니다. 그 사용자의 로그인이 즉시 끝나고 API 토큰도 함께 삭제됩니다.

11.2 알림

[알림] 화면은 본인에게 온 알림과 전 사용자 대상 알림(실행·Job 변경·시스템 경고)을 한곳에서 확인하고 읽음 처리·삭제하는 공간입니다. 상태(전체/안 읽음/읽음)·종류·우선순위(LOW/MEDIUM/HIGH/URGENT) 세 가지 필터로 좁혀 볼 수 있고, 안 읽은 알림은 좌측 파란 막대로 강조됩니다. 우상단 [모두 읽음 처리]로 한 번에 정리할 수 있습니다. 고른 종류는 주소(?type=)에 남아 새로고침·공유해도 유지되며, 알림 벨 드롭다운의 묶인 알림을 누르면 그 종류로 걸러진 채 열립니다.



◢ *알림함 — 상태/종류/우선순위 필터 + 카드 목록*

종류	언제 생기나
Execution Started / Completed	실행 시작 · 완료
Execution Failed	실행 실패. 타임아웃으로 끝난 실행도 이 종류로 오며 제목이 'Execution Timed Out: <Job>'
Execution Cancelled	사용자 취소 · 일괄 취소 · DAG 실행 취소 · Job 비활성화 · 인스턴스 종료로 인한 취소
Job Created / Updated / Deleted	누가 바꿨든 모든 사용자의 목록에 쌓임 (우선순위 LOW)
System Alert	경고 규칙·알림 임계값 경고. 계정 잠금 알림은 활성 ADMIN 에게만

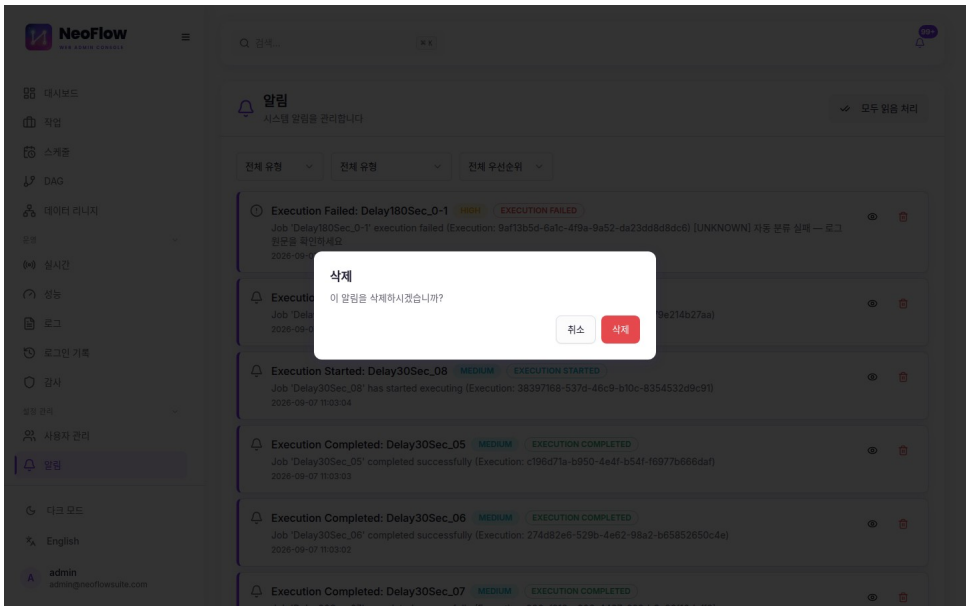
읽음 처리와 삭제는 누른 사용자에게만 적용되어, 전 사용자 대상 알림도 다른 사용자의 목록과 읽음 상태는 그대로입니다. 각 카드의 🗑 삭제 버튼은 확인 대화상자를 거치며, 삭제한 알림은 본인 목록에서 복구되지 않습니다. 알림은 보관 기간(기본 90 일, 설정 > 스토리지 탭)이 지나면 매일 03:45 에 자동으로 지워집니다. 새로 만든 계정에는 계정을 만들기 전에 생긴 알림이 읽음으로 보입니다. DAG 실행 알림은 이 목록에 쌓이지 않고 웹훅으로만 나가며, 알림 종류별 음소거 기능은 없습니다.

상단 탑바의 알림 벨(종 모양) 아이콘은 알림함으로 들어가지 않고도 새 알림을 빠르게 확인하는 입구입니다. 빨간 배지로 안 읽은 알림 개수를 보여주고, 클릭하면 최근 알림을 미리보기로 펼치며, [전체 알림 보기]를 누르면 위 알림 화면으로 이동합니다. 전 사용자 대상 알림은 도착하는 즉시 배지가 늘고, 한 사용자에게만 온 알림(계정 잠금 등)은 30 초마다 다시 세는 배지 갱신 때 나타납니다.

화면 우측 상단에 뜨는 실시간 토스트 팝업은 기본값이 꺼짐입니다. 분 단위 스케줄이 있는 환경에서는 실행마다 팝업이 쌓여 헤더 버튼을 가리고 클릭까지 먹어 버리기 때문입니다. 팝업으로도 즉시 보고 싶다면 알림 벨 드롭다운 하단의 [실시간 토스트 알림] 스위치를 켜십시오. 꺼져 있어도 알림은 그대로 수신·기록되고 알림 벨 배지의 안 읽음 개수도 계속 갱신되며, 사라지는 것은 우측 상단 팝업뿐입니다. 이 설정은 본인 브라우저에 저장되어 새로고침·재로그인 후에도 유지되고, '저장 완료·실패' 같은 작업 결과 토스트는 이 스위치와 무관하게 항상 표시됩니다.

레이어팝업(모달) 알림 카드의 삭제는 확인 대화상자를 거칩니다.

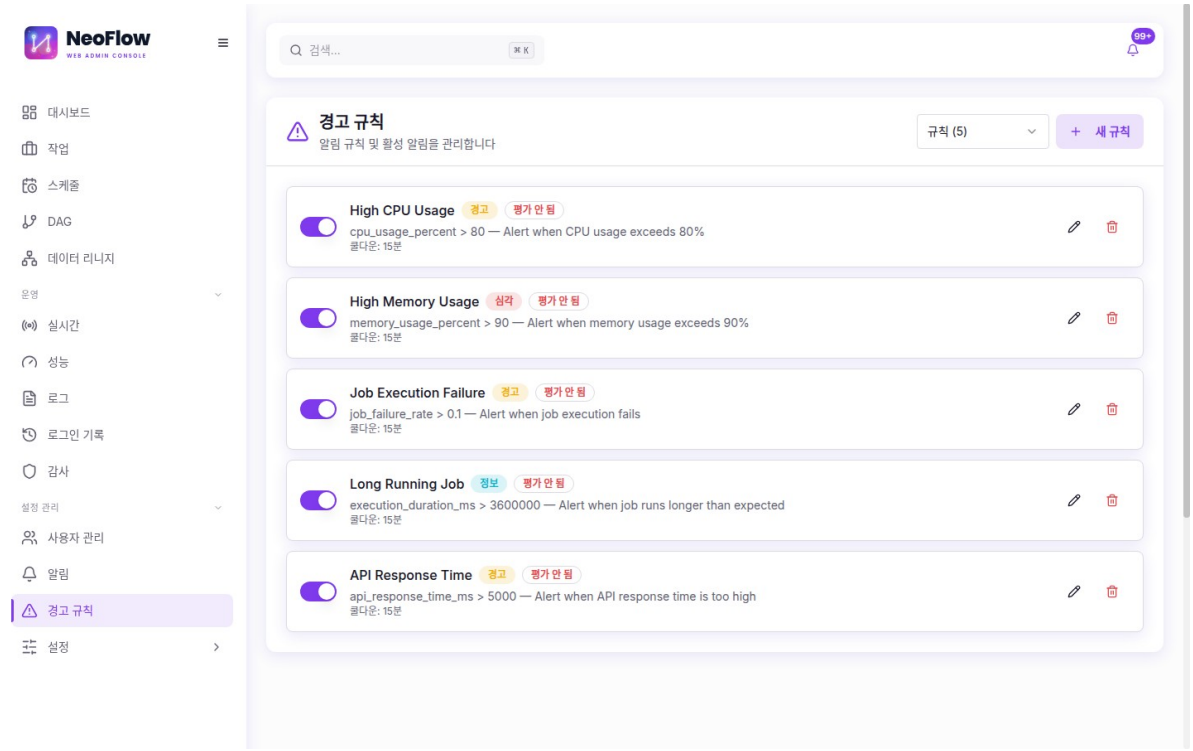
▸ **알림 삭제 확인**



- 카드의 휴지통 아이콘으로 엽니다.
- 삭제한 알림은 본인 목록에서 영구히 사라집니다(다른 사용자의 목록에는 남음). 목록만 정리하려면 [모두 읽음 처리]를 먼저 고려하세요.

11.3 경고 규칙

[경고 규칙(Alerts)]은 지표가 조건을 만족하면 알림·웹훅·메일을 보내는 규칙을 관리하는 화면입니다. 모든 사용자가 규칙과 활성 경고를 보고 경고를 확인(Ack)할 수 있으며, 규칙 작성·수정·삭제·켜고 끄기와 경고 해결(Resolve)은 ADMIN 만 할 수 있습니다(ADMIN 이 아니면 해당 버튼이 보이지 않고 활성 토글은 비활성으로 표시). 규칙 카드마다 활성 토글·심각도 배지(정보/경고/심각)·조건 요약(지표 비교기호 임계값)·쿨다운(분)과 실패 관련 지표의 집계 구간(분)이 표시되며, 지표나 조건이 지원 목록에 없는 규칙에는 '평가 안 됨' 배지가 붙습니다. 뷰 전환으로 등록된 규칙과 실제로 울린 활성 경고를 오가며 볼 수 있고, 활성 경고는 '활성'이면 [확인]·[해결], '확인됨'이면 [해결] 버튼을 보여줍니다.



◢ 경고 규칙 — 규칙 목록 / 활성 경고 + 규칙 폼

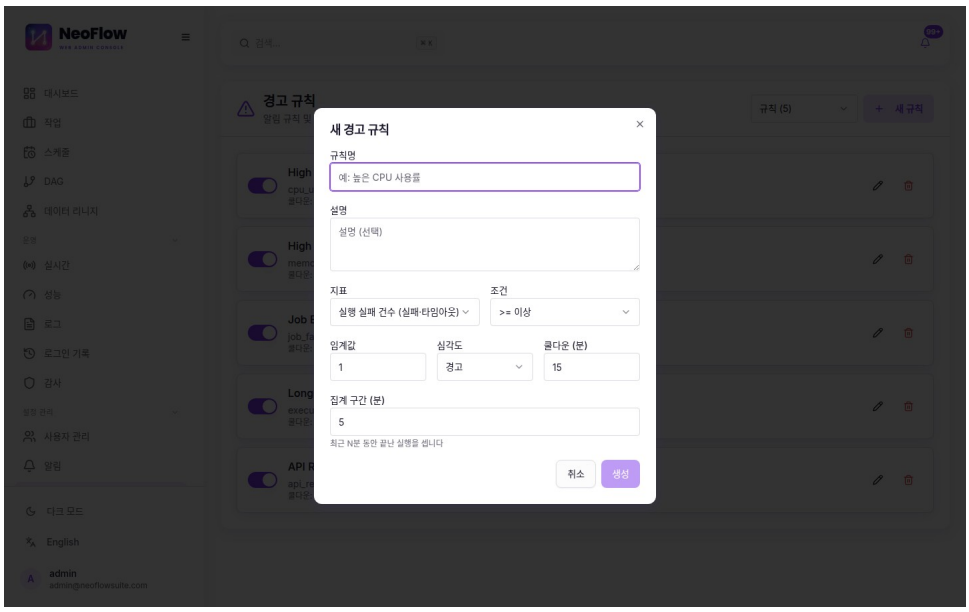
켜진 규칙은 1 분마다 평가됩니다. 지표는 실행 실패 건수(execution_failures)·실행 실패율(execution_failure_rate)·실행 대기 건수(queue_size)·가장 오래 실행 중인 Job 분(long_running_minutes)·CPU 사용률(cpu_usage, 규칙을 평가한 인스턴스의 최근 15 초 평균)·Heap 사용량 MB(heap_used_mb) 6 종입니다. 조건이 맞고 쿨다운이 지났으면 활성 경고를 만들고, 모든 사용자의 알림 목록(System Alert, 제목 'Alert: <규칙명>')·SYSTEM_ALERT 웹훅·활성 ADMIN 전원의 메일로 보냅니다. 쿨다운은 마지막으로 울린 시각부터 세므로 확인·해결해도 쿨다운 안에는 다시 울리지 않으며, HA 에서도 한 규칙은 쿨다운마다 한 번만 울립니다.

> **주의** [설정 > 알림 임계값] 탭의 CPU·메모리 경고는 경고 규칙과 따로 동작합니다. 같은 조건을 양쪽에 걸면 알림이 두 번 나가므로 둘 중 하나만 쓰세요. 규칙을 만들었는데 울리지 않으면 '평가 안 됨' 배지, 활성 토글, 쿨다운을 차례로 확인하세요.

규칙 삭제는 확인 대화상자를 거치며, 문구에 삭제할 규칙 이름이 들어가므로 대상이 맞는지 확인한 뒤 진행하십시오. 삭제한 규칙은 복구되지 않습니다. 잠시 멈추는 것이 목적이라면 삭제 대신 규칙 카드의 활성 토글을 끄십시오.

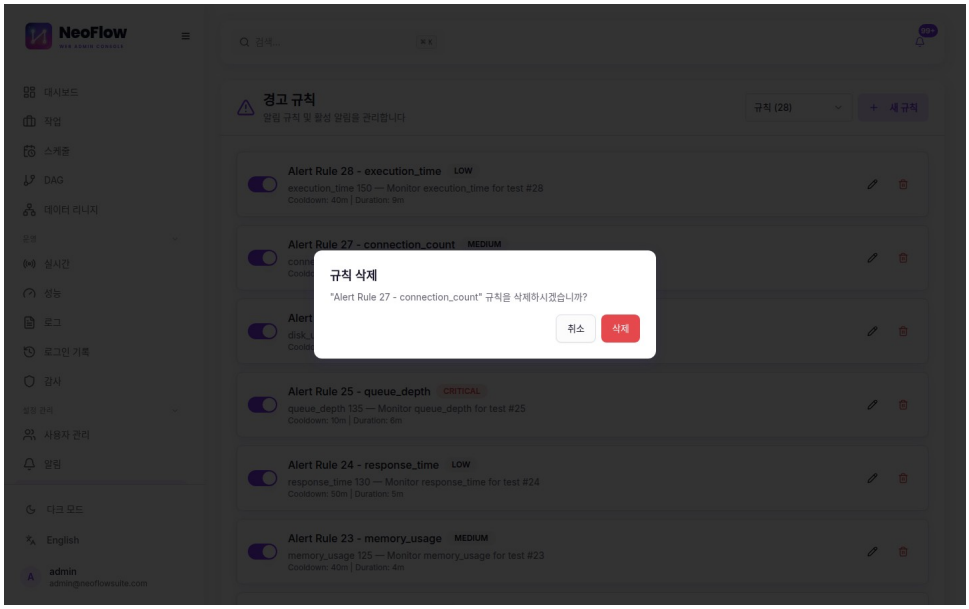
레이어팝업(모달) 경고 규칙 생성·편집은 지표·조건·심각도를 설정하는 모달에서 합니다(ADMIN).

▸ 경고 규칙 설정



- [+ 새 규칙] 또는 카드의 편집(연필)으로 엽니다.
- 이름·설명·지표(6 종 중 선택)·조건(> 초과, < 미만, >= 이상, <= 이하, = 같음, != 같지 않음)·임계치·심각도(정보/경고/심각)·쿨다운(분)을 입력하고, 실패 관련 지표면 집계 구간(분)도 입력합니다.
- 새 규칙의 기본값은 '실행 실패 건수 >= 1, 집계 구간 5 분, 쿨다운 15 분, 심각도 경고'입니다.
- 저장하면 켜진 규칙은 1 분 안에 평가가 시작됩니다.

▸ 규칙 삭제 확인

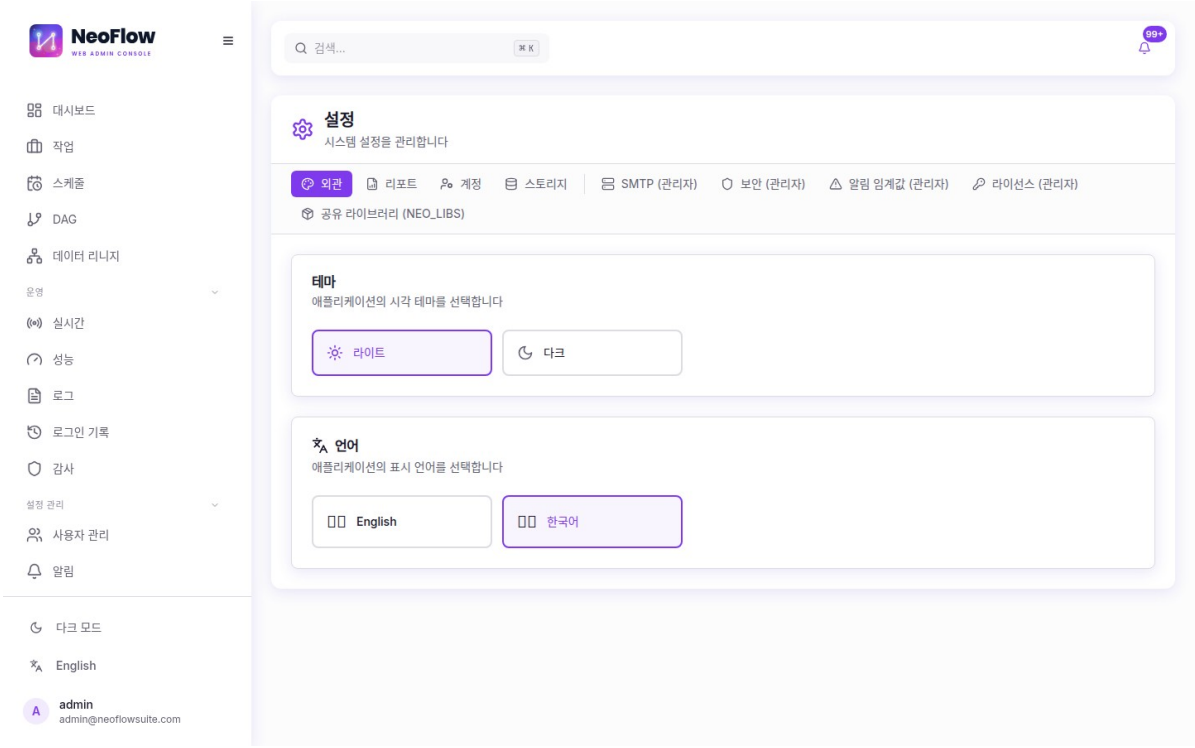


- 규칙 카드의 휴지통 아이콘으로 엽니다. 문구에 삭제할 규칙 이름이 들어갑니다.
- 삭제한 규칙은 복구되지 않습니다. 잠시 멈추는 것이 목적이면 활성 토글을 끄세요.

11.4 설정

[설정(Settings)] 화면은 외관·리포트·계정 같은 공통 탭, ADMIN 에게만 보이는 스토리지 탭, 그리고 구분선 뒤 관리자 전용 탭(SMTP·보안·알림 임계값·라이선스·공유 라이브러리)으로 구성됩니다. 외관 탭(테마·언어)은

누르는 즉시 적용되어 브라우저에 저장되고, 나머지 탭은 탭 안의 저장 버튼으로 각각 저장합니다(공통 저장 바는 없음). 리포트 탭은 10.9, 계정 탭(비밀번호 변경·계정 삭제)은 4.2 를 참고하세요.



◢ 설정 — 외관 탭(테마 · 언어) + 공통 탭 · 관리자 전용 탭

관리자 탭	내용
SMTP	메일 서버 host/port/계정/SSL. [SMTP 사용]을 켜면 모든 메일(실행·경고 알림, 주간 리포트, 테스트 메일)을 이 서버로 보내고, 끄면 서버 설정 MAIL_HOST 로 보내며, 둘 다 없으면 메일이 나가지 않음. 포트 465 는 [SSL/TLS]를 켜야 하고(끈 채 465 를 입력하면 화면에 경고), 587 은 SSL/TLS 를 끄고 STARTTLS 로 연결합니다. 연결은 10 초, 응답 읽기·쓰기는 각 30 초 안에 끝나지 않으면 실패로 처리합니다
보안	Rate limit(이 탭에서 켜고 저장해야 동작, 기본 분당 300·인증 60), 최대 실패 횟수(1~100)·잠금 시간(1~1440 분), 세션 유지 시간(5~10080 분, 로그인 시각부터 세는 access 토큰 수명). 저장 후 다음 로그인부터 적용되며 HA 는 30 초 안에 전 인스턴스에 반영
알림 임계값	활성화 스위치 + CPU 임계값(%)·메모리 임계값(MB, JVM heap)·알림 쿨다운(분). 인스턴스마다 60 초마다 판정해(CPU 는 최근 15 초 평균) 넘으면 System Alert 알림·웹훅. 실행 시간·실패율·큐 크기 경고는 경고 규칙(11.3)으로 만듦
라이선스 · 공유 라이브러리	아래 '라이선스'와 '공유 라이브러리 (NEO_LIBS)' 참조

스토리지 탭(ADMIN)의 보관 기간 카드에서는 실행 이력 보관 기간(일)과 알림 보관 기간(일, 0 이면 지우지 않음)을 버튼 하나로 함께 저장합니다. 보안·알림 임계값·보관 기간 카드는 한 번도 저장하지 않았으면 서버 배포 설정값이 적용되고 화면도 그 값을 보여줍니다.

> **참고** Job 타임아웃·재시도·동시 실행 수는 이 화면에서 바꾸지 않습니다. 타임아웃·재시도는 Job 별 값이, 동시 실행 상한은 서버 설정(MAX_CONCURRENT_JOBS, 기본 35)과 라이선스가 정합니다. Job 패키지의 저장 위치도 설치 시 서버 설정(STORAGE_TYPE·STORAGE_ROOT)으로 정해지며, 배포판 기본은 별도 MinIO 서버가 아니라 앱 서버의 파일 경로입니다(HA 구성이면 모든 노드가 같은 공유 마운트를 사용).

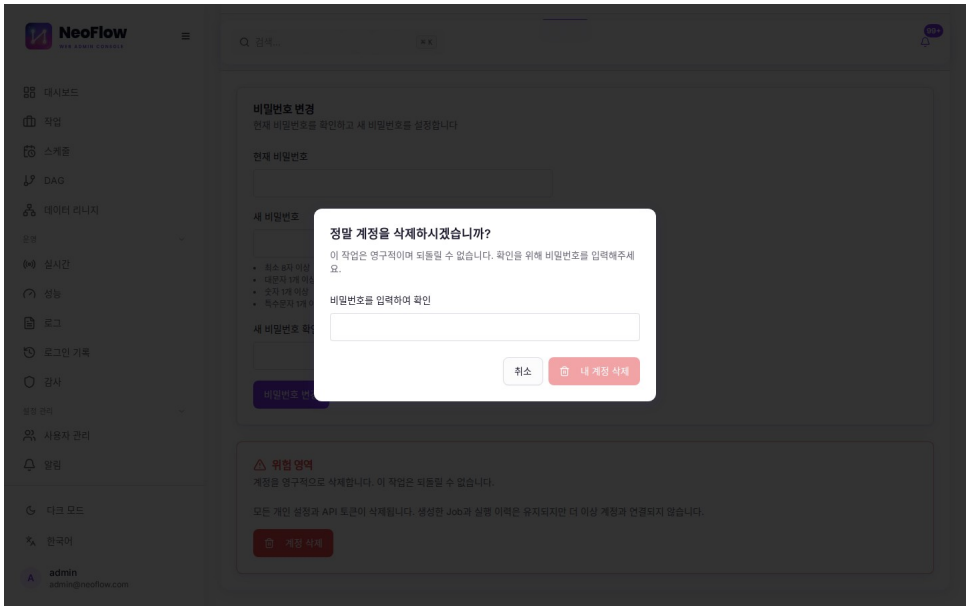
보관 기간 카드의 [지금 실행]은 저장이 아니라 삭제입니다. 보관 기간이 지난 실행 이력과 실행 로그를 즉시 지우며 되돌릴 수 없으므로 확인 대화상자를 한 번 거치고, 대화상자 문구에는 현재 보관 기간(일)이 그대로 들어갑니다. 완료 후에는 삭제 건수(실행 N 건, 로그 N 건)가 토스트로 표시됩니다. 알림은 이 버튼으로 지워지지 않고 매일 03:45 정리에서만 지워집니다.

> **참고** 실행 로그 본문이 쌓이는 job_execution_logs 테이블은 실행 이력과 외래키로 묶여 있지 않아 실행 이력을 지워도 함께 지워지지 않으므로, 같은 보관 기간으로 별도 정리됩니다.

> **참고** Job 을 삭제해도 패키지 파일(.zip)은 곧바로 지워지지 않습니다. 실행 이력이 그 파일을 직접 가리키고 있어, 함께 지우면 남아 있는 이력의 파일 다운로드가 깨지기 때문입니다. 파일은 Job 과 그 파일로 실행된 이력이 모두 사라진 뒤(보관 기간 만료)에야 회수 대상이 되며, 회수는 스케줄로 돌지 않고 ADMIN 이 API 로 실행합니다. POST /api/admin/storage/orphans 는 기본이 미리보기(dryRun=true)라 대상 목록과 건수만 돌려주고, ?dryRun=false 를 붙여야 실제로 삭제합니다. 업로드 도중에 실행하면 막 올라간 파일이 대상에 섞일 수 있으니, 업로드가 없는 시간대에 미리보기 결과를 확인한 뒤 실행하십시오.

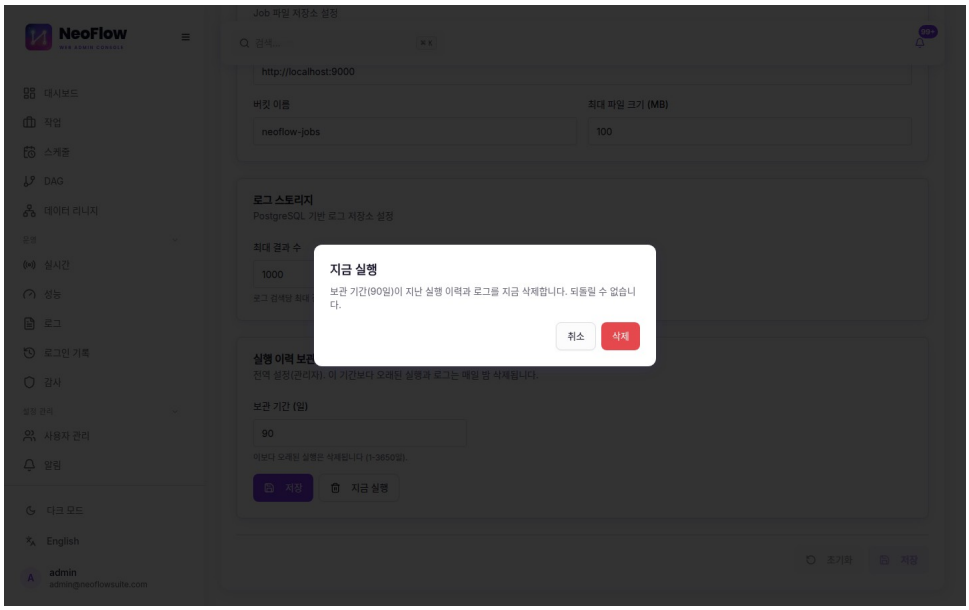
레이어팝업(모달) 되돌릴 수 없는 작업(계정 삭제 등)은 확인 모달을 거칩니다.

▸ 계정 삭제 확인



- 계정 탭의 [위험 영역 → 계정 삭제]로 엽니다.
- [비밀번호를 입력하여 확인] 칸에 비밀번호를 넣고 [내 계정 삭제]를 누르면 본인 계정이 삭제되어 원래 사용자명으로 다시 로그인할 수 없습니다. 관리자 화면의 사용자 목록에는 '삭제됨'으로 남으며 되돌릴 수 없습니다.
- 활성 ADMIN 이 본인뿐이면 삭제가 거부됩니다. 다른 ADMIN 을 먼저 만드세요.

▸ 보관 정리 즉시 실행 확인



- 스토리지 탭(ADMIN)의 보관 기간 카드에서 [지금 실행]으로 엽니다.
- 문구에 현재 보관 기간(일)이 그대로 들어가므로 며칠이 지난 데이터를 지우는지 확인하고 누르세요.
- 확인하면 보관 기간이 지난 실행 이력과 로그가 즉시 삭제되며 되돌릴 수 없습니다. 알림은 이 버튼으로 지워지지 않습니다.

라이선스

[설정 > 라이선스 (관리자)] 탭은 ADMIN 에게만 보이며, 라이선스 상태 확인·머신 지문 복사·라이선스 파일 업로드를 한곳에서 처리합니다. NeoFlow 라이선스는 전자서명된 .lic 파일로 제공되며, 서버는 기동할 때와 그 뒤 5 분마다 서명·만료일·하드웨어 지문을 다시 확인합니다.

화면 항목	내용
라이선스 상태	상태(유효 / 유예 기간 / 만료 / 하드웨어 불일치 / 라이선스 없음 / 서명 오류 / 비활성) · 고객사 · 에디션 · 만료일(기간 제한이 없으면 '영구') · 만료까지 남은 일수 · 강제 적용('강제 적용' 또는 '경고만')
용량 사용량	Job · DAG · 사용자 · 동시 실행 Job 의 현재 수와 라이선스 한도
라이선스 기능	라이선스에 포함된 기능 목록. 없으면 '활성화된 기능 없음'
머신 지문	이 서버의 하드웨어 지문. [지문 복사]로 복사해 라이선스 발급·갱신을 요청할 때 전달
라이선스 업로드	서명된 .lic 파일을 골라 [업로드]. 성공하면 "라이선스가 갱신되었습니다"가 표시되고 상태가 바로 다시 계산됨

라이선스를 새로 받거나 갱신하는 절차는 다음과 같습니다.

1. [머신 지문] 카드의 [지문 복사]를 눌러 지문을 복사하고, 발급 담당자(주식회사 위블리스)에게 전달합니다.
2. 받은 .lic 파일을 [라이선스 업로드] 카드에서 골라 [업로드]를 누릅니다. 서명이 맞지 않거나 읽을 수 없는 파일은 거부되며, 이때 기존 라이선스 파일은 그대로 유지됩니다.
3. [라이선스 상태]가 '유효'로 바뀌었는지, 만료일과 용량 한도가 계약 내용과 같은지 확인합니다.

라이선스에 주의가 필요하면 모든 사용자의 화면 위쪽에 배너가 나타납니다.

상황	배너 문구
강제 적용 중인데 라이선스가 없거나, 서명 오류·만료(유예 기간 경과)·하드웨어 불일치	“라이선스가 유효하지 않습니다 — 쓰기 작업이 차단되었습니다. 관리자에게 문의하세요.” (빨간 배너)
만료 후 유예 기간(기본 14 일) 중	“라이선스가 만료되어 유예 기간 중입니다. 곧 갱신해 주세요.”
만료 30 일 전부터	“라이선스가 N 일 후 만료됩니다.”

라이선스 문제로 서버 기동이 막히지는 않습니다(소프트 블록). 로그인과 조회는 언제나 허용되고, 차단되는 것은 생성·수정·삭제 같은 변경 요청뿐이며 이때 서버는 HTTP 402 로 응답하고 화면에는 위 빨간 문구가 토스트로 뜹니다. 차단 중에도 라이선스 탭의 업로드는 쓸 수 있으므로 유효한 파일을 올려 바로 풀 수 있습니다. 유예 기간 중에는 변경 요청도 허용됩니다.

> **참고** 변경 요청 차단은 서버 환경변수 NEOFLOW_LICENSE_ENFORCE=true 일 때만 동작하며, false 이면 ‘경고만’ 모드로 문제를 배너로 알리기만 하고 아무것도 차단하지 않습니다(경고만 모드에서는 라이선스가 없거나 유효하지 않은 사유를 서버가 보낸 영문 문구로 배너에 표시). 설치 스크립트는 설치할 때 라이선스 파일(config/license/neoflow.lic)이 없으면 이 값을 false 로 기록합니다. 라이선스를 업로드한 뒤 compose/.env 의 값을 true 로 바꾸고 docker compose up -d 로 앱 컨테이너를 다시 만들면 강제 적용이 켜집니다.

공유 라이브러리 (NEO_LIBS)

[설정 > 공유 라이브러리 (NEO_LIBS)] 탭은 여러 Job 이 함께 쓰는 라이브러리 파일(.jar, SAP JCo 같은 네이티브 .so)을 한곳에 모아 두는 공용 풀을 관리하며, ADMIN 에게만 보입니다. 풀에 올린 파일은 Job 을 실행할 때마다 그 실행의 작업 폴더로 내려받아지고, 그 경로가 환경변수 NEO_LIBS 로 Job 프로세스에 전달되어 classpath 에 포함됩니다(.so 파일은 LD_LIBRARY_PATH 로 찾음). 그래서 JDBC 드라이버처럼 여러 Job 이 쓰는 jar 를 Job 패키지마다 넣지 않아도 됩니다. 파일은 Job 패키지와 같은 오브젝트 저장소에 보관됩니다.

업로드 절차는 다음과 같습니다.

1. 탭의 파일 영역에 .jar·.so 파일을 끌어다 놓거나, 영역을 눌러 파일을 고릅니다. 여러 파일을 한 번에 고를 수 있습니다.
2. 다른 확장자 파일은 “.jar/.so 가 아닌 파일 N 개는 제외했습니다” 안내와 함께 빠지고, 나머지가 올라가면 “N 개 라이브러리를 업로드했습니다”가 표시됩니다.
3. 목록에서 파일명·크기·업로드한 사용자·업로드 시각을 확인합니다. 새 파일은 그 뒤에 시작되는 실행부터 쓰입니다.

파일당 최대 100MB 이며, 같은 파일명을 다시 올리면 풀에 있던 파일을 덮어씁니다. Job 업로드 때 패키지의 lib/ 폴더에서 자동 등록되는 jar 는 이미 같은 이름이 있으면 덮어쓰지 않으므로(6.2 참조), 공유 jar 를 새 버전으로 바꾸는 것은 이 탭에서만 할 수 있습니다. 삭제는 행의 휴지통 아이콘으로 하며, “"<파일명>" 을(를) 삭제할까요? 이 jar 를 참조하는 Job 실행에 영향을 줄 수 있습니다.” 확인 대화상자를 거칩니다.

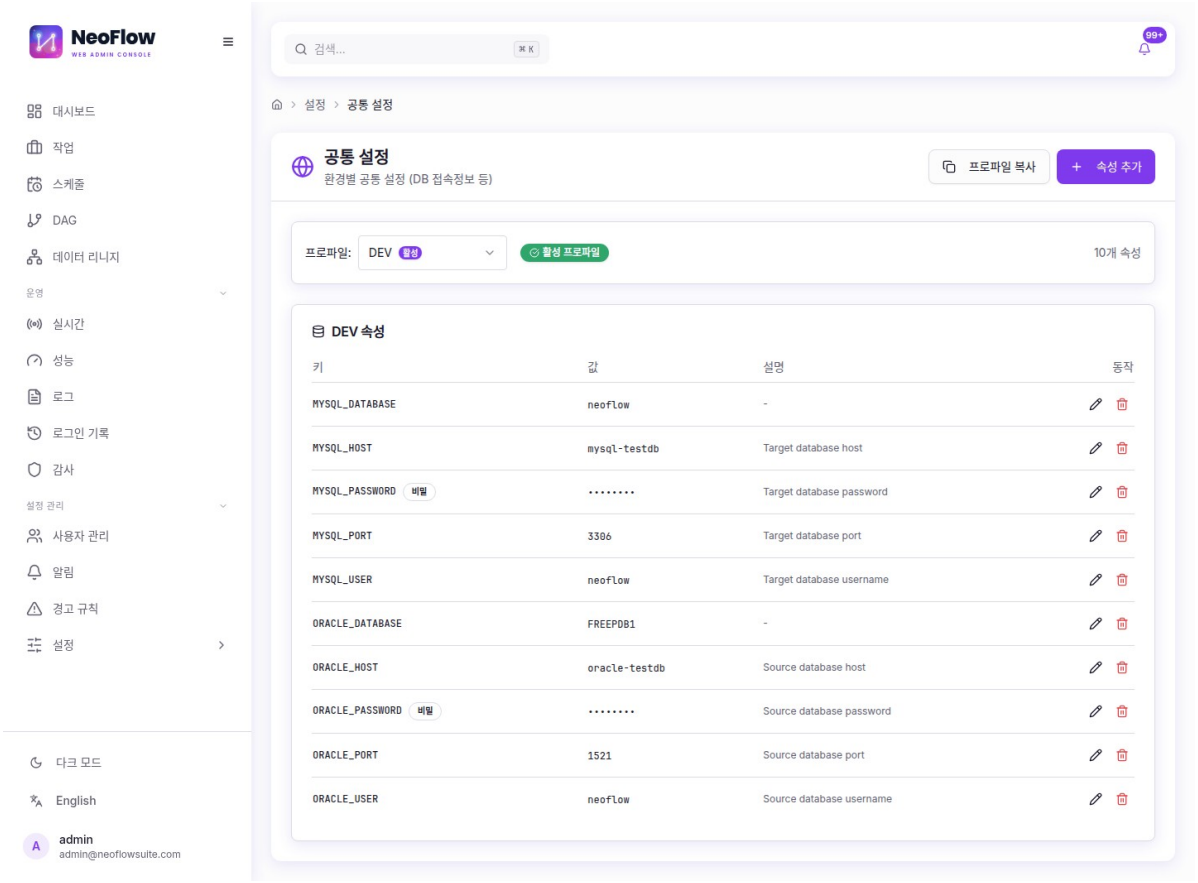
> **주의** 풀의 파일은 모든 Job 의 실행에 함께 실립니다. 공유 jar 를 교체하거나 삭제하면 그 뒤에 시작되는 모든 Job 실행에 곧바로 영향을 주므로, 그 라이브러리를 쓰는 Job 을 먼저 확인하고 실행이 적은 시간대에 바꾸십시오.

11.5 공통 설정 (Global Properties)

[공통 설정(Global Properties)]은 DEV/TEST/PROD 같은 프로파일별로 DB 접속정보 등 환경 변수를 키-값으로 관리하는 화면입니다. 모든 사용자가 조회할 수 있고(비밀 값은 가려짐), 매니저는 속성 추가·편집·삭제까지, ADMIN 은 활성 프로파일 전환·프로파일 복사를 포함한 전체를 할 수 있습니다. Job 실행 시 활성 프로파일의 값이 주입되며, 같은 키가 Job 파라미터에도 있을 때의 우선순위는 6.4 를 참조하세요. 프로파일을 통째로 복제해 새 환경을 빠르게 구성할 수도 있습니다.

속성은 [+ 속성 추가]나 행의 연필 아이콘으로 여는 다이얼로그에서 키(대문자 자동)·값·설명·비밀 토글을 설정하며, 기존 속성의 키(KEY)도 수정할 수 있어 저장 시 새 키로 rename 됩니다. 비밀 여부는 키 이름과 관계없이 이 토글로만 정합니다. 비밀 속성은 목록에서 비밀 배지와 함께 점으로 가려지고, 서버가 모든 역할에게 값을 가려서 보내므로 저장한 뒤에는 화면에서 값을 다시 볼 수 없습니다. 편집할 때 값 칸을 비워 두면 기존 값이 유지됩니다. DB 에는 enc:v1: 접두어와 함께 AES 로 암호화되어 저장됩니다(비-시크릿 값은 가독성을 위해 평문 유지).

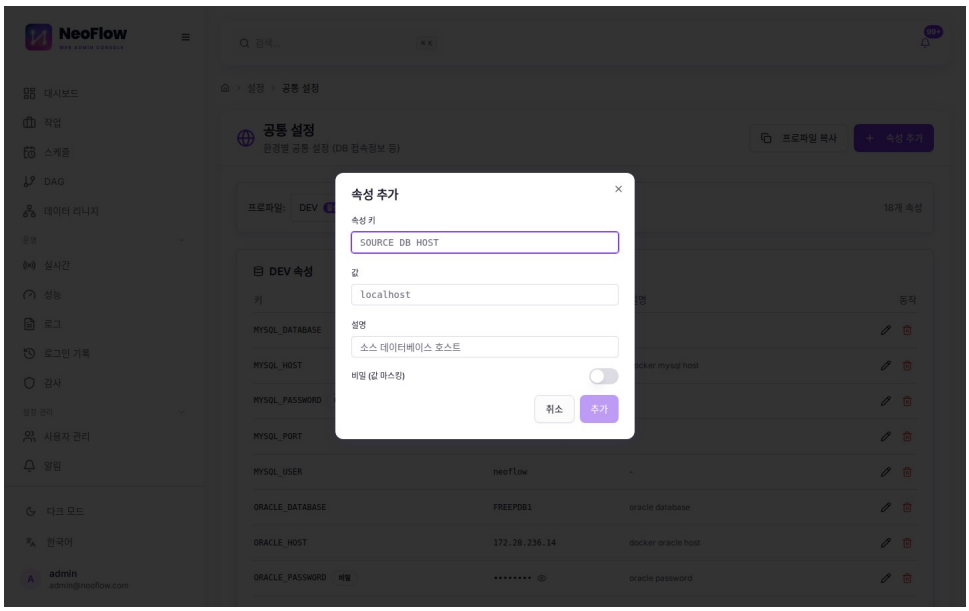
> **참고** 비밀 속성은 키 이름과 관계없이 실행 로그와 실행 API 응답에서도 ********로 가려집니다. 프로세스 명령줄에 실리는 실행 인자(--context_param)로는 넘어가지 않고 PARAM_* 환경변수로만 Job 에 전달되므로, 이런 키를 context 변수로 쓰는 Job 은 NeoFlowContext routine 이나 System.getenv 로 값을 받아야 합니다(6.4 참조).



▲ 공통 설정 — 프로파일별 키-값 + 비밀 마스킹

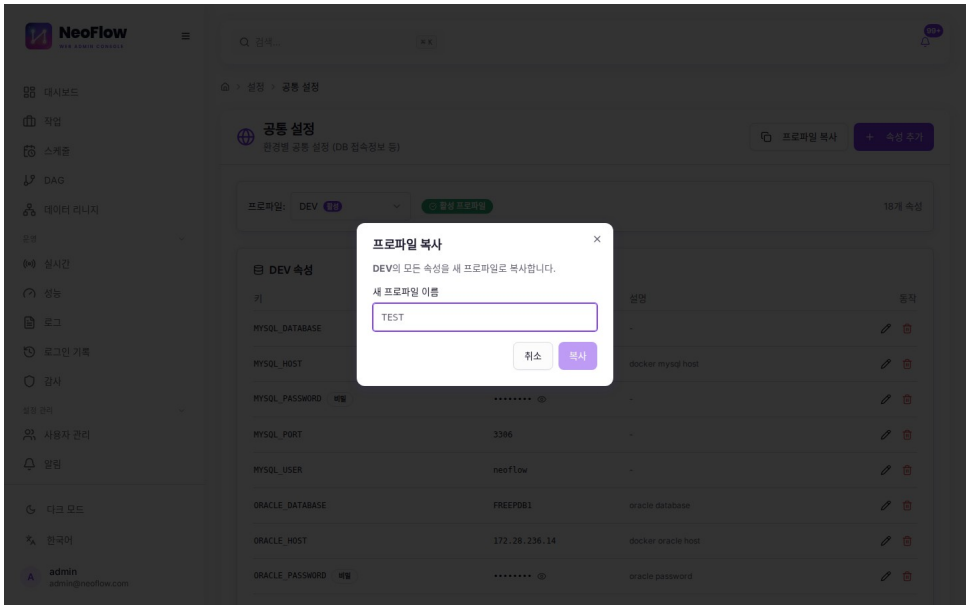
레이어팝업(모달) 속성 추가·편집과 프로파일 복사·삭제는 모달에서 처리합니다.

▸ 속성 추가·편집



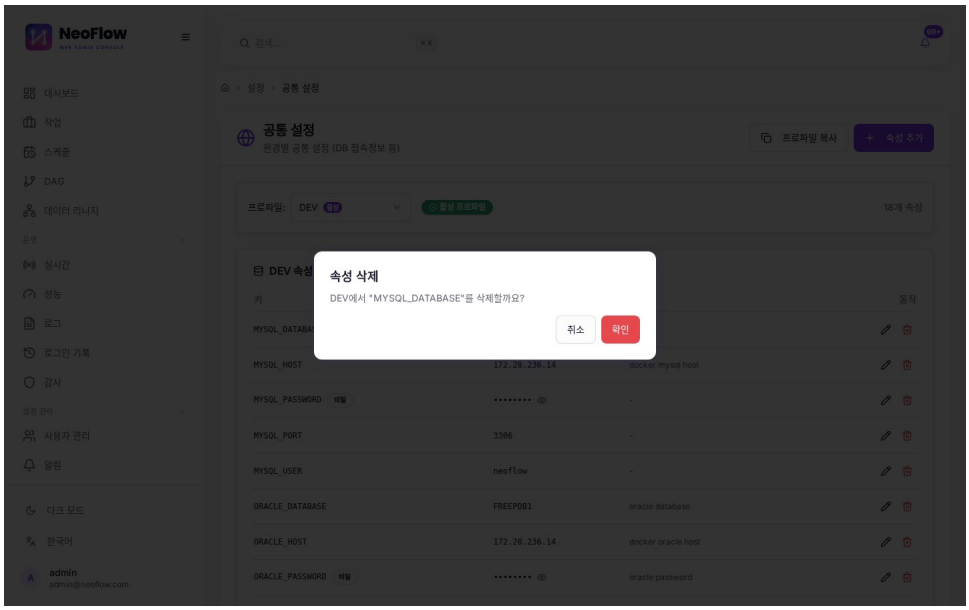
- [+ 속성 추가] 또는 행의 연필 아이콘으로 엽니다.
- 키(대문자 자동)·값·설명·비밀 (값 마스킹) 토글을 입력합니다. 비밀 여부는 키 이름이 아니라 이 토글로 정합니다.
- 비밀값은 목록에서 ••••••••로 가려지고, 실행 로그와 실행 API 응답에서도 가려집니다(11.5 참조).

▸ 프로파일 복사



- 헤더 [프로파일 복사]로 엽니다.
- 새 프로파일 이름을 입력하면 현재 프로파일의 속성 전체가 복제됩니다.

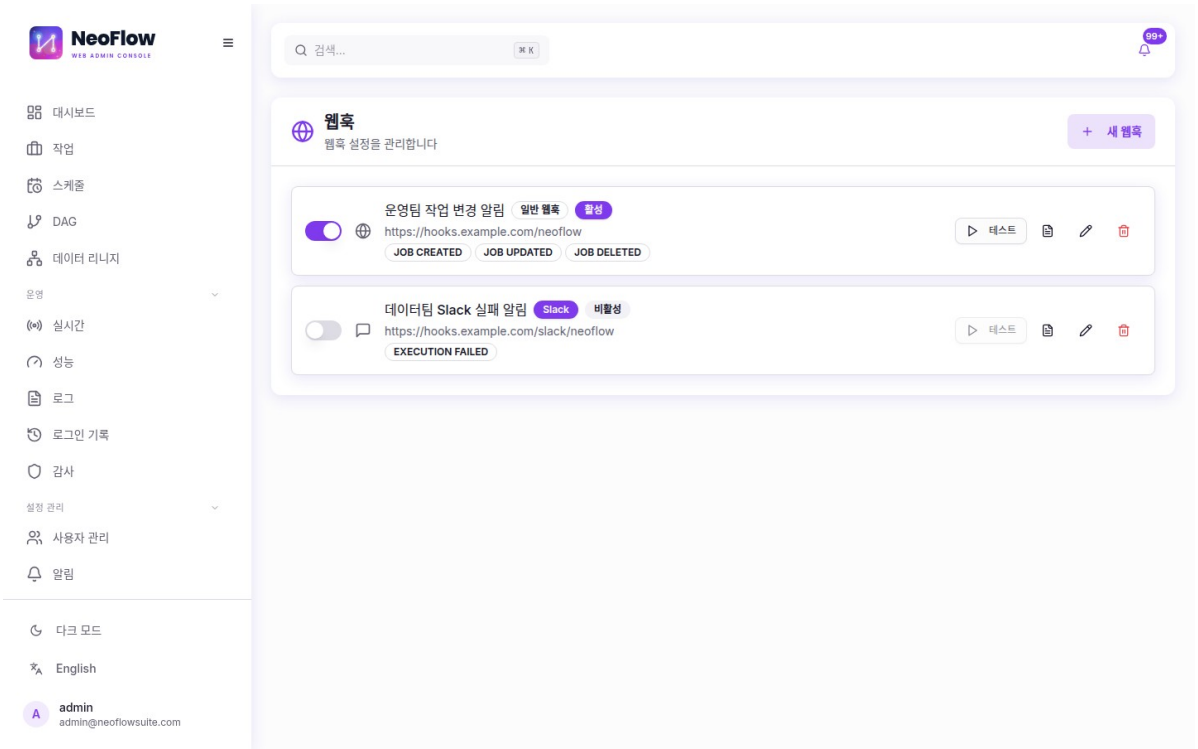
▸ 속성 삭제 확인



- 행의 [삭제](휴지통)로 엽니다.
- 대상 키·프로파일을 확인한 뒤 삭제합니다.

11.6 웹훅 (ADMIN)

[웹훅(Webhooks)]은 Job·실행·DAG 이벤트나 시스템 알림을 Slack 또는 일반 HTTP endpoint 로 push 하는 ADMIN 전용 통합 설정입니다. 카드마다 활성 토글·채널 배지·구독 이벤트 칩이 표시되고, 테스트 발송으로 즉시 연결을 확인하거나 발송 이력(응답 코드·시도 횟수·에러)을 조회할 수 있습니다. 일반 웹훅 채널에 시크릿을 입력하면 페이로드에 HMAC-SHA256 서명이 첨부됩니다.



▲ *웹훅 — Slack/일반 웹훅 채널 + 이벤트 구독 + 발송 이력*

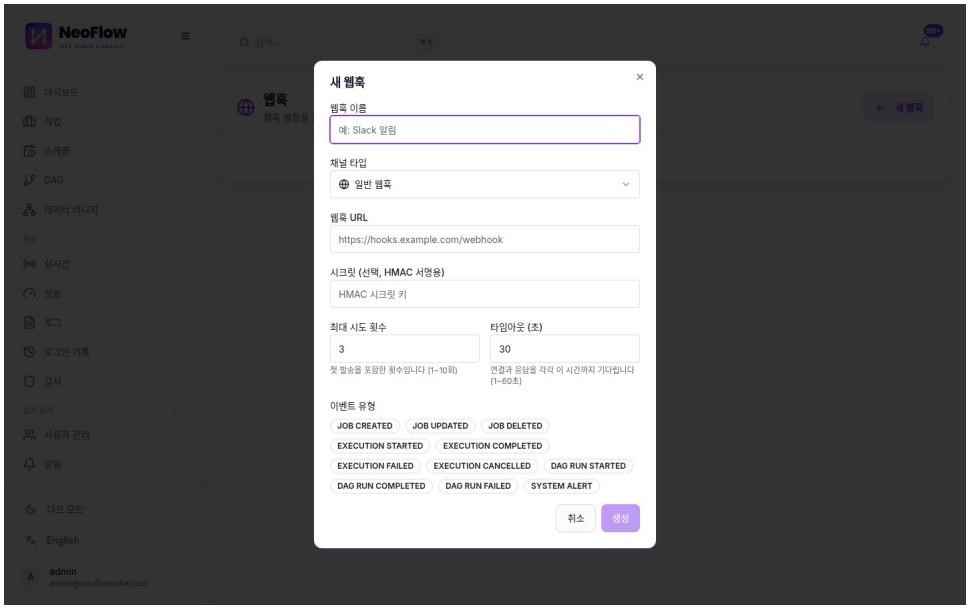
발송은 웹훅의 타임아웃(기본 30 초, 1~60) 동안 연결과 응답을 각각 기다리고, 넘기거나 실패하면 최대 시도 횟수(첫 발송 포함, 1~10)까지 재시도합니다. HA 에서는 재시도를 선점한 인스턴스 하나만 보냅니다. 응답이 늦는 수신측이 늦게라도 처리했다면 같은 이벤트를 두 번 받을 수 있으므로, 수신측은 받자마자 2xx 로 답하고 처리는 나중에 하게 하세요. 실행 취소 시에는 EXECUTION_CANCELLED 가, 타임아웃으로 끝난 실행은 status 가 TIMEOUT 인 EXECUTION_FAILED 가 발송되고, 실패 이벤트 payload 에는 장애 진단의 category·summary 가 붙습니다.

웹훅 URL 과 서명 키(secret)는 데이터베이스에 AES 로 암호화해 저장하며, 발송 이력의 오류 메시지에는 URL 을 가려서 남깁니다. URL 은 500 자, 서명 키는 255 자까지 입력할 수 있습니다.

웹훅 삭제는 확인 대화상자를 거치며, 문구에 삭제할 웹훅 이름이 들어갑니다. 확인하면 웹훅 설정과 발송 이력이 함께 사라지므로, 잠시 멈추는 것이 목적이라면 삭제 대신 카드의 활성 토글을 끄십시오.

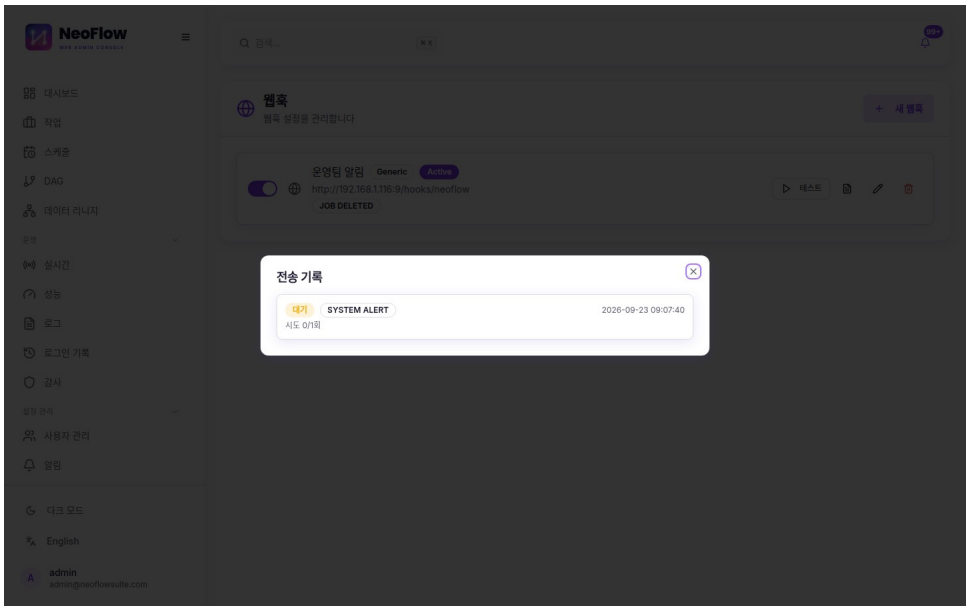
레이어팝업(모달) 웹훅 등록·편집과 배송 로그 확인은 모달에서 처리합니다.

▸ 웹훅 추가·편집



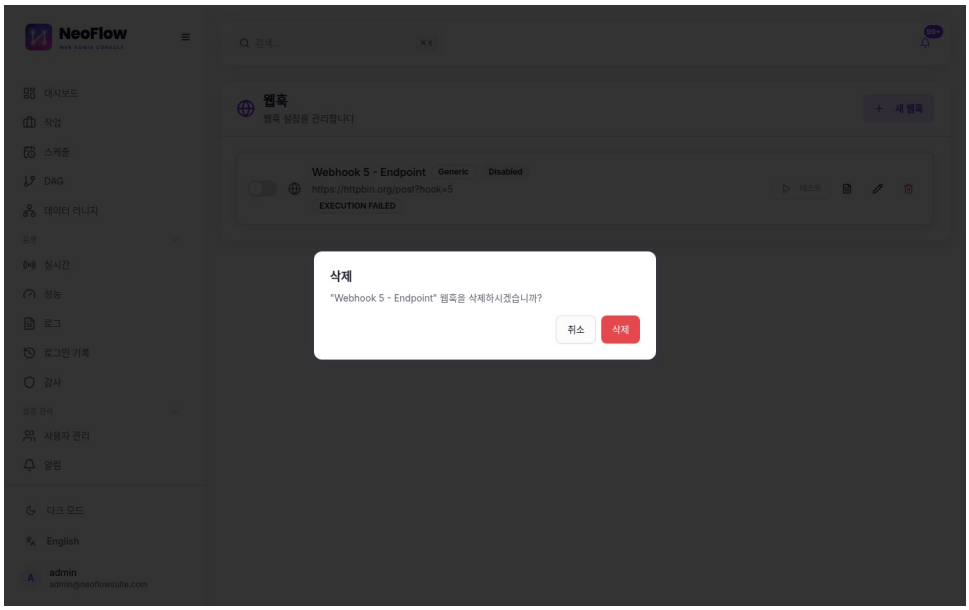
- 헤더 [웹훅 추가] 또는 카드 편집으로 엽니다.
- 이름·채널(일반 웹훅/Slack)·URL·시크릿(일반 웹훅 전용, HMAC 서명용)·최대 시도 횟수·타임아웃(초)을 설정합니다.
- 최대 시도 횟수는 첫 발송을 포함한 횟수로 1~10(기본 3 = 첫 발송 1 번 + 재시도 2 번), 타임아웃은 연결과 응답을 각각 기다리는 시간으로 1~60 초(기본 30)입니다. 범위 밖 값은 저장되지 않습니다.
- 구독할 이벤트 유형(JOB_*, EXECUTION_*, DAG_RUN_*, SYSTEM_ALERT)을 칩으로 고르며, 최소 1 개가 필요합니다.

▸ 발송 이력



- 카드의 [로그](문서 아이콘)로 엽니다.
- 항목별 상태(대기/성공/재시도 중/실패)·이벤트·HTTP 코드·시각·'시도 n/m 회'·에러를 확인합니다.
- 보내는 중인 첫 발송은 '대기'(시도 0/m 회)로 보입니다. 결과를 저장하지 못한 발송(DB 장애 등)은 '대기'로 남고 다시 보내지 않으므로, 오래 남아 있으면 수신측에서 도착 여부를 확인하세요.

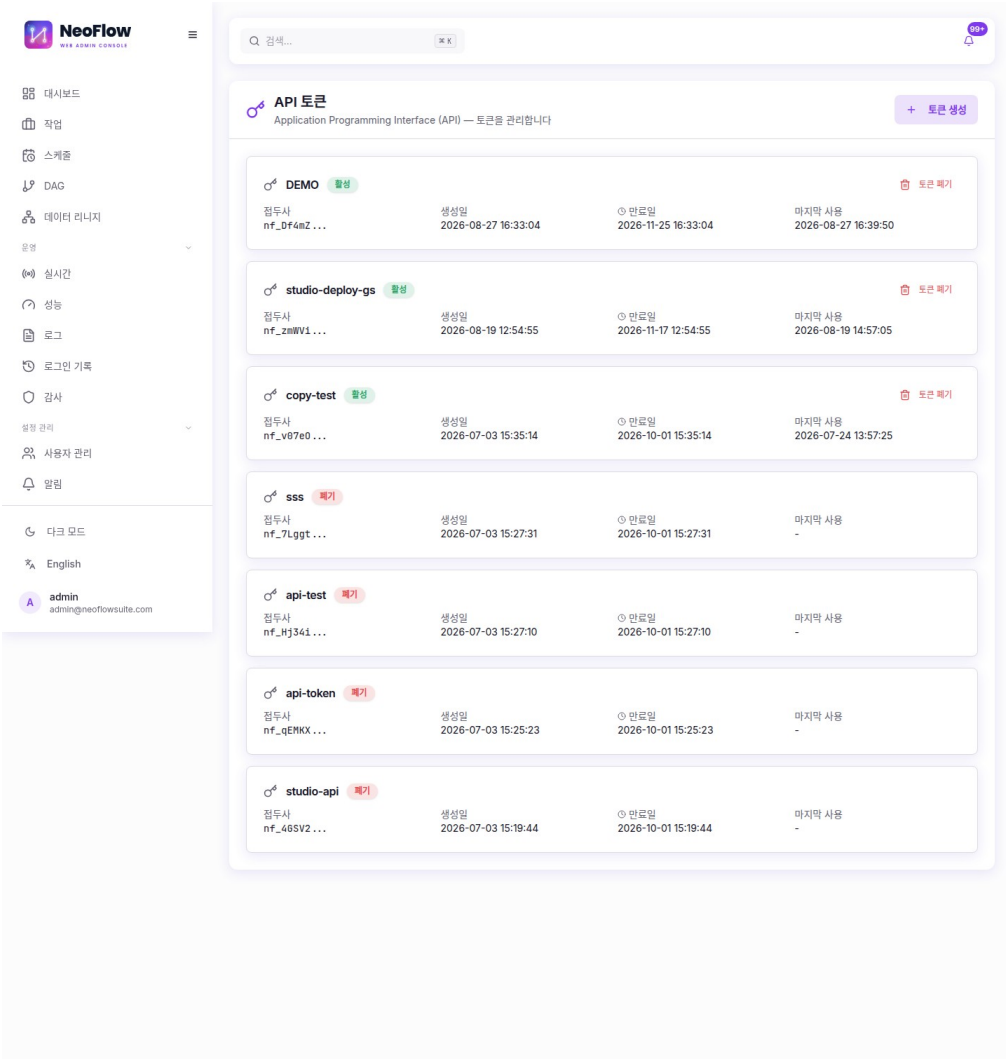
▸ 웹훅 삭제 확인



- 카드의 휴지통 아이콘으로 엽니다. 문구에 삭제할 웹훅 이름이 들어갑니다.
- 확인하면 웹훅 설정과 발송 이력이 함께 사라집니다. 잠시 멈추려면 토글을 끄세요.

11.7 API 토큰

[API 토큰]은 CI/CD 나 외부 스크립트에서 NeoFlow API 를 호출할 때 쓰는 인증 수단을 발급·관리하는 화면입니다. 모든 사용자가 쓸 수 있으며 각자 본인 토큰만 봅니다(ADMIN 도 다른 사용자의 토큰은 보거나 폐기할 수 없음). 토큰 이름과 만료 기간(30 일/60 일/90 일/만료 없음)을 정해 발급하며, 발급 직후 평문 토큰을 단 한 번만 보여주므로 그 자리에서 복사해 안전하게 보관해야 합니다. 카드에는 접두사·생성일·만료일·마지막 사용이 표시됩니다. 마지막 사용은 5 분에 한 번만 기록하므로 최대 5 분 늦게 보일 수 있습니다.



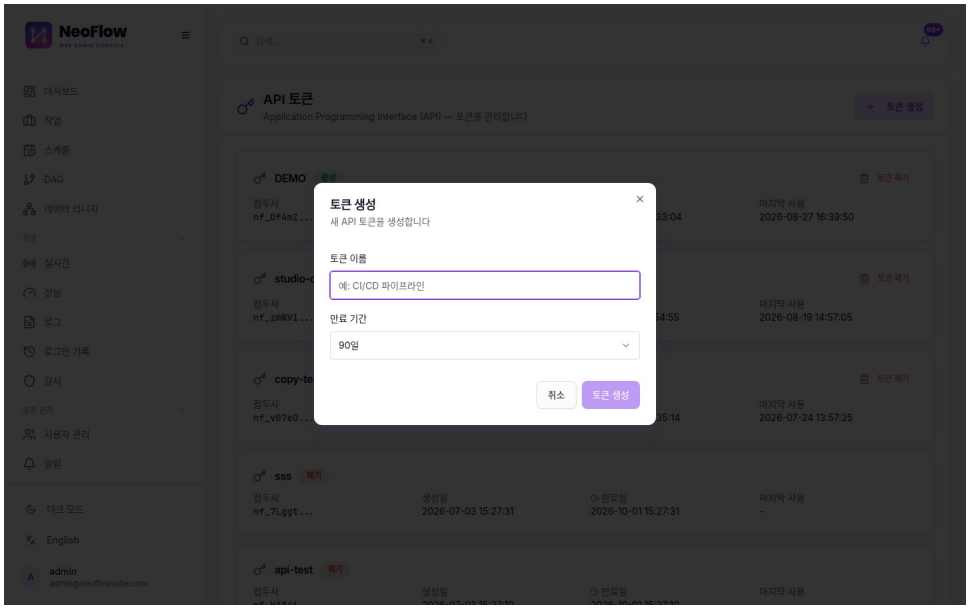
▲ API 토큰 — 발급 · 만료 · 1회 평문 노출

토큰에는 별도 권한 설정이 없고, 요청마다 소유자의 현재 역할로 동작합니다. 소유자 계정이 비활성·정지되면 그 사용자의 토큰은 인증되지 않고(401), 다시 활성이 되면 다시 통합니다. 관리자가 소유자를 삭제하면 토큰도 함께 삭제됩니다. 로그아웃·비밀번호 변경·비밀번호 재설정은 API 토큰을 끝내지 않습니다.

> **주의** 발급된 토큰 평문은 다시 볼 수 없습니다. 발급 직후 복사하지 못했다면 토큰을 폐기하고 새로 발급하세요. 만료 연장 기능은 없으므로 새 토큰으로 교체한 뒤 기존 토큰을 폐기합니다. 사람 계정에 묶인 토큰을 CI 에 쓰고 있다면 그 사람을 정지하기 전에 다른 계정의 토큰으로 바꾸세요.

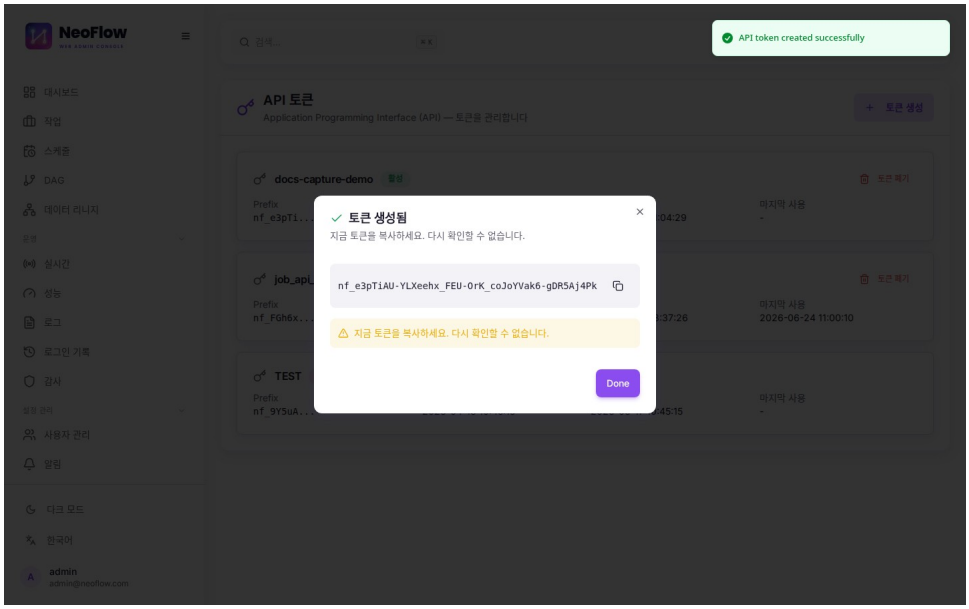
레이어팝업(모달) 토큰 발급·표시·폐기는 모달로 이뤄지며, 평문 토큰은 발급 직후 단 한 번만 보입니다.

▸ 토큰 생성



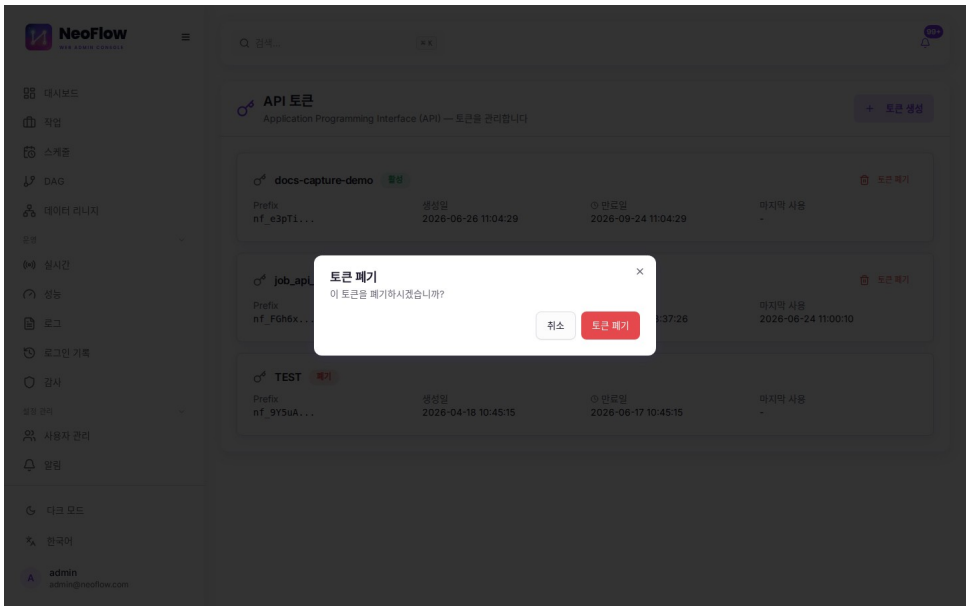
- 헤더 [+ 토큰 생성] 버튼으로 엽니다.
- 토큰 이름과 만료 기간(30 일/60 일/90 일/만료 없음)을 정해 발급합니다.

▸ 토큰 생성됨 (1 회 표시)



- 발급 직후 평문 토큰을 단 한 번만 표시합니다 — [완료]로 닫으면 다시 볼 수 없습니다.
- [복사]로 즉시 복사해 안전한 곳에 보관하세요. 분실 시 재발급이 필요합니다.

▸ 토큰 폐기



- 토큰 카드의 [토큰 폐기] 버튼으로 엽니다.
- 폐기 즉시 해당 토큰 인증이 차단되며 복구할 수 없습니다.

12. 설치·삭제·업그레이드

이 장은 NeoFlow 서버를 설치·업그레이드·삭제하는 방법과, 데이터를 백업·복구하고 이중화(HA) 구성으로 운영하는 방법을 다룹니다. 이 장의 작업은 서버의 root(또는 sudo) 권한과 Docker 사용 권한을 가진 시스템 관리자가 수행합니다. 명령은 오프라인 설치 패키지 neoflow-1.0.0-offline.tar.gz 를 기준으로 하며, 압축을 풀면 neoflow-1.0.0-offline 폴더(이하 설치 폴더)가 만들어집니다.

12.1 시스템 요구사항

NeoFlow 서버는 Docker 컨테이너로 제공됩니다. 앱·웹 화면·PostgreSQL·Redis 이미지가 모두 설치 패키지에 들어 있어 인터넷이 없는 폐쇄망에도 설치할 수 있습니다. 서버 한 대에 앱 하나를 띄우는 싱글 인스턴스와, 앱 인스턴스 2 개를 로드밸런서 뒤에 두는 HA(Active-Active) 구성 중에서 고릅니다.

하드웨어	싱글 인스턴스 (최소)	HA 구성 (권장)
CPU	4 코어	8 코어
메모리	8GB	16GB
디스크	50GB (설치 시 여유 10GB 이상)	100GB

소프트웨어·환경	요구 사항
운영체제	Ubuntu 20.04 LTS 이상, RHEL·Rocky Linux 8 이상 (x86_64)
컨테이너 런타임	Docker Engine 20.10 이상, Docker Compose v2 (docker compose 명령)
포함 구성요소	Java 17 런타임(앱 이미지), PostgreSQL 15, Redis 7.2, Nginx 1.30 — 별도 설치 불필요
시각 동기화	HA 구성은 모든 노드에 NTP(chrony 등) 필수 — 예약 시각과 타임아웃을 각 노드의 시계로 판정
웹 브라우저	최신 Chrome·Edge 또는 Firefox. 화면 해상도 1440×900 이상 권장
NeoFlow Studio (작성 PC)	Windows 10/11, JDK 11

Job 은 앱 서버의 자식 프로세스로 실행되므로, 동시에 실행하는 Job 수에 맞춰 앱 서버의 메모리를 잡습니다 (동시 실행 한도는 인스턴스마다 MAX_CONCURRENT_JOBS, 기본 35). 설치 후 서버가 여는 포트는 다음과 같으며, 설치 중에 바꿀 수 있습니다.

포트	용도
80	웹 화면과 API 접속 (싱글: 웹 화면 컨테이너, HA: Nginx 로드밸런서)
8080	앱 API 직접 접속 (싱글 인스턴스만 호스트에 열림)
5432	PostgreSQL
6379	Redis

> **참고** PostgreSQL(5432)과 Redis(6379)도 호스트 포트로 열립니다. 서버 밖에서 접속할 일이 없다면 방화벽으로 막으십시오.

12.2 설치

설치 패키지를 USB 나 내부 네트워크로 서버에 옮긴 뒤 압축을 풀고, 설치 폴더에서 install.sh 를 실행합니다.

```
tar xzf neoflow-1.0.0-offline.tar.gz

cd neoflow-1.0.0-offline

sudo ./install.sh
```

install.sh 는 먼저 패키지에 들어 있는 SHA256SUMS 로 모든 파일의 무결성을 확인합니다. 한 파일이라도 다르면 "패키지 무결성 검증 실패: 위 파일이 SHA256SUMS 와 다릅니다. 패키지를 다시 받으세요."를 출력하고 설치를 멈춥니다. 이때는 패키지를 다시 받아 옮긴 뒤 설치합니다.

라이선스 파일을 이미 받았다면 실행하기 전에 config/license/neoflow.lic 로 넣어 두십시오. 파일이 있으면 라이선스를 강제 적용하는 모드로, 없으면 경고만 하는 모드로 설치됩니다(11.4 라이선스 참조). 설치 스크립트는 다음 단계를 차례로 수행합니다.

단계	내용
사전 요구사항 확인	Docker Engine 20.10 이상, Docker Compose v2, Docker 데몬 실행, 설치 폴더의 디스크 여유 10GB 이상을 확인. 조건이 맞지 않으면 사유를 보여 주고 멈춤
배포 모드 선택	[1] 싱글 인스턴스(기본) 또는 [2] HA Active-Active(Nginx LB + 앱 2 대)
설정 입력	아래 표의 항목을 물음. Enter 를 누르면 괄호 안의 기본값(자동 생성값 포함)을 씀
환경 설정 · 이미지 로드	compose/.env 생성(권한 600), 비밀 값 암호화 키(NEOFLOW_ENCRYPTION_KEY) 자동 생성, Docker 이미지 로드
서비스 시작	선택한 모드로 컨테이너 기동. 앱이 처음 뜰 때 데이터베이스 스키마를 자동으로 만듦
서비스 상태 확인	PostgreSQL · Redis · 앱 · 웹 화면(HA 는 앱 2 대와 LB)이 정상(healthy)이 될 때까지 대기. 실패하면 로그 일부를 보여 주고 멈춤
관리자 비밀번호 적용	설정 단계에서 정한 비밀번호로 초기 관리자(admin)의 기본 비밀번호를 교체. 이후 기본 비밀번호로는 로그인되지 않음

설정 항목	기본값	설명
PostgreSQL 비밀번호	자동 생성	메타데이터 DB 접속 비밀번호
PostgreSQL 호스트 포트	5432	
Redis 비밀번호	자동 생성	토큰 무효화 목록·로그인 잠금 저장소 비밀번호
Redis 호스트 포트	6379	
오브젝트 저장 경로	/var/lib/neoflow/storage	Job 패키지(.zip)와 공유 라이브러리를 저장하는 호스트 폴더(STORAGE_ROOT). 백업 대상
JWT 시크릿	자동 생성 (64 자)	로그인 토큰 서명 키
Frontend 포트	80	웹 화면 접속 포트 (HA 는 Nginx LB 포트)
Backend API 포트	8080	싱글 인스턴스에서만 물음
관리자 비밀번호	자동 생성	초기 관리자(admin) 비밀번호. 8 자 이상, 대문자·숫자·특수문자 포함. 설치 마지막 단계에서 기본 비밀번호를 이

설정 항목	기본값	설명
		값으로 교체

오브젝트 저장 경로는 설치 스크립트가 앱 컨테이너 사용자(uid 999) 소유, 권한 0770 으로 만듭니다. 설치 뒤에 compose/.env 의 STORAGE_ROOT 를 다른 경로로 바꾸면 새 경로에도 같은 소유자와 권한을 주어야 합니다. 빠뜨려도 서비스는 정상으로 뜨지만 첫 Job 업로드에서 실패합니다. 이미 쓰고 있는 포트를 입력하면 경고가 표시되므로 다른 포트를 고르십시오.

설치가 끝나면 완료 화면에 접속 URL(http://<서버 IP>:<Frontend 포트>), 오브젝트 저장소 경로, 관리자 계정(ID admin 과 설정 단계에서 정한 비밀번호)과 설정 파일 위치가 표시됩니다.

> **주의** compose/.env 에는 DB·Redis 비밀번호, JWT 시크릿, 암호화 키가 평문으로 들어 있습니다. 권한(600)을 그대로 두고, 서버 밖 안전한 곳에 백업하십시오. 암호화 키를 잃으면 암호화해 저장한 비밀 값(공통 설정의 비밀 속성, SMTP 비밀번호 등)을 되살릴 수 없습니다.

12.3 설치 확인과 초기 로그인

설치 폴더의 compose 폴더에서 모든 서비스가 정상인지 확인합니다. STATUS 가 모두 Up (healthy)이면 정상입니다.

```
cd neoflow-1.0.0-offline/compose

docker compose ps                                                  # 싱글 인스턴스

docker compose -f docker-compose.yml -f docker-compose.ha.yml ps   # HA
```

싱글 인스턴스는 앱의 상태 확인 주소로도 기동 여부를 볼 수 있습니다. 응답의 status 값이 UP 이면 앱이 요청을 받을 준비가 된 것입니다. 로그인하지 않은 요청에는 전체 상태만, 로그인한 사용자에게는 구성요소(데이터베이스·Redis 등)별 상태까지만 보이며 세부 정보는 제공하지 않습니다. 그 밖의 운영 지표(/actuator/metrics·/actuator/prometheus)는 ADMIN 만 조회할 수 있습니다. HA 구성은 앱 포트를 호스트에 열지 않으므로 위의 컨테이너 상태로 확인합니다.

```
curl http://localhost:8080/actuator/health
```

브라우저로 완료 화면의 접속 URL 에 들어가 로그인 화면이 뜨는지 확인합니다. 설치 스크립트로 설치했다면 완료 화면에 표시된 관리자 비밀번호로 바로 로그인됩니다. 설치 스크립트를 쓰지 않는 구성(Kubernetes·OpenShift, 수동 구성)에서는 초기 관리자 계정(admin)에 기본 비밀번호(admin123)로 처음 로그인하면 '비밀번호 변경 필요' 화면이 뜨며, 비밀번호를 바꾸기 전에는 다른 화면을 쓸 수 없습니다(4.2 참조). 이어서 필요한 사용자를 [사용자 관리]에서 만들고(11.1), 라이선스를 등록합니다(11.4). 문제가 있으면 아래 명령으로 앱 로그를 확인합니다.

```
docker compose logs -f neoflow-app      # HA 는 neoflow-app-1, neoflow-app-2
```

12.4 부팅 로그의 보안 경고

앱은 운영 프로파일(prod)로 기동할 때 비밀 설정값을 검사하고 결과를 앱 로그에 남깁니다. 로그에는 변수 이름만 나오고 값은 찍히지 않습니다.

앱 로그	의미	조치
ERROR 후 기동 중단: Refusing to start: production secrets not configured — …	JWT_SECRET 또는 NEOFLOW_ENCRYPTION_KEY 가 비었거나 개발용 기본값(또는 JWT 키가 64 바이트 미만)	compose/.env 에 고유한 값을 넣고 다시 기동. install.sh 로 설치했다면 자동 생성되므로 나타나지 않음
WARN: DB_PASSWORD, REDIS_PASSWORD are the built-in default password — set a unique one on the service and here.	DB·Redis(S3 호환 저장소를 쓰면 MINIO_SECRET_KEY 포함) 비밀번호가 제품 기본값. 기동과 동작에는 영향 없음	해당 서비스의 비밀번호를 고유한 값으로 바꾸고 같은 값을 .env 에 적은 뒤 앱을 다시 기동
WARN: NEOFLOW_ENCRYPTION_SALT is not set: secrets at rest use the built-in salt. …	비밀 값 암호화에 제품 내장 솔트를 쓰는 중. 설치 스크립트로 새로 설치하면 고유 솔트가 자동으로 기록되므로 나타나지 않음	설치 스크립트를 쓰지 않는 새 설치라면 첫 기동 전에 고유한 값(openssl rand -hex 16 출력)을 .env 의 NEOFLOW_ENCRYPTION_SALT 에 넣음. 이미 내장 솔트로 운영 중인 설치는 저장된 비밀 값이 그 솔트로 암호화되어 있고 다시 암호화하는 도구가 없으므로 그대로 둠 (upgrade.sh 도 솔트·암호화 키를 추가하거나 바꾸지 않음)
ERROR 후 기동 중단: NEOFLOW_ENCRYPTION_SALT must be an even number of hex characters, …	솔트 값이 16 진수 형식이 아님	openssl rand -hex 16 출력처럼 짝수 길이의 16 진수로 고친 뒤 다시 기동

기본 비밀번호 경고는 해당하는 변수만 나열하며(하나면 is, 여럿이면 are), 공개된 시험용 비밀번호로 구성한 시험 환경에서도 나타납니다. 운영 환경에서는 경고가 사라질 때까지 비밀번호를 바꾸는 것을 권장합니다.

12.5 업그레이드

업그레이드는 새 패키지의 압축을 푼 뒤, 현재 설치 폴더에 있는 upgrade.sh 에 새 패키지 폴더 경로를 넘겨 실행합니다. 업그레이드 전에는 반드시 데이터베이스를 덤프해 두십시오. 데이터베이스 스키마는 새 버전 앱이 기동할 때 자동으로 바뀌며 되돌리는 기능이 없으므로, 이전 버전으로 돌아가려면 이 덤프가 필요합니다.

1. 사전 백업 — 현재 설치의 compose 폴더에서 DB 를 덤프하고, 오브젝트 저장소와 .env 도 함께 백업합니다(12.7 참조).
2. 새 패키지를 서버에 옮겨 압축을 풉니다. 예를 들어 새 버전이 1.1.0 이면 tar xzf neoflow-1.1.0-offline.tar.gz 로 neoflow-1.1.0-offline 폴더가 만들어집니다.
3. 새 패키지 폴더에서 sha256sum -c SHA256SUMS 를 실행해 모든 줄이 OK 인지 확인합니다. 현재 설치본의 upgrade.sh 에 무결성 확인 기능이 있으면 다음 단계에서 같은 확인을 자동으로 하지만, 이전 설치본의 upgrade.sh 에는 이 기능이 없으므로 직접 확인합니다.
4. 현재 설치 폴더에서 upgrade.sh 를 실행합니다. 새 패키지의 SHA256SUMS 로 무결성을 먼저 확인한 뒤, 현재 버전과 새 버전을 보여 주고 yes 를 입력해야 진행합니다.
5. 완료 화면을 확인한 뒤 docker compose ps 로 모든 서비스가 healthy 인지, docker compose logs neoflow-app 으로 스키마 변경과 기동이 끝났는지 확인합니다.

```
cd neoflow-1.0.0-offline/compose

docker compose exec -T postgres pg_dump -U neoflow neoflow | gzip >
~/neoflow-before-upgrade.sql.gz

cd .. && sudo ./upgrade.sh /opt/pkg/neoflow-1.1.0-offline    # 새 패키지를
/opt/pkg 아래에 푼 경우
```

upgrade.sh 는 compose/.env 를 같은 폴더에 .env.backup.날짜시각 (예: .env.backup.20260929140000) 이름으로 복사한 뒤, 새 버전에 추가된 설정 항목이 있으면 값을 물어 .env 끝에 추가합니다. 이어서 서비스를 모두 멈추고(데이터 볼륨과 오브젝트 저장 경로는 그대로), 새 Docker 이미지를 읽어 들이고, compose·config 파일과 VERSION 을 새 패키지 것으로 바꾼 다음 서비스를 다시 기동해 최대 약 2 분 동안 상태를 확인합니다. HA 구성도 같은 명령으로 올리며, 모든 앱 인스턴스를 함께 내렸다가 올리므로 그동안 서비스가 중단됩니다.

> **주의** 업그레이드가 실패해 이전 버전으로 되돌릴 때는 이전 패키지 폴더의 images/app-images.tar 를 docker load -i 로 다시 읽어 들이고, 이전 패키지의 compose 파일과 업그레이드 때 만들어진 .env 백업을 compose 폴더에 되돌린 뒤 기동합니다. 새 버전이 이미 데이터베이스 스키마를 바꿨다면 사전 덤프로 데이터베이스도 복원해야 합니다(12.7 복원 순서).

12.6 삭제

설치 폴더에서 uninstall.sh 를 실행합니다. 스크립트는 세 번 묻고, yes 라고 답한 항목만 지웁니다. HA 모드로 설치했는지는 스크립트가 알아서 판단합니다.

```
cd neoflow-1.0.0-offline && sudo ./uninstall.sh
```

질문	yes 라고 답하면
NeoFlow 를 제거합니다. 계속하시겠습니까?	모든 NeoFlow 컨테이너와 네트워크를 멈추고 지웁니다. yes 가 아니면 아무것도 하지 않고 끝남
데이터 볼륨(DB·캐시)도 삭제하시겠습니까?	PostgreSQL 데이터·Redis 데이터·작업 폴더 볼륨을 지웁니다. 이어서 지우지 않고 남긴 오브젝트 저장 경로를 알려 줌
Docker 이미지도 삭제하시겠습니까?	NeoFlow 앱·웹 화면·LB 이미지와 PostgreSQL·Redis 이미지를 지웁니다

uninstall.sh 는 다음을 지우지 않습니다. 오브젝트 저장 경로(STORAGE_ROOT)의 Job 패키지와 공유 라이브러리는 Docker 볼륨이 아니라 호스트 폴더라서 남으며, 데이터 볼륨을 지우면 "업로드한 Job 파일과 NEO_LIBS 는 삭제되지 않았습니다."라는 안내와 함께 그 경로가 표시됩니다. 설치 폴더(compose/.env, .env 백업, config/license 의 라이선스 파일 포함)도 남습니다. 서버에서 NeoFlow 를 완전히 없애려면 필요한 백업을 받은 뒤 이 두 곳을 직접 지웁니다.

> **주의** 데이터 볼륨 삭제는 되돌릴 수 없습니다. 삭제하기 전에 데이터베이스와 오브젝트 저장소를 백업하십시오 (12.7). 오브젝트 저장 경로를 설치 폴더 안으로 지정했다면 설치 폴더를 지울 때 Job 패키지도 함께 사라집니다.

12.7 백업 및 복구

NeoFlow 의 데이터는 세 곳에 나뉘어 있습니다. 데이터베이스만 복원하면 Job 목록은 보이지만 실행할 패키지 파일이 없으므로, 데이터베이스와 오브젝트 저장소는 같은 시각에 함께 백업합니다. Redis 에는 만료되는 데이터(토큰 무효화 목록·로그인 잠금)만 있어 백업하지 않습니다.

대상	들어 있는 것	백업 방법
PostgreSQL (neoflow DB)	Job·DAG·스케줄 정의, 실행 이력과 실행 로그, 사용자·권한, 감사 로그·로그인 기록, 설정	pg_dump
오브젝트 저장소	Job 패키지(.zip)와 공유 라이브러리(NEO_LIBS)	filesystem 이면 STORAGE_ROOT 폴더를 tar, S3 호환 저장소면 mc mirror
설정 파일	compose/.env(비밀번호·JWT 시크릿·암호화 키), config/license/neoflow.lic	파일 복사 (변경할 때마다)

아래 명령은 설치 폴더의 compose 폴더에서 실행합니다. 백업 파일은 서버 밖 저장소로 옮겨 보관하고, 매일 같은 시각에 cron 으로 데이터베이스와 저장소를 함께 백업하는 것을 권장합니다.

```
docker compose exec -T postgres pg_dump -U neoflow neoflow | gzip >
/backup/neoflow-$(date +%Y%m%d-%H%M).sql.gz

grep '^STORAGE_ROOT=' .env        # 저장 경로 확인 (기본 /var/lib/neoflow/storage)

sudo tar czf /backup/storage-$(date +%Y%m%d-%H%M).tar.gz -C
/var/lib/neoflow/storage .

sudo cp .env /backup/env-$(date +%Y%m%d).backup && sudo chmod 600
/backup/env-*.backup
```

오브젝트 저장소로 S3 호환 저장소(STORAGE_TYPE=s3)를 쓰는 설치는 저장소 자체의 버전 관리·복제 기능으로 백업하거나, MinIO Client(mc)로 버킷(MINIO_BUCKET, 기본 neoflow-jobs)을 다른 곳에 복사합니다. 접속 정보는 .env 의 MINIO_* 값을 씁니다.

```
set -a; . ./.env; set +a

mc alias set src "$MINIO_ENDPOINT" "$MINIO_ACCESS_KEY" "$MINIO_SECRET_KEY"

mc mirror --overwrite src/neoflow-jobs /backup/neoflow-jobs
```

복원은 다음 순서로 합니다. 데이터베이스와 오브젝트 저장소는 같은 시각의 백업을 씁니다.

1. 앱을 멈춥니다: docker compose stop neoflow-app (HA 는 neoflow-app-1 neoflow-app-2). 가동 중에 복원하면 데이터가 어긋납니다.
2. 설정을 맞춥니다. 같은 서버라면 백업한 .env 를 되돌리고, 새 서버라면 같은 버전으로 설치한 뒤 백업한 .env 의 JWT_SECRET·NEOFLOW_ENCRYPTION_KEY(설정했다면 NEOFLOW_ENCRYPTION_SALT 도)를 새 .env 에 옮겨 적고 STORAGE_ROOT 를 확인합니다. 암호화 키가 다르면 암호화해 둔 비밀 값을 읽지 못합니다.
3. 데이터베이스를 비우고 덤프를 넣습니다(아래 명령).

4. 오브젝트 저장소를 되돌립니다. filesystem 이면 tar 를 STORAGE_ROOT 에 풀고 소유자를 uid 999 로 맞추고, S3 호환 저장소면 mc mirror 를 반대 방향(백업 → 버킷)으로 실행합니다.
5. 앱을 기동합니다(docker compose up -d). 관리자 로그인, Job·DAG·실행 이력 표시, 백업 전에 등록한 Job 1 개 실행과 Job 상세의 파일 다운로드를 확인합니다. 백업 이후에 올린 Job 패키지는 백업에 없으므로 다시 업로드합니다.

```
docker compose exec -T postgres psql -U neoflow -d neoflow -c "DROP SCHEMA
public CASCADE; CREATE SCHEMA public;"

gunzip < /backup/neoflow-20260929-0200.sql.gz | docker compose exec -T
postgres psql -U neoflow neoflow

sudo tar xzf /backup/storage-20260929-0200.tar.gz -C
/var/lib/neoflow/storage && sudo chown -R 999:999 /var/lib/neoflow/storage
```

서버 여러 대로 나눈 구성은 데이터베이스와 저장소가 있는 노드에서 백업합니다. 아래는 PostgreSQL·Redis·MinIO 를 공유 데이터 노드 한 대에 compose 로 두고, 앱 노드들이 S3 방식(STORAGE_TYPE=s3, 버킷 neoflow-jobs)으로 MinIO 를 쓰는 구성의 예입니다. 공유 데이터 노드의 compose 폴더에서 서비스 이름 postgres 와 minio-init(MinIO Client 이미지)을 써서 백업하며, MinIO 접속 계정은 같은 폴더의 minio.env(MINIO_ROOT_USER·MINIO_ROOT_PASSWORD)에서 읽습니다.

```
mkdir -p /data/backup

docker compose exec -T postgres pg_dump -U neoflow neoflow | gzip >
/data/backup/neoflow-$(date +%Y%m%d-%H%M).sql.gz

docker compose run --rm -v /data/backup:/backup --entrypoint sh minio-init -
c 'mc alias set nf http://minio:9000 "$MINIO_ROOT_USER"
"$MINIO_ROOT_PASSWORD" && mc mirror --overwrite nf/neoflow-jobs
/backup/neoflow-jobs'
```

이 구성에서 복원할 때는 모든 앱 노드의 compose 폴더에서 docker compose stop neoflow-app 으로 앱을 먼저 멈춘 뒤, 공유 데이터 노드에서 위 복원 순서의 데이터베이스 명령을 실행하고 MinIO 는 위의 docker compose run 명령에서 mc mirror 방향만 바꿔(mc mirror --overwrite /backup/neoflow-jobs nf/neoflow-jobs) 되돌린 다음 모든 앱 노드를 docker compose up -d 로 기동합니다.

12.8 고가용성(HA) 운영

HA 구성은 앱 인스턴스 2 대를 Active-Active 로 두고, 앞단 로드밸런서(Nginx)가 요청을 나눠 보내며 응답하지 않는 인스턴스는 자동으로 제외합니다. 두 인스턴스는 같은 PostgreSQL·Redis 와 같은 오브젝트 저장소(모든 노드가 같은 공유 마운트, 또는 같은 S3 호환 버킷)를 써야 하며, JWT 시크릿·암호화 키·솔트도 같은 값이어야 합니다. 설치 스크립트의 HA 모드는 서버 한 대에 앱 2 개와 LB 를 띄우고, 서버 여러 대로 나누는 구성(예: LB 1 대, 앱 2 대, 공유 데이터 1 대)도 같은 원리로 동작합니다.

항목	HA 에서의 동작
로그인 유지	로그인 토큰(JWT)은 서버에 세션을 두지 않으므로, 요청이 다른 인스턴스로 가거나 한 인스턴스가 멈춰도 다시 로그인할 필요가 없습니다. 로그아웃 등으로 끝난 토큰 목록은 Redis 에 있어 두 인스턴스가 함께 봅니다.
스케줄 실행	두 인스턴스가 모두 1 분마다 예약을 확인하지만, DB 에서 그 예약분을 먼저 선점한 인스턴스만 실행하므로 같은 예약이 두 번 실행되지 않습니다(7 장 참조).

항목	HA 에서의 동작
실행 큐	대기(QUEUED) 실행은 DB 에 있고, 빈 실행 슬롯이 있는 인스턴스가 하나씩 가져가 실행합니다. 동시 실행 한도는 인스턴스마다 적용됩니다(MAX_CONCURRENT_JOBS, 기본 35).
인스턴스 장애	각 인스턴스는 10 초마다 DB 에 생존 신호(하트비트)를 남깁니다. 신호가 60 초 넘게 끊기면 살아 있는 인스턴스가 1 분마다 하는 점검에서, 멈춘 인스턴스가 실행 중(일시정지 포함)이던 실행을 FAILED 로 정리합니다(오류 메시지 "Instance <host:port> stopped while this execution was in progress"). Job 프로세스는 그 인스턴스와 함께 끝났으므로 재시도 설정과 관계없이 자동으로 다시 실행되지 않으며, 필요하면 실시간 실행 화면의 재시도(↻)나 Job 에서 다시 실행합니다. 아직 시작되지 않은 대기 실행은 살아 있는 인스턴스가 이어받습니다.
실시간 알림	실시간 토스트와 실행 상태·로그의 실시간 전달은 브라우저가 연결된 인스턴스에서 일어난 일만 즉시 전달됩니다. 다른 인스턴스에서 실행된 Job 의 토스트는 뜨지 않을 수 있지만, 알림 목록과 알림 벨 배지(30 초마다 다시 셈), 실행 상세(3 초마다 다시 조회)에는 빠짐없이 반영됩니다.
DAG 실행 화면	DAG 실행 상세와 DAG 상세의 실행 그래프는 실시간 연결 중에도 10 초마다 서버에서 상태를 다시 읽습니다. 다른 인스턴스가 진행하는 DAG 실행은 화면에 최대 약 10 초 늦게 반영됩니다.
실행 취소	다른 인스턴스가 실행 중인 Job 을 취소하면 그 인스턴스가 5 초 안에 요청을 받아 프로세스를 종료합니다(10.2 참조).
실행 인스턴스 표시	실행 이력의 '실행 인스턴스'(host:port)로 어느 인스턴스가 실행했는지 확인합니다. 노드마다 호스트 이름이 달라야 구분됩니다.

참고 한 인스턴스를 점검하려고 내릴 때는 그 인스턴스에서 실행 중인 Job 이 끝난 뒤에 내리십시오. 정상 종료(docker compose stop 등)하면 그 인스턴스에서 실행 중이던 Job 은 취소(CANCELLED, 사유 Application shutdown)로 끝나고, 비정상 종료하면 위의 인스턴스 장애처럼 FAILED 로 정리됩니다. 어느 쪽이든 자동으로 다시 실행되지 않습니다.

13. 부록

13.1 키보드 단축키

NeoFlow 의 1 차 키보드 진입점은 명령 팔레트입니다. 어디서든 아래 단축키로 페이지 이동과 Job/DAG 검색을 키보드만으로 처리할 수 있습니다.

키	동작
Ctrl + K / Cmd + K	명령 팔레트 열기 / 닫기
↑ / ↓	팔레트 결과 이동
Enter	팔레트에서 선택
Esc	팔레트 · 모달 닫기

명령 팔레트를 여는 단축키 외에 화면 전체에서 동작하는 단축키는 없습니다. 데이터를 다시 불러오려면 각 화면의 [새로고침] 버튼이나 브라우저 새로고침을 씁니다.

13.2 Cron 치트시트

스케줄 편집 화면에는 5 필드(분 시 일 월 요일)를 입력하고, 저장값과 REST API 는 앞에 초가 붙은 6 필드(초 분 시 일 월 요일)를 씁니다. API 에 5 필드나 연도가 붙은 7 필드를 보내면 400 으로 거부됩니다. 식은 Spring CronExpression 으로 해석하며, 요일 번호는 Linux cron 과 같이 1 = 월요일입니다(1 = 일요일로 세는 Quartz 와 다름). 헷갈리지 않도록 MON·FRI 처럼 이름을 쓰는 편이 안전합니다.

화면 입력 (5 필드)	저장값 · API (6 필드)	의미
*/5 * * * *	0 */5 * * * *	5 분마다
0 * * * *	0 0 * * * *	매시 정각
0 2 * * *	0 0 2 * * *	매일 02:00
0 9 * * MON-FRI	0 0 9 * * MON-FRI	평일 09:00
0 9 * * MON	0 0 9 * * MON	매주 월요일 09:00
0 0 1 * *	0 0 0 1 * *	매월 1 일 00:00
0 0 L * *	0 0 0 L * *	매월 마지막 날 00:00
0 9 * * FRI#3	0 0 9 * * FRI#3	매월 셋째 금요일 09:00

cron 외에 @hourly(매시 정각)·@daily·@midnight(매일 00:00)·@weekly(매주 일요일 00:00)·@monthly(매월 1 일 00:00)·@yearly·@annually(매년 1 월 1 일 00:00) 매크로와 PT 기간식(PT5M, PT30M, PT1H 등 — 매일 00:00 부터 그 간격, 하루는 PT24H)도 받습니다. 간격은 1 분 이상이어야 합니다.

> **주의** 일과 요일을 둘 다 지정하면 두 조건을 모두 만족하는 날에만 실행됩니다(둘 중 하나만 맞아도 실행하는 Linux cron 과 다름). 예를 들어 0 9 1 * MON 은 1 일이면서 월요일인 날에만 09:00 에 실행되므로, 일과 요일 중 하나는 *로 두세요. ?는 *와 같은 뜻입니다. 서버는 1 분마다 실행 대상을 확인하므로 1 분보다 잦은 cron 도 1 분에 한 번만 실행됩니다.

13.3 HTTP 상태 코드

NeoFlow 는 별도 에러 코드 체계 없이 HTTP 상태 코드로 분기하고, 상세 사유는 응답 본문의 message 필드로 전달합니다.

코드	의미	발생
200 / 201	OK / Created	성공 / 리소스 생성
202	Accepted	Job·DAG 실행 큐 등록
400	Bad Request	입력 검증 실패, 비즈니스 규칙 위반
401	Unauthorized	토큰 누락/만료, 로그인 실패, 비활성·정지된 사용자의 API 토큰
402	Payment Required	라이선스 미충족 (기능 미포함, 동시 실행 한도 초과 등)
403	Forbidden	권한 부족, 승인 대기 계정의 로그인 시도, 비밀번호 변경 전 요청
409	Conflict	중복, 또는 지금 상태로 할 수 없는 요청(비활성 Job 실행, 끝난 실행 취소, 마지막 ADMIN 변경 등)
413	Payload Too Large	업로드 500MB 초과
423	Locked	로그인 brute-force 잠금
429	Too Many Requests	Rate limit 초과

13.4 오류 메시지와 조치

화면에 자주 나타나는 오류·안내 메시지와 조치 방법입니다. 메시지는 화면 언어가 한국어일 때의 문구이며, 서버가 보낸 사유를 그대로 보여 주는 경우는 영문으로 표시될 수 있습니다.

상황	화면 메시지	조치
로그인 실패 (401)	“로그인에 실패했습니다. 다시 시도해주세요.”	사용자명(이메일 아님)과 비밀번호를 확인합니다. 계속 틀리면 계정이 잠깁니다(4.1)
계정 잠금 (423)	“로그인 실패가 너무 많아 계정이 일시적으로 잠겼습니다. 잠시 후 다시 시도해주세요.”	잠금 시간(기본 15 분)이 지난 뒤 다시 로그인합니다. 급하면 관리자가 [사용자 관리]에서 비밀번호를 재설정해 잠금을 풉니다
승인 대기 (403)	“관리자 승인 대기 중인 계정입니다. 승인 후 로그인할 수 있습니다.”	관리자가 [사용자 관리]에서 [승인]을 누릅니다(4.3, 11.1)
정지된 계정 (403)	“정지된 계정입니다. 관리자에게 문의하세요.”	관리자가 [사용자 관리]에서 상태를 활성으로 바꿉니다(11.1)
회원가입 중복 (409)	“이미 사용 중인 사용자 이름 또는 이메일입니다.”	다른 사용자명·이메일로 신청합니다
비밀번호 변경 필요	화면 제목 ‘비밀번호 변경 필요’, “초기 비밀번호(설치 기본값 또는 관리자가 발급한 비밀번호)로 로그인했습니다. 계속하려면 새 비밀번호로 변경하세요.”	설치 기본 비밀번호의 첫 로그인이거나, 관리자가 만들거나 재설정한 계정의 첫 로그인입니다. 방금 로그인한 비밀번호와 새 비밀번호를 입력합니다(4.2)
현재 비밀번호 오류	“현재 비밀번호가 올바르지 않습니다”	방금 로그인한(또는 지금 쓰는) 비밀번호를 정확히 입력합니다

상황	화면 메시지	조치
비밀번호 재사용	“최근에 사용한 비밀번호는 다시 사용할 수 없습니다.” / 현재와 같으면 “현재 비밀번호와 다른 비밀번호를 입력해주세요”	현재 비밀번호를 포함한 최근 3 개와 다른 비밀번호를 정합니다
비밀번호 변경 실패	“비밀번호 변경에 실패했습니다. 현재 비밀번호를 확인하세요.”	현재 비밀번호와 정책(8 자 이상, 대문자·숫자·특수문자 각 1 개 이상)을 확인합니다
로그인 만료 (401)	메시지 없이 로그인 화면으로 이동	다시 로그인합니다. 로그인 유지 시간은 [설정 > 보안]의 세션 유지 시간입니다(4.1)
권한 부족 (403)	“권한이 없습니다. 관리자에게 문의하세요.” (토스트)	역할로 할 수 없는 작업입니다(3 장). 필요하면 관리자가 역할을 올립니다
업로드 형식 오류	“<파일명>: ZIP 파일만 지원됩니다”	NeoFlow Studio 에서 .zip 으로 export 한 파일을 고릅니다(6.2)
Job 파일 무결성 오류	실행 상세 오류: “Job file integrity check failed: the stored file does not match the SHA-256 recorded at upload (…). Upload the Job file again.” (서버 원문)	저장소의 Job 파일이 업로드 때와 달라진 것입니다. Job 파일을 다시 업로드한 뒤 실행합니다(6.2)
설치 패키지 무결성 오류	“패키지 무결성 검증 실패: 위 파일이 SHA256SUMS 와 다릅니다. 패키지를 다시 받으세요.” (install.sh·upgrade.sh 출력)	패키지를 다시 받아 서버로 옮긴 뒤 설치·업그레이드합니다(12.2, 12.5)
업로드 크기 초과 (413)	“<파일명>: 파일 크기 초과” (API 로 올리면 HTTP 413)	파일당 500MB 이하로 나눕니다
라이선스 차단 (402)	“라이선스가 유효하지 않습니다 — 쓰기 작업이 차단되었습니다. 관리자에게 문의하세요.” (배너·토스트)	관리자가 유효한 .lic 파일을 업로드합니다(11.4 라이선스)
라이선스 유예·만료 임박	“라이선스가 만료되어 유예 기간 중입니다. 곧 갱신해 주세요.” / “라이선스가 N 일 후 만료됩니다.”	머신 지문을 전달해 갱신 라이선스를 받아 업로드합니다
요청 한도 초과 (429)	Rate limit exceeded. Try again in N seconds. (서버 응답)	안내된 시간 뒤 다시 시도합니다. 관리자는 [설정 > 보안]의 Rate limit 값을 확인합니다
테스트 메일 실패 — 시간 초과	The SMTP server did not respond in time. Check the host, the port and the SSL/TLS setting (port 465 needs SSL/TLS on), … (서버 원문)	호스트·포트를 확인하고, 465 면 [SSL/TLS]를 켭니다. 서버가 그 포트로 외부 연결을 할 수 있는지(방화벽) 확인합니다
테스트 메일 실패 — 인증	The SMTP server rejected the username or password. (서버 원문)	SMTP 계정·비밀번호를 확인합니다. Gmail 은 앱 비밀번호를 씁니다
DAG 저장 실패	“DAG 저장에 실패했습니다: <사유>”	[검증]으로 오류 항목을 확인해 고친 뒤 저장합니다(8.2)

13.5 용어집

용어	설명
NeoFlow Studio	ETL Job 을 설계·빌드하는 작성 도구. 여기서 export 한 .zip 을 NeoFlow 에 업로드·

용어	설명
	동기화
NeoFlowContext	PARAM_* 환경변수를 같은 이름의 context 변수로 매핑하는 NeoFlow Studio routine
Job	NeoFlow Studio .zip 한 개로 등록되는 실행 단위
Schedule	Job 을 자동 실행시키는 cron 규칙 (화면 5 필드, 저장값·API 6 필드)
Execution	Job/Task 의 1 회 실행 인스턴스
TIMEOUT	Job 타임아웃(기본 3600 초)을 넘겨 강제 종료된 실행 상태. 실패 알림과 같은 종류로 알림
승인 대기	회원가입으로 만들어져 아직 관리자가 승인하지 않은 INACTIVE 계정. 승인 전에는 로그인 불가
실행 인스턴스	해당 실행을 집행한 앱 인스턴스 식별자 host:port (워커 노드 클러스터가 아님)
실행 큐	앱 인스턴스 내부의 동시 실행 제어 큐. 슬롯이 가득 차면 QUEUED 로 대기
오브젝트 저장소	Job 패키지(.zip)와 공유 라이브러리(jar)를 보관하는 곳. 배포판 기본은 앱 서버의 파일 경로(STORAGE_ROOT, HA 면 노드 간 공유 마운트), S3 호환 스토리지는 옵션. 실행 로그는 여기가 아니라 DB 에 저장
DAG	여러 Job/Task 를 의존성으로 묶은 워크플로우
DAG Run	1 회 DAG 실행 인스턴스 (여러 Task Execution 포함)
Trigger Rule	부모 Task 종료 상태에 따른 자식 실행 조건. 모두 성공(ALL_SUCCESS) 등 7 종 (8.2 참조)
SLA	목표 소요시간 약속. 실행을 요청할 때의 목표로 판정하며, 위반은 대시보드·SLA 위반 화면·주간 리포트에 집계
BizDate	증분 적재 Job 의 업무 기준일 워터마크
Load Type	Job 적재 유형 메타: NONE(기본)/INITIAL/INCREMENTAL/BACKFILL
RBAC	역할 기반 권한 제어 (1 사용자 = 1 역할)
API Token	외부/CI 호출용 인증 토큰 (각 사용자가 본인 토큰 발급, 소유자의 현재 역할로 동작)
Webhook	이벤트를 Slack/HTTP 로 push 하는 외부 연동
NEO_LIBS	모든 Job 이 함께 쓰는 공용 라이브러리(.jar·.so) 풀. 설정 > 공유 라이브러리 탭에서 관리
라이선스 (.lic)	전자서명된 라이선스 파일. 에디션·만료일·용량 한도·기능을 담으며 설정 > 라이선스 탭에서 업로드
HA (Active-Active)	앱 인스턴스 2 대 이상을 로드밸런서 뒤에 두고 함께 요청을 처리하는 구성 (12.8 참조)